M04 — Integración de APIs y seguridad básica

Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: TEORÍA

AUTOR: JAIME GALLO

Índice del módulo

- 4.1 — Integración HTTP con Blazor
- 4.2 — Servicios tipados e inyección de dependencias
- 4.3 — Construcción de una API REST
- 4.4 — Autenticación, autorización, Identity y JWT (ampliación)

Orientación de lectura

Este manual desarrolla los fundamentos y los ejemplos completos **antes** de los ejercicios. La secuencia esencial es comprender GET/POST y los DTO; extraer la comunicación a un servicio; construir una API compatible; y diferenciar autenticación, autorización y visualización de permisos. Los estudios de caso y las profundizaciones permiten continuar aprendiendo fuera de la sesión, sin obligar a ejecutar todos los experimentos durante las seis horas del módulo.

Los fragmentos de código se presentan para explicar el mecanismo. El itinerario de construcción, sus estados intermedios y los resultados que debe observar el alumno están en el manual de prácticas.

4.1. Consumir una API HTTP desde Blazor: contratos, JSON, operaciones asíncronas y errores

Objetivo y conexión con el módulo anterior

En M03 hemos construido una aplicación Blazor Web App sobre .NET 8 en modo Interactive Server. La página de productos administra una colección de objetos `Producto` y conoce cómo dibujar tarjetas, filtrar, seleccionar y actualizar información. En el último ejercicio, el catálogo podía obtenerse desde `IProductosMemoriaService`: un servicio dentro del mismo proceso. Ahora cambia la procedencia de los datos, no la responsabilidad de la interfaz.

En este capítulo el origen pasa a ser una **API HTTP auxiliar**. Blazor será cliente: enviará peticiones y leerá respuestas. La API será servidor: recibirá peticiones en direcciones como `/api/productos` y devolverá respuestas JSON. Ambos procesos podrán ejecutarse en la misma máquina, pero son aplicaciones distintas.

Al terminar entenderemos el camino completo, no solo la llamada a un método: componente → petición HTTP → endpoint → respuesta JSON → deserialización → estado de la interfaz. En la práctica P4.1 activaremos ese circuito conservando un punto de retorno claro al catálogo en memoria de M03.

4.1.1. ¿Qué cambia entre un servicio local y uno remoto?

Con `IProductosMemoriaService`, un componente llama a un método que ejecuta código del mismo servidor Blazor. Con una API, solicita información a otro servidor HTTP. El contrato visible no es una interfaz C# compartida, sino el método HTTP, la ruta, el cuerpo, el código de estado y el JSON intercambiado.

Servicio en memoria de M03	API HTTP en M04
Una llamada C# dentro del mismo proceso	Petición y respuesta HTTP entre procesos
Devuelve objetos .NET directamente	Devuelve bytes, generalmente JSON
Puede fallar por reglas internas	También por conexión, tiempo de espera o estados HTTP
No tiene rutas ni verbos HTTP	Usa rutas y métodos GET/POST
Su modelo puede cambiar junto con Blazor	Necesita un contrato de intercambio estable

Esta diferencia explica por qué **no debemos insertar llamadas HTTP en todos los componentes**. En P4.1 veremos primero el mecanismo; en 4.2 lo encapsularemos en un servicio específico que sea fácil de sustituir y probar.

Laboratorio conceptual: interpretar HTTP antes de escribir Razor

En el catálogo local del módulo anterior una llamada C# permanecía dentro del mismo proceso. La integración remota introduce un protocolo y, por tanto, un conjunto de situaciones que antes no aparecían: el host podría no estar iniciado, la ruta podría ser incorrecta, el servidor podría rechazar

una entrada o la respuesta podría ser válida como JSON pero incompatible con el contrato del cliente.

Consideremos un usuario que visita la página /catalogo-remoto. La navegación lleva a un componente de Blazor Interactive Server. Su código C# se ejecuta en el servidor que aloja Blazor, no automáticamente dentro del proceso de la API ni dentro del JavaScript del navegador. Desde ese proceso se emite la petición HTTP a otro servidor, la API auxiliar del laboratorio. Ese servidor busca un endpoint, lee sus productos y devuelve JSON. La aplicación Blazor deserializa la respuesta y actualiza el estado del componente, y el navegador recibe los cambios de su interfaz.

Ese recorrido explica por qué tres direcciones no deben confundirse: la URL de la página Blazor, la dirección base del cliente HTTP y la ruta del endpoint de la API. La primera permite navegar hasta una pantalla; la segunda indica a qué host y puerto se envía la petición; la tercera identifica el recurso dentro de la API.

Una respuesta 200 con un arreglo vacío significa que no existen elementos disponibles, pero el protocolo ha funcionado. Una respuesta 404 implica que el servidor recibió la petición y no encontró el recurso. Una excepción por conexión rechazada significa que no obtuvimos respuesta HTTP de la API. La distinción parece elemental, pero determina mensajes de usuario, diagnósticos y decisiones de implementación muy diferentes.

4.1.2. Petición y respuesta: más allá de «llamar a una URL»

Una operación HTTP tiene un método, un destino y, opcionalmente, cabeceras y cuerpo.

```
GET /api/productos HTTP/1.1
Host: localhost:5081
Accept: application/json
```

En nuestro ejemplo, una respuesta correcta devolverá estado **200 OK** y un arreglo JSON:

```
HTTP/1.1 200 OK
Content-Type: application/json

[
  { "id": 1, "nombre": "Cuaderno", "precio": 2.50 },
  { "id": 2, "nombre": "Lápiz", "precio": 0.80 }
]
```

La dirección contiene esquema, servidor, puerto y ruta. `http://localhost:5081` representa la **dirección base de ejemplo** de la API, y `api/productos` es el recurso. Si cambiamos el puerto, la ruta no tiene por qué cambiar. El puerto del servidor API y el de Blazor deben ser distintos cuando ambas aplicaciones se ejecutan a la vez.

Una alta típica usa POST:

```
POST /api/productos HTTP/1.1
Host: localhost:5081
Content-Type: application/json

{ "nombre": "Regla", "precio": 1.75 }
```

La API valida la entrada, asigna un identificador y responde **201 Created** con el producto creado y, preferiblemente, su ubicación. GET no debe dar de alta productos; POST se emplea aquí para crear

uno. Los códigos son parte del contrato: un cuerpo JSON con formato válido no convierte en éxito una operación rechazada.

Las cabeceras HTTP también forman parte de la comunicación

Al consumir una API conviene examinar algo más que el cuerpo JSON. Una petición POST incluye Content-Type para indicar cómo debe interpretarse el contenido enviado. Cuando el cuerpo está serializado como JSON, ese valor es application/json. La respuesta del servidor puede incluir igualmente un Content-Type para describir qué datos devuelve.

La cabecera Accept permite expresar el formato que prefiere el cliente para la respuesta. Nuestro laboratorio utiliza JSON y no implementa un sistema de formatos alternativos, pero distinguir Accept de Content-Type evita una confusión frecuente: el primero se refiere a la representación que el cliente desea recibir, mientras que el segundo describe la representación del cuerpo que realmente se envía.

Consideremos este intercambio:

```
POST /api/productos HTTP/1.1
Host: localhost:5081
Accept: application/json
Content-Type: application/json

{"nombre":"Agenda","precio":6.40}
```

El servidor lee el cuerpo de acuerdo con el formato de entrada, valida los campos y devuelve el recurso creado. Si el servidor responde 201, la aplicación cliente todavía debe leer correctamente el objeto que acompaña a la respuesta. La confirmación HTTP y la conversión de datos son etapas distintas.

Cuando se produce un rechazo, el servidor puede elegir otro formato para describirlo. En la API sencilla del curso devolvemos mensajes reducidos; no implantamos un modelo corporativo de errores. Esa limitación explica por qué el cliente debe comprobar el código de estado antes de interpretar el cuerpo como ProductoDto: el contenido de un 400 no tiene por qué tener la forma de un producto creado.

4.1.3. Contratos de datos y DTO: no enviar la interfaz entera

En M03 existe este modelo:

```
namespace CursoWeb.Blazor.Models;

public sealed record Producto(int Id, string Nombre, decimal Precio);
```

Representa un producto en la aplicación Blazor. Para transportar datos usamos DTO (*Data Transfer Object*). Un DTO distingue qué recibe y qué envía la API, evitando que detalles de la interfaz se conviertan accidentalmente en contrato público.

El producto leído incluye el identificador asignado por el servidor:

```
namespace CursoWeb.Blazor.Models;

public sealed record ProductoDto(int Id, string Nombre, decimal Precio);
```

La petición de alta **no** incluye identificador, porque todavía no existe:

```
namespace CursoWeb.Blazor.Models;

public sealed record CrearProductoDto(string Nombre, decimal Precio);
```

Aunque hoy coincidan tres propiedades, `Producto` y `ProductoDto` no son el mismo concepto. La conversión hace explícito el límite:

```
Producto Convertir(ProductoDto dto) =>
    new(dto.Id, dto.Nombre, dto.Precio);
```

Un `record` posicional simplifica la representación en este laboratorio. No equivale a una entidad de EF Core ni introduce persistencia. La API del capítulo mantendrá su catálogo en memoria; reiniciar el proceso puede restaurar los datos iniciales.

El DTO como contrato, no como duplicación accidental

Los DTO del cliente son tipos que describen los datos intercambiados. El servidor puede utilizar sus propios tipos equivalentes sin referenciar el proyecto Blazor. El acoplamiento que realmente importa aparece en el contrato JSON: campos, tipos, forma de la colección y significados.

Comparemos tres documentos JSON:

```
[
  { "id": 1, "nombre": "Cuaderno", "precio": 2.50 }
]
```

```
[]
```

```
{ "cantidad": 1, "precioTotal": 2.50 }
```

Los dos primeros son arreglos JSON compatibles con una lista de ProductoDto. El tercero es un objeto válido, pero no es un catálogo. Aunque el servidor devolviera código 200, intentar deserializarlo como una lista sería un error. Por eso el endpoint de resumen de 4.3 conserva su propia ruta y no sustituye la salida del listado.

El identificador es otro aspecto del contrato. El DTO de lectura tiene Id porque la API ya ha creado el recurso. El DTO de creación lo omite porque la asignación corresponde al servidor. Si el componente asignara su propio identificador, dos clientes distintos podrían producir decisiones contradictorias. Nuestro laboratorio no modela un sistema distribuido completo, pero sí mantiene clara la frontera de responsabilidad.

Usamos decimal para precios porque representa cantidades de base decimal y evita algunas dificultades de aproximación de los números binarios de coma flotante. No estamos construyendo un motor de facturación: no se modelan monedas, impuestos, reglas de redondeo ni transacciones.

Caso de estudio: el catálogo devuelve 200, pero Blazor no presenta productos

Consideremos una incidencia deliberadamente diseñada para aprender a diagnosticar contratos. La API responde en la dirección correcta y, al consultar desde una herramienta HTTP, observamos la respuesta siguiente:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"productos":[{"id":1,"nombre":"Cuaderno","precio":2.50}]}
```

El servidor existe, la ruta coincide y el cuerpo es JSON válido. Sin embargo, el servicio Blazor intenta convertirlo directamente a una lista de ProductoDto. El problema reside en la forma del contrato: el servidor ha enviado un objeto con una propiedad productos, no un arreglo como raíz.

Hay dos correcciones posibles, pero no son equivalentes. Podemos restaurar en el servidor el arreglo esperado, manteniendo el contrato original de GET /api/productos, o podemos cambiar el cliente y definir un DTO contenedor que corresponda a la nueva forma. En nuestro laboratorio elegimos la primera opción porque ya existen consumidores que dependen del contrato original y no hemos planteado una versión nueva de la API.

Ahora imaginemos otra respuesta:

```
HTTP/1.1 200 OK
Content-Type: application/json

[{"id":1,"nombre":"Cuaderno","precio":"dos euros"}]
```

Aquí la raíz sí es un arreglo, pero el tipo de precio no corresponde a decimal. Un campo de texto no numérico no debe convertirse silenciosamente en un precio válido. La API tiene que recuperar el contrato declarado o el cliente tendría que implementar una transformación explícita que justificara esa diferencia. No corregiríamos esta incidencia cambiando el modo de renderizado de Blazor: la incompatibilidad sucede antes de representar la lista.

El procedimiento didáctico consiste en formular hipótesis y comprobarlas una por una. Primero confirmamos el estado HTTP, después inspeccionamos el cuerpo, comparamos raíz y propiedades con los DTO y finalmente verificamos que la página transforma correctamente los datos recibidos. Esta secuencia permite distinguir una incidencia de comunicación de una incidencia de serialización.

4.1.4. JSON y serialización con System.Net.Http.Json

En JSON los nombres de propiedades no llevan mayúsculas obligatorias. La configuración web habitual de `System.Text.Json` usa nombres en *camelCase*, de manera que `Nombre` se representa como `nombre`.

```
{
  "id": 3,
  "nombre": "Regla",
  "precio": 1.75
}
```

Para transformar objetos .NET en JSON o a la inversa, .NET 8 ofrece las extensiones de `System.Net.Http.Json`. `GetFromJsonAsync<T>` combina la lectura HTTP con la deserialización; `PostAsJsonAsync` serializa un objeto antes de enviarlo.

No obstante, **deserializar no es validar el negocio**: si la API devuelve un nombre vacío o un precio inválido con 200, el JSON puede ser técnicamente legible. La validación de los datos recibidos o de la petición sigue siendo una responsabilidad explícita.

El significado de una respuesta se compone de varias piezas

Una operación HTTP tiene método, ruta, cabeceras, cuerpo opcional y un resultado. Ninguna pieza aislada describe por completo lo que ha ocurrido. Un 200 puede incluir un catálogo válido, una lista vacía o un cuerpo incompatible con el DTO. Un 201 puede indicar que se ha creado un recurso, pero el consumidor aún necesita conocer cuál se ha creado y con qué identificador.

Cuando diseñamos un formulario Blazor que publica productos, debemos leer esas piezas en un orden comprensible: se construye CrearProductoDto, se envía POST con JSON, se comprueba el estado de respuesta, se lee el ProductoDto confirmado y se actualiza la pantalla. Si se rechaza la petición, el componente conserva los valores del usuario y presenta el error. Si el servidor no responde, el cliente no inventa un estado HTTP.

Esta secuencia sirve para evaluar código que aparentemente funciona. Un componente que añade el producto antes de recibir respuesta está presentando una creación que todavía no se ha confirmado. Un servicio que devuelve una lista vacía al capturar toda excepción puede esconder un servidor apagado. Una API que acepta cualquier precio porque confía en el formulario de Blazor no está validando su propio contrato.

4.1.5. HttpClient, dirección base e inyección de dependencias

`HttpClient` es el objeto mediante el cual un programa .NET envía solicitudes HTTP. Se configura a través del contenedor de servicios, en vez de crear uno nuevo por cada pulsación de botón. Para el laboratorio de M04 usaremos un cliente con nombre:

```
// Program.cs de la aplicación Blazor (ejemplo de configuración para M04)
builder.Services.AddHttpClient("ProductosApi", client =>
{
    client.BaseAddress = new Uri(
        builder.Configuration["ProductosApi:BaseUrl"]
        ?? "http://localhost:5081/");
    client.Timeout = TimeSpan.FromSeconds(10);
});
```

La dirección se puede declarar en `appsettings.json`:

```
{
  "ProductosApi": {
    "BaseUrl": "http://localhost:5081/"
  }
}
```

La barra final en `BaseAddress` evita confusiones al combinar rutas relativas. Con una ruta como `"api/productos"` (sin barra inicial), `HttpClient` construye la dirección completa usando esa base.

En Blazor Interactive Server, esta llamada se efectúa **desde el proceso servidor de Blazor**, no directamente desde JavaScript del navegador. Por tanto, un escenario HTTP servidor-a-servidor no tiene las mismas restricciones CORS que una llamada fetch desde el navegador. Aun así, un servidor API apagado o una URL equivocada impiden obtener datos.

Elegir el nivel de abstracción HTTP

La familia GetFromJsonAsync combina consulta, validación básica de respuesta y deserialización. Resulta clara para un primer GET, porque deja ver cuál es el dato que espera recibir el componente.

Si necesitamos analizar con exactitud el código HTTP antes de leer el cuerpo, podemos separar las operaciones:

```
using var respuesta = await cliente.GetAsync("api/productos");
if (!respuesta.IsSuccessStatusCode)
{
    error = $"La API respondió {(int)respuesta.StatusCode}.";
    return;
}
productos = await respuesta.Content
    .ReadFromJsonAsync<List<ProductoDto>>() ?? [];
```

El código es más largo, pero también permite distinguir los distintos rechazos del servidor. No debemos intentar leer una respuesta de error arbitraria como si necesariamente contuviera un catálogo válido. Un 400 o un 500 podrían devolver un objeto de diagnóstico, una cadena u otro contrato diferente.

En escenarios avanzados existe SendAsync junto con HttpRequestMessage, que permite configurar la solicitud de manera detallada. No necesitamos introducirlo como requisito práctico de este módulo; conocer que existe ayuda a situar GetAsync y PostAsJsonAsync dentro de la misma abstracción HTTP.

Relación entre configuración y despliegue

La dirección local usada en el laboratorio permite ejecutar dos procesos en la misma máquina. No significa que la API pública de producción deba utilizar HTTP sin cifrar ni un puerto fijo de desarrollo. En otro entorno, la dirección base será distinta y se configurará de forma apropiada. El contrato del componente no debe depender de escribir ese valor como una cadena dentro de cada pantalla.

Este módulo mantiene deliberadamente una configuración sencilla. La jornada siguiente presentará mecanismos elementales de ejecución y publicación, pero no es necesario introducir ahora balanceadores, descubrimiento de servicios ni una plataforma de contenedores para comprender la llamada HTTP.

Caso de estudio: falla solo después de cambiar el puerto

La API suministrada escucha en 5081 y el cliente usa la clave ProductosApi:BaseUrl. El docente desea demostrar que la ubicación de un servicio es configuración y arranca una nueva API en 5088. Su contrato permanece igual, pero el cliente sigue apuntando a 5081.

En esa situación no debemos modificar ProductoDto, ni PostAsJsonAsync ni el marcado HTML. Primero comprobamos con un cliente HTTP que la nueva API responde en 5088. Después cambiamos la dirección base de la aplicación Blazor, reiniciamos cuando sea necesario para aplicar la configuración y volvemos a consultar la página.

El hecho de que la API responda en otra dirección no significa que el contrato haya cambiado. Tampoco debemos suponer que una misma ruta relativa funciona sin configurar correctamente su host. Esta distinción vuelve a aparecer cuando una aplicación se ejecuta bajo Kestrel, IIS o Docker

en el módulo 5: el destino puede variar mientras que las operaciones GET y POST mantienen su significado.

Podemos plantear un experimento inverso: conservamos el puerto pero cambiamos solamente el nombre de la ruta a api/catalogo. Ahora la conectividad HTTP existe, pero la API no publica el recurso que el cliente solicita. La prueba directa devolverá un error de ruta y el componente podrá mostrarlo. El alumno debe describir qué ha cambiado: en el primer caso, la ubicación; en el segundo, el contrato de endpoint.

4.1.6. GET asíncrono y carga de productos

Una primera aproximación dentro de un componente Blazor sería:

```
@page "/catalogo-remoto"
@rendermode InteractiveServer
@inject IHttpClientFactory Clientes

<h1>Productos de la API</h1>

@if (cargando)
{
    <p role="status">Cargando catálogo...</p>
}
else if (error is not null)
{
    <p role="alert">@error</p>
}
else
{
    <ul>
        @foreach (var producto in productos)
        {
            <li>@producto.Nombre — @producto.Precio</li>
        }
    </ul>
}

@code {
    private List<ProductoDto> productos = [];
    private bool cargando = true;
    private string? error;

    protected override async Task OnInitializedAsync()
    {
        try
        {
            var cliente = Clientes.CreateClient("ProductosApi");
            productos = await cliente.GetFromJsonAsync<List<ProductoDto>>(
                "api/productos") ?? [];
        }
        catch (HttpRequestException)
        {
            error = "No se pudo obtener el catálogo remoto.";
        }
        catch (TaskCanceledException)
        {
            error = "La petición superó el tiempo disponible.";
        }
        catch (System.Text.Json.JsonException)
        {
            error = "La respuesta no contiene un catálogo válido.";
        }
```

```
        finally
        {
            cargando = false;
        }
    }
}
```

Lectura razonada del ejemplo:

- `@inject IHttpClientFactory Clientes` pide al contenedor la fábrica registrada en `Program.cs`.
- `OnInitializedAsync` se ejecuta cuando el componente se inicializa; `await` permite esperar la respuesta sin bloquear manualmente un hilo.
- `GetFromJsonAsync` puede lanzar una excepción ante un estado HTTP no satisfactorio, un problema de red o JSON inválido. No es suficiente asumir que siempre devuelve una lista.
- `?? []` trata una respuesta deserializada como `null` como lista vacía para este ejemplo. No debe confundirse con un error de comunicación: ese caso se muestra por separado.
- `finally` actualiza el indicador de carga independientemente de éxito o error.
- `role="status"` y `role="alert"` hacen los estados más claros también para tecnologías de asistencia.

El modelo `ProductoDto` debe estar importado mediante `@using CursoWeb.Blazor.Models` o en `Components/_Imports.razor`. Este ejemplo presupone el registro de cliente y los DTO anteriores; no se presenta como archivo aislado que funcione sin ellos.

Asincronía explicada con estados observables

Una operación await no equivale a ejecutar permanentemente otra tarea en un hilo dedicado. Su función en el consumo HTTP es permitir que el código de la aplicación espere una operación de entrada/salida de manera asíncrona. El método devuelve una tarea, y la continuación se ejecuta cuando la operación finaliza o falla.

El componente necesita conservar un estado claro durante la espera. Al comenzar la carga, cargando es verdadero. Después de obtener datos o capturar un error, finalmente pasa a falso. Esta última transición debe producirse incluso cuando se lanza una excepción; por eso la situamos en finally. De lo contrario un error de conectividad podría dejar el texto Cargando catálogo visible indefinidamente.

En la creación de productos interviene otra transición: enviando permanece verdadero mientras se realiza POST, para impedir que el usuario pulse el botón varias veces por impaciencia. Eso limita duplicaciones desde la interfaz, pero no hace que POST sea una operación idempotente ni evita peticiones repetidas procedentes de otros clientes. La API valida los datos con independencia del botón de Blazor.

Un principio útil es actualizar el catálogo después de recibir confirmación del servidor, no antes. Si el servidor devuelve un producto con un ID nuevo, incorporamos ese DTO a la pantalla. Si el POST falla, conservamos nombre y precio para que el usuario pueda corregirlos y reintentar cuando corresponda.

La carga inicial es una transición del componente

En Blazor Interactive Server, OnInitializedAsync permite iniciar una operación asíncrona relacionada con la inicialización del componente. Nuestro catálogo utiliza ese lugar para obtener la lista inicial. Elegimos prerender: false en el ejemplo para que la demostración resulte más simple al observar

cuándo comienza la operación interactiva. Esa elección no es una regla general sobre cómo deben construirse todas las aplicaciones Blazor.

Durante la espera, cargando es verdadero. Cuando la operación termina correctamente, productos contiene el resultado y la interfaz puede presentar la lista. Cuando hay una excepción, error contiene un mensaje y la interfaz muestra la situación correspondiente. El bloque finally desactiva cargando tanto en éxito como en fallo.

Si el alumno utiliza una página que conserva siempre cargando=true, lo primero es comprobar si el método de carga se está ejecutando, si contiene una excepción no tratada o si nunca se restaura el estado. La ausencia de productos en pantalla no prueba, por sí misma, que la API haya devuelto un arreglo vacío.

También distinguimos el estado de consulta del estado de creación. cargando representa la obtención inicial de la lista; enviando representa un POST iniciado por el usuario. Una aplicación más compleja podría modelar estados independientes de actualización, paginación y cancelación, pero en el laboratorio basta con evitar que una única variable signifique varias operaciones distintas.

¿Qué significa realmente que el cliente sea asíncrono?

Imaginemos una API que tarda tres segundos en responder a un GET. El código await permite continuar cuando llega la respuesta sin bloquear de forma síncrona todo el tiempo. No convierte la operación en instantánea ni garantiza que el usuario la perciba como rápida. De hecho, esa demora nos obliga a diseñar un estado de carga comprensible.

Comparar await con una llamada bloqueante ayuda a entender la decisión. Si utilizáramos Result sobre una tarea HTTP desde código de servidor, podríamos bloquear hilos innecesariamente y complicar la ejecución. Con async/await, la operación de E/S se integra en el flujo asincrónico normal de ASP.NET Core y Blazor.

Otra cuestión es la cancelación. HttpClient dispone de opciones de timeout y puede recibir CancellationToken para controlar operaciones. En este módulo configuramos un tiempo máximo sencillo. No implementamos todavía una experiencia completa de cancelación de peticiones, porque esa ampliación requiere decidir qué ocurre con el estado de UI, con el servidor y con operaciones de escritura ya iniciadas.

El alumno debe conservar una idea precisa: asíncrono significa esperar de manera apropiada una operación sin bloquear su ejecución como si fuera puramente síncrona. La calidad de la experiencia depende además de gestionar estados, errores y confirmaciones.

4.1.7. POST asíncrono y resultado de una creación

El envío de un producto debe diferenciar la entrada del usuario del resultado que asigna el servidor:

```
using System.Net.Http.Json;
using CursoWeb.Blazor.Models;

public static async Task<ProductoDto> CrearProductoAsync(
    HttpClient cliente,
    string nombre,
    decimal precio)
{
    var solicitud = new CrearProductoDto(nombre.Trim(), precio);
```

```
    using var respuesta = await cliente.PostAsJsonAsync(
        "api/productos", solicitud);

    // No confundimos una respuesta de error con un alta correcta.
    respuesta.EnsureSuccessStatusCode();

    return await respuesta.Content.ReadFromJsonAsync<ProductoDto>()
           ?? throw new InvalidOperationException(
               "La API confirmó el alta sin devolver un producto.");
}
```

`PostAsJsonAsync` crea el cuerpo JSON y envía POST. **No lanza automáticamente una excepción por todos los estados 4xx/5xx**: por eso invocamos `EnsureSuccessStatusCode`. Solo tras confirmar el estado leemos el producto creado y podemos incorporarlo a la interfaz.

En una interfaz real también hay que deshabilitar el botón durante el envío, informar del fallo y decidir qué hacer si se pierde la respuesta después de que el servidor ya haya procesado el POST. Para esta práctica no implementaremos reintentos automáticos de POST, pues podrían generar duplicados.

Caso de estudio: el botón responde, pero se crean dos productos

Un usuario pulsa dos veces el botón Crear porque la interfaz tarda en responder. Si ambas pulsaciones originan un POST, el servidor puede crear dos recursos distintos: una primera respuesta 201 con ID 3 y una segunda respuesta 201 con ID 4. Cada operación ha sido correcta individualmente, aunque el resultado conjunto no era el deseado.

La primera medida didáctica consiste en mantener una variable enviando en el componente y deshabilitar el botón mientras se espera. Observemos por qué ese mecanismo es solamente una medida de presentación. Un cliente HTTP diferente, una automatización o una conexión que repita una petición siguen pudiendo enviar POST adicionales a la API. No existe ninguna garantía de unicidad de la operación por haber cambiado una propiedad HTML.

Por eso tampoco introducimos un reintento automático indiscriminado cuando se agota el tiempo de POST. La excepción en el cliente indica que dejó de recibir el resultado a tiempo; no demuestra que la API no haya procesado la solicitud. Un diseño profesional podría incorporar claves de idempotencia o una regla de negocio que impidiera crear duplicados, pero eso excede la pequeña API didáctica de esta jornada.

El alumno debe poder explicar esta diferencia en términos de protocolo: dos GET suelen representar dos lecturas independientes del mismo recurso; dos POST de creación pueden representar dos cambios de estado. La semántica HTTP orienta nuestras decisiones de diseño incluso antes de elegir qué componente Razor dibuja el botón.

Lectura guiada de un intercambio POST completo

Consideremos una solicitud correcta de creación:

```
POST /api/productos HTTP/1.1
Host: localhost:5081
Content-Type: application/json
Accept: application/json

{"nombre":"Regla","precio":1.75}
```

La API deserializa el cuerpo como CrearProductoApi y comprueba que el nombre no está vacío y que el precio es mayor que cero. Después asigna un identificador que pertenece al estado de su servidor. La respuesta representa la creación efectiva:

```
HTTP/1.1 201 Created
Location: /api/productos/3
Content-Type: application/json

{"id":3,"nombre":"Regla","precio":1.75}
```

¿Qué debe hacer el componente con esa información? Primero comprobar que la operación fue satisfactoria, después deserializar el objeto creado y finalmente incorporarlo a la lista. La cabecera Location permite conocer la dirección relativa de la nueva entidad. La respuesta 201 no exige que el componente navegue automáticamente allí; la interfaz de este curso permanece en el catálogo y actualiza sus elementos.

Ahora cambiamos la solicitud:

```
POST /api/productos HTTP/1.1
Host: localhost:5081
Content-Type: application/json

{"nombre":"","precio":0}
```

El servidor rechaza la entrada con 400 y no asigna un identificador. El cliente no debe interpretar ese cuerpo como un producto. Por eso la implementación usa EnsureSuccessStatusCode antes de intentar leer ProductoDto. Si queremos mostrar en pantalla la razón exacta del rechazo, habría que diseñar un contrato estable de error para la API y leerlo en el cliente; no basta con suponer que todas las respuestas de error tienen la misma forma que las de éxito.

Esta comparación es especialmente útil para estudiantes que ya conocen Web API: no se trata de memorizar los nombres GET y POST, sino de reconocer qué consecuencias tiene para la aplicación receptora cada decisión de estado y cuerpo.

4.1.8. Respuestas HTTP, errores y decisiones de presentación

Situación	Interpretación	Respuesta de la interfaz
200 OK en GET	El catálogo se devolvió correctamente	Mostrar colección
201 Created en POST	Se creó un recurso	Mostrar producto creado
400 Bad Request	Datos de alta rechazados	Corregir los campos; no añadir el producto
404 Not Found	No existe la ruta o el recurso pedido	Mostrar fallo de consulta
500 Internal Server Error	Error del servidor	Mostrar mensaje no técnico
Conexión rechazada	API inaccesible	Mostrar problema de comunicación
Tiempo agotado	No llegó respuesta a tiempo	Permitir reintentar GET de forma explícita
JSON incompatible	El cuerpo no se ajusta al contrato	Mostrar error y revisar cliente/API

El código de estado y la excepción no son equivalentes: una respuesta HTTP 400 demuestra que **sí** se ha alcanzado algún servidor; un error de conexión puede significar que no se ha alcanzado ninguno.

Para un tratamiento más fino podemos leer `HttpResponseMessage` de forma explícita y distinguir `StatusCode`, como veremos en el ejercicio de error controlado. Los detalles internos y trazas del servidor no deben exponerse directamente al usuario de la aplicación.

Interpretación de situaciones que parecen iguales

Experimento	Significado del resultado	Qué debe hacer la interfaz
GET correcto devuelve dos objetos	El contrato funciona	Presentar los dos productos
GET correcto devuelve arreglo vacío	Catálogo sin elementos	Mostrar vacío sin acusar fallo
GET recibe 404	Ruta o recurso inexistente	Mostrar fallo HTTP
POST recibe 400	Datos no aceptados	Conservar la entrada y explicar rechazo
Se apaga la API	No se obtiene respuesta HTTP	Mostrar error de conectividad
La respuesta no corresponde al DTO	Desajuste de contrato	Informar incompatibilidad
La operación supera el timeout	Cliente dejó de esperar	Informar de demora
POST confirma 201 y un ID	Alta realizada	Añadir objeto real y limpiar formulario

La fila del timeout merece atención: cuando el cliente deja de esperar una respuesta, no podemos deducir sin más que el servidor dejó de trabajar. Un POST puede haberse procesado aunque su confirmación no llegara. Esa es una razón para no programar reintentos ciegos de operaciones de creación en este laboratorio.

Diagnóstico sistemático de una pantalla sin productos

El diagnóstico se realiza desde los límites del sistema. Primero se comprueba si la API está escuchando y responde directamente en la URL del listado. Segundo, se verifica que la respuesta tiene la forma de una lista y los campos esperados. Tercero, se comprueba que la URL de configuración del cliente apunta al mismo servidor y que la ruta relativa compone la dirección prevista. Cuarto, se estudia el tratamiento de errores del servicio y finalmente el estado visual de la pantalla.

No se cambia el componente, la configuración y el servidor a la vez. Modificar varios límites simultáneamente hace imposible averiguar cuál de ellos contenía el fallo. Separar observaciones y decisiones es parte del aprendizaje sobre integración, no una actividad administrativa.

Una guía de lectura de errores para el estudiante

Cuando GET o POST falla, pregúntate en primer lugar si existe una respuesta HTTP. Si no la hay, revisa el proceso API, el puerto, la dirección y el tiempo de espera. Si la hay, revisa método y ruta, código de estado y forma del cuerpo. Solo después estudia qué hace el componente con el resultado.

Para GET correcto, el cuerpo debe ser una lista de ProductoDto. Para POST correcto, un ProductoDto que incorpora el ID asignado. Para un error de reglas, la respuesta no representa un producto

creado, de modo que no debe incorporarse a la lista. Para una ruta inexistente, el servidor puede devolver 404 y el cliente debe comunicar que no obtuvo el recurso previsto.

Esta manera de razonar permite mantener el material del curso centrado en cuatro conocimientos: HTTP, contratos, responsabilidades y estados visuales. La sintaxis de las llamadas ayuda a ejecutarlos, pero la competencia profesional consiste en poder explicar qué sucedió cuando una de esas fronteras no respondió como esperábamos.

4.1.9. API auxiliar: contrato disponible antes de construir la práctica

Para que P4.1 no dependa de una API aprendida «en el futuro», el laboratorio facilita al inicio del módulo una pequeña aplicación ASP.NET Core con:

- `GET /api/productos`: devuelve un catálogo inicial con 200.
- `POST /api/productos`: valida nombre y precio, asigna un ID y responde 201.
- `GET /api/productos/{id}`: puede servir para mostrar el caso 404 (operación auxiliar del laboratorio).

La API reside en un proyecto independiente del cliente Blazor. La aplicación utiliza almacenamiento **en memoria** deliberadamente: el alumno ya conoce Web API y EF Core, y este módulo no pretende repetir esos cursos. En 4.3 se abrirá ese código, se explicará su funcionamiento y se modificará su respuesta.

Ejemplo de endpoint mínimo que materializa el contrato, con el modelo declarado dentro de su propio proyecto API:

```
using System.Collections.Concurrent;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var productos = new ConcurrentDictionary<int, ProductoApi>();
productos[1] = new ProductoApi(1, "Cuaderno", 2.50m);
productos[2] = new ProductoApi(2, "Lápiz", 0.80m);
var siguienteId = 2;

app.MapGet("/api/productos", () =>
    Results.Ok(productos.Values.OrderBy(p => p.Id)));

app.MapPost("/api/productos", (CrearProductoApi entrada) =>
{
    if (string.IsNullOrWhiteSpace(entrada.Nombre) ||
        entrada.Precio <= 0)
        return Results.BadRequest("Nombre y precio deben ser válidos.");

    var id = Interlocked.Increment(ref siguienteId);
    var creado = new ProductoApi(id, entrada.Nombre.Trim(), entrada.Precio);
    productos[id] = creado;
    return Results.Created($"/api/productos/{id}", creado);
});

app.MapGet("/api/productos/{id:int}", (int id) =>
    productos.TryGetValue(id, out var producto)
        ? Results.Ok(producto)
        : Results.NotFound());

app.Run();
```

```
record ProductoApi(int Id, string Nombre, decimal Precio);
record CrearProductoApi(string Nombre, decimal Precio);
```

La colección concurrente protege las operaciones básicas de lectura y escritura entre peticiones, y `Interlocked.Increment` asigna identificadores distintos. No estamos creando una capa de persistencia ni una arquitectura empresarial. El contrato JSON debe seguir siendo compatible con los DTO usados por Blazor.

4.1.10. Una secuencia razonable de trabajo

Una integración HTTP se comprende mejor comprobando cada capa por separado: primero ejecutamos la API y comprobamos GET; después iniciamos Blazor y verificamos la lista; a continuación enviamos POST; por último apagamos la API para reconocer un error real. Si hacemos todos los cambios de golpe, un puerto incorrecto, una ruta inexistente y un DTO incompatible se manifestarán como «no funciona», sin aportar pistas sobre su causa.

Esta secuencia también evita un error pedagógico frecuente: **modificar la aplicación de M03 sin explicar qué funciones conserva y cuáles se sustituyen**. La primera versión remota será deliberadamente pequeña, separada de `/productos`. Así el alumno podrá comparar ambos flujos, sin perder los filtros, la selección y la edición local que ya ha aprendido.

Síntesis de 4.1

Consumir una API no consiste únicamente en escribir `await cliente.GetFromJsonAsync`. Requiere conocer el contrato HTTP, representar correctamente los datos intercambiados, configurar el cliente, distinguir éxito y fracaso y actualizar el estado visible de Blazor con una respuesta confiable. En el siguiente punto, 4.2, retiraremos esa responsabilidad de los componentes y la situaremos en un servicio especializado.

Un ejercicio de diseño antes de continuar

Supón que se desea añadir un botón Actualizar que vuelva a consultar el catálogo. ¿Qué debería reutilizarse? El código de obtención HTTP debería tener una ubicación clara y la interfaz debería mantener su estado cargando y error. Si ya hubiéramos encapsulado la llamada en un servicio tipado, el botón necesitaría invocar ObtenerAsync y actualizar la colección, no conocer una URL.

Este ejercicio anticipa la necesidad de 4.2. Observa que podemos comenzar con un componente completamente funcional y descubrir una responsabilidad repetible a partir del uso real. Eso produce una refactorización motivada, en lugar de introducir interfaces antes de que el alumno entienda qué problema están resolviendo.

Al terminar 4.1, el alumnado debe poder describir con sus palabras al menos cuatro contrastes: consulta local frente a consulta remota; lista vacía frente a error; petición enviada frente a creación confirmada; fallo de ruta frente a fallo de conexión. Esos contrastes proporcionan una base conceptual que una simple demostración de GetFromJsonAsync no ofrece.

Ampliación 4.1 — Diseñar un cliente HTTP que informe sin confundir al usuario

En la primera versión del catálogo aprendimos a consumir JSON mediante métodos convenientes. Un objetivo posterior, especialmente útil cuando el servicio remoto no está bajo nuestro control, consiste en **distinguir explícitamente las situaciones de error**. La finalidad no es escribir más

condiciones por escribirlas, sino construir un comportamiento que el usuario pueda interpretar y el desarrollador pueda investigar.

El diseño de un cliente comienza por una decisión semántica: ¿qué considera el componente un resultado válido? En nuestro catálogo, una respuesta 200 con una lista vacía es válida. Una respuesta 200 con un objeto que no puede convertirse en lista no lo es. Una respuesta 400 tras un POST significa que el servidor rechazó datos; no significa que el servidor esté apagado. Si se rompe la conexión, ni siquiera disponemos de un código de estado del servidor remoto.

Una clasificación de resultados independiente de los textos visuales

Podemos modelar las observaciones con un tipo pequeño y explícito:

```
public enum TipoFalloRemoto
{
    Ninguno,
    RechazoHttp,
    Conexion,
    TiempoAgotado,
    ContenidoIncompatible
}

public sealed record FalloRemoto(
    TipoFalloRemoto Tipo,
    int? CodigoHttp,
    string Descripcion);
```

Este tipo no obliga a utilizar excepciones como texto de la interfaz. Permite hablar del problema en términos que pertenecen a la comunicación: código HTTP, conectividad, espera y contrato. La pantalla puede elegir después un mensaje apropiado.

El código HTTP se guarda como nullable porque en una desconexión o un timeout puede no haber un código de respuesta de la API. Atribuirles arbitrariamente 404 o 500 sería engañoso. Los valores 404 y 500 solo deberían mostrarse como tales cuando realmente se hayan recibido de un servidor.

No necesitamos convertir este pequeño modelo en una biblioteca genérica de resultados. Lo usamos para explicar una separación conceptual: **lo que se observó en la comunicación no es necesariamente el texto que se enseñará al usuario**.

Leer primero el estado y después el contenido

Para estudiar el protocolo con mayor detalle podemos escribir un método que reciba un HttpClient ya configurado y devuelva una colección únicamente cuando la respuesta haya sido satisfactoria:

```
public static async Task<List<ProductoDto>> ObtenerCatalogoExplicitoAsync(
    HttpClient cliente,
    CancellationToken cancelacion = default)
{
    using var respuesta = await cliente.GetAsync(
        "api/productos", cancelacion);

    if (!respuesta.IsSuccessStatusCode)
    {
        throw new HttpRequestException(
            $"La API devolvió {(int)respuesta.StatusCode}.",
            null,
            respuesta.StatusCode);
    }
```

```
    var productos = await respuesta.Content
        .ReadFromJsonAsync<List<ProductoDto>>(
            cancellationToken: cancelacion);

    return productos ?? [];
}
```

El método realiza tres operaciones separadas: obtiene una respuesta HTTP, comprueba que su estado es satisfactorio y convierte el cuerpo a un tipo C#. En las versiones compactas de GetFromJsonAsync esos pasos están reunidos; aquí aparecen desglosados para explicar su responsabilidad.

La comprobación IsSuccessStatusCode evita intentar convertir un cuerpo de error en una lista de productos. Sin embargo, no garantiza que el JSON sea compatible; la deserialización todavía puede fallar. Por eso ambos tipos de error deben pensarse por separado.

El parámetro CancellationToken también merece una explicación. Permite expresar que el consumidor ya no necesita seguir esperando una operación. No debe confundirse con una orden universal que deshace todo lo que el servidor remoto pudiera estar haciendo. Si cancelamos una lectura, el efecto es distinto de cancelar una creación después de que el servidor haya cambiado su estado.

Evitar un error frecuente con los códigos de respuesta

Supongamos que el servicio realiza GET y la API devuelve 404. El componente puede recibir una HttpRequestException con StatusCode cuando se usa la familia de métodos que valida estados de éxito. Si capturamos esa excepción sin estudiar su código, perdemos la diferencia entre ruta equivocada y red desconectada.

Una alternativa de interfaz es mostrar una frase comprensible mientras registramos por separado el tipo de incidencia. Por ejemplo, el usuario puede ver «No se ha podido consultar el catálogo», y el diagnóstico técnico puede expresar «HTTP 404 en /api/productos». No es necesario mostrar detalles internos del servidor, nombres de clases o trazas de excepción en una aplicación pública.

En el laboratorio docente sí tiene sentido permitir que el alumno vea el código HTTP, porque su objetivo es aprender a distinguir situaciones. Esa es la razón de haber creado una pantalla opcional de diagnóstico diferente de la página que utilizaría un usuario de negocio.

Por qué no debemos reintentar automáticamente cualquier operación

Los errores temporales invitan a introducir reintentos. En un GET de catálogo, repetir una lectura tras una desconexión breve podría ser razonable, siempre que se establezcan límites y condiciones. En un POST de creación, la decisión es más delicada: el servidor podría haber creado el recurso y perderse únicamente la respuesta.

Imaginemos esta cronología:

```
Cliente                      Servidor
   | POST Regla                  |
   |---------------------------->|
   |                             | crea ID 3
   |       (respuesta perdida)   |
   |<------ conexión interrumpida |
   | reintenta POST Regla        |
```

```
    |-------------------------->|
    |                           | crea ID 4
```

El cliente puede interpretar que la primera operación no quedó confirmada, pero no sabe que el servidor creó un producto. Si aplica una repetición ciega, puede terminar con dos. Este ejemplo explica la diferencia entre éxito del servidor y conocimiento del éxito por parte del cliente.

Un sistema real puede resolver situaciones semejantes mediante identificadores de solicitud o claves de idempotencia, cuya interpretación y almacenamiento corresponden al contrato del servidor. No las añadimos automáticamente al laboratorio porque diseñar ese mecanismo implica decisiones de persistencia que no forman parte de M04.

La conclusión no es «nunca repetir HTTP», sino **no equiparar un fallo al recibir la confirmación con la certeza de que no se ejecutó la operación**.

Timeout y cancelación: dos razones distintas para dejar de esperar

Un timeout definido al configurar HttpClient expresa un límite máximo de espera para la operación. Se utiliza para evitar que una petición permanezca pendiente indefinidamente. Un CancellationToken suele representar una señal externa de cancelación, por ejemplo que el usuario abandonó una operación o que el componente ya no necesita el resultado.

Ambos pueden manifestarse como finalización cancelada desde el punto de vista de ciertas APIs. El código de captura debe evitar prometer al usuario que el servidor «canceló el alta» si no dispone de esa evidencia. Podemos informar de que se agotó la espera y permitir comprobar el estado real del catálogo antes de volver a crear el producto.

Consideremos el caso de una pantalla que carga la lista mientras el usuario navega hacia otra página. Una implementación avanzada podría cancelar la operación pendiente cuando desaparece el componente. En este curso la versión obligatoria usa un flujo más sencillo; la lectura sobre cancelación sirve para entender las decisiones posteriores sin exigir un sistema nuevo de gestión de tareas.

Un criterio de diagnóstico basado en evidencias

Cuando el catálogo no funciona, no cambiamos componentes y puertos al azar. Seguimos un orden:

1. Consultamos directamente la API desde un cliente HTTP, sin Blazor.
2. Confirmamos que el host y puerto son los que aparecen en la configuración del servicio.
3. Comprobamos el método y la ruta real de la solicitud.
4. Leemos el código de estado recibido, si existe.
5. Comparamos el cuerpo JSON con el DTO esperado.
6. Estudiamos el estado de presentación del componente.

Si el primer paso falla por conexión, todavía no tenemos evidencia de un defecto en el DTO. Si la API responde 200 con una lista correcta pero la página muestra un error, la investigación debe desplazarse al cliente o al componente. Si responde 400 a un POST inválido, la regla del servidor está funcionando como se diseñó.

Este procedimiento es transferible a otras integraciones. No depende de que la tecnología remota sea exactamente nuestra Minimal API de .NET 8, porque las observaciones relevantes pertenecen al protocolo y al contrato de datos.

Ejemplo razonado: la lista del componente no es la base de datos

El componente mantiene una List porque necesita representar el catálogo. Esa lista es una instantánea local del estado que obtuvo de la API; no es una fuente de verdad universal.

Si otro cliente envía un POST válido, la API crea un producto. La pantalla ya abierta no recibe automáticamente una notificación de cambio. Hasta que vuelva a consultar el servidor o implemente una estrategia de actualización, puede mostrar una lista anterior. Esto no demuestra que GET falle ni que el DTO esté mal; demuestra que estamos observando dos estados en instantes diferentes.

Podríamos añadir un botón «Actualizar» que llame de nuevo a ObtenerAsync. La operación no tiene por qué reconstruir el componente entero: puede cargar datos nuevos, actualizar la colección y representar el resultado. En P4.2 ese botón se escribiría contra IProductosApiService, sin reintroducir detalles de URL en la pantalla.

En una aplicación empresarial habría que decidir si se usan actualizaciones periódicas, avisos de cambio o una caché con caducidad. No necesitamos implantarlas para enseñar el principio: **una respuesta HTTP es una representación obtenida en un momento, no una conexión mágica que sincroniza todos los consumidores**.

Qué significa realmente garantizar la compatibilidad de JSON

El tipo ProductoDto de Blazor contiene tres campos conocidos. La API los envía como propiedades del JSON. Si añadimos una nueva propiedad opcional en la respuesta, algunos consumidores podrán ignorarla sin problemas, mientras que otros podrían imponer reglas estrictas. Si cambiamos la raíz de una lista a un objeto, nuestro cliente actual dejará de corresponder a la respuesta.

Por tanto, no basta con que los cuerpos sean JSON válido ni con que ambas aplicaciones utilicen C#. Lo que las hace interoperables es que acuerden métodos, rutas, tipos, estructura, códigos de estado y significados. El servicio tipado de 4.2 concentra ese conocimiento técnico en un lugar, pero no elimina la obligación de respetar el contrato remoto.

La API reconstruida en 4.3 permitirá comprobar esta idea de manera directa: dos proyectos escritos de forma independiente pueden trabajar juntos mientras mantienen el contrato, aunque no compartan sus clases concretas.

Sobre errores y mensajes de detalle en producción

Un cliente HTTP que recibe un error debe poder informar al usuario sin revelar información que no necesita conocer. No sería apropiado mostrar automáticamente todos los detalles de una excepción interna, una cadena de conexión o una traza del servidor en una página pública.

En el entorno de desarrollo, los mensajes técnicos son útiles para el aprendizaje y la depuración. En un entorno publicado, la gestión de errores debe estar diseñada para aportar una referencia diagnóstica sin exponer datos internos. Esta diferencia también explica por qué la demostración de seguridad del punto 4.4 restringe ciertas rutas a Development.

No convertimos aquí el módulo en un curso de observabilidad, pero sí establecemos un hábito profesional: distinguir el mensaje que orienta al usuario de la información que necesita un desarrollador para investigar el fallo.

Síntesis de la ampliación HTTP

Un cliente robusto no es el que contiene más bloques catch, sino el que conserva correctamente el significado de las situaciones que encuentra. En nuestro laboratorio eso significa diferenciar lectura vacía y error, consulta y creación, código HTTP y desconexión, éxito del servidor y confirmación recibida, contrato JSON y estado visual.

La interfaz seguirá siendo sencilla. El valor añadido de este desarrollo teórico es que ahora podemos justificar cada una de sus decisiones y reconocer por qué una implementación aparentemente corta puede ser correcta o engañosa según lo que haga con las respuestas.

Ampliación 4.1 — Origen de las peticiones, JavaScript y CORS

Para diagnosticar correctamente una integración entre Blazor y una API es necesario saber **dónde se ejecuta el cliente HTTP**. En nuestros componentes Interactive Server, el código C# que usa HttpClient se ejecuta en el servidor Blazor. La API recibe una petición originada por otro proceso .NET, no directamente por una instrucción JavaScript del navegador.

Si, en cambio, ejecutamos JavaScript en una página web y utilizamos fetch para consultar la misma API, la petición sale del navegador. En ese escenario pueden entrar en juego las reglas CORS, que los navegadores utilizan para controlar el acceso del código web a respuestas de otros orígenes.

Comparar dos recorridos

En el ejercicio obligatorio el camino es:

```
Navegador → host Blazor Interactive Server → HttpClient .NET → API
```

En un ejemplo JavaScript sería:

```
Navegador → fetch JavaScript → API
```

Aunque ambos terminen solicitando GET /api/productos, no se ejecutan bajo las mismas reglas. La comunicación ordinaria entre dos procesos de servidor no necesita superar la comprobación CORS del navegador. Por ello no añadimos una política CORS como requisito para P4.1 y P4.2.

El origen web se identifica principalmente por esquema, host y puerto. Así, http://localhost:5091 y http://localhost:5081 son orígenes distintos, aunque ambos utilicen localhost. Una página alojada en el primero que intenta ejecutar fetch hacia el segundo realiza una solicitud de origen cruzado.

Un ejemplo JavaScript distinto del cliente Blazor

El código siguiente podría ejecutarse desde una página con JavaScript:

```javascript
const respuesta = await fetch("http://localhost:5081/api/productos");
const productos = await respuesta.json();
console.log(productos);
```

Si la API no permite a ese origen acceder a la respuesta, el navegador puede impedir que el código JavaScript la lea. Al mismo tiempo, curl.exe o un cliente HttpClient de servidor podrían consultar la API correctamente. Esto no es contradictorio: CORS es una restricción aplicada por el navegador en un contexto específico.

Por tanto, no conviene afirmar «la API está caída» únicamente porque un fetch de navegador muestre un error CORS. Tampoco conviene añadir políticas que permitan cualquier origen para resolverlo sin estudiar los requisitos de la aplicación.

CORS no es autenticación

Autorizar a un origen web a leer determinadas respuestas no significa autenticar a la persona que utiliza la página. Tampoco sustituye una política RequireAuthorization del recurso. Una API pública puede permitir acceso desde ciertos orígenes de navegador y seguir siendo pública para otros clientes HTTP.

En el punto 4.4 veremos autenticación mediante cookies y autorización por roles. Esos mecanismos responden preguntas diferentes. CORS se refiere a la relación entre el código del navegador y los orígenes de las respuestas; la autorización controla si una identidad tiene permiso para utilizar un recurso.

Es importante no confundir estos límites cuando aparecen mensajes de error aparentemente relacionados con «acceso» o «permiso».

Cómo se define una política restringida cuando es necesaria

ASP.NET Core proporciona una infraestructura para configurar CORS. Si un proyecto concreto necesitara permitir fetch desde el origen de desarrollo de Blazor, podría definir una política explícita:

```
builder.Services.AddCors(opciones =>
{
    opciones.AddPolicy("ClienteDesarrollo", politica =>
        politica.WithOrigins("http://localhost:5091")
                .AllowAnyHeader()
                .WithMethods("GET", "POST"));
});
```

Después, en el lugar adecuado del pipeline, se puede utilizar:

```
app.UseCors("ClienteDesarrollo");
```

Esta configuración es un ejemplo conceptual y **no se añade al proyecto obligatorio**. El alumno debe reconocer que permitir un origen es una decisión deliberada para clientes de navegador, no una receta automática para cualquier problema de HttpClient.

Por qué algunas solicitudes de navegador usan OPTIONS

Según el método y las cabeceras de una petición de origen cruzado, el navegador puede enviar previamente una solicitud OPTIONS para comprobar qué operaciones admite el servidor desde ese origen. Una solicitud POST con JSON y determinadas cabeceras puede desencadenar ese intercambio previo.

La respuesta de esa comprobación no es el producto creado: es una decisión sobre el acceso desde el origen web. Confundirla con el POST de negocio llevaría a interpretar de forma incorrecta los estados y cuerpos de respuesta.

Nuestro ejemplo principal de Blazor Interactive Server no necesita que el alumno construya ese flujo. Lo presentamos como una lectura adicional para que pueda comprenderlo cuando trabaje con JavaScript o con aplicaciones Blazor que utilicen otros modelos de ejecución.

Diagnóstico práctico de la diferencia

Si el catálogo Blazor no carga, comprueba primero si la solicitud utiliza HttpClient en el host Interactive Server. Si es así, investiga servidor API, puerto, ruta, estado HTTP y contrato JSON. No atribuyas sin evidencia una excepción de conexión a CORS.

Si el error procede de fetch ejecutado en la consola JavaScript del navegador, examina el origen de la página y el destino de la petición. Observa si el navegador ha realizado una solicitud preliminar y qué cabeceras de respuesta se han recibido. Solo entonces tiene sentido evaluar una política CORS concreta.

La competencia que se busca es identificar **qué proceso inició realmente la comunicación**. Esa pregunta permite distinguir fallos de red, problemas de contrato, restricciones del navegador y decisiones de seguridad del servidor sin modificar indiscriminadamente todas las capas de la aplicación.

4.2. Extraer el consumo HTTP a una capa de servicio

De un componente que hace todo a dos responsabilidades

En 4.1 `CatalogoRemoto.razor` aprendió a hablar HTTP: creó un `HttpClient`, conoció la ruta `api/productos`, interpretó respuestas JSON y mostró resultados. Esa mezcla fue útil para comprender el mecanismo, pero ahora presenta un inconveniente: cualquier componente que necesite productos debería repetir esos detalles de red. Cambiar la ruta, el tratamiento de un error o el contrato JSON obligaría a editar varias páginas.

La mejora consiste en separar **qué hace la interfaz** de **cómo se recuperan los datos**. El componente mantiene los formularios, los mensajes, la carga y el estado visual. Una clase de servicio conoce la API y concentra `GetFromJsonAsync`, `PostAsJsonAsync` y la conversión de errores.

4.2.1. El contrato IProductosApiService

Declaramos una interfaz con operaciones del negocio que necesita nuestra pantalla, no con detalles de transporte:

```
using CursoWeb.Blazor.Models;

namespace CursoWeb.Blazor.Services;

public interface IProductosApiService
{
    Task<IReadOnlyList<ProductoDto>> ObtenerAsync();
    Task<ProductoDto> CrearAsync(CrearProductoDto solicitud);
}
```

El nombre `ObtenerAsync` expresa la intención de consultar productos; no revela si la implementación utiliza HTTP, un fichero o datos de prueba. El `Task` indica que la operación puede completarse más tarde. `IReadOnlyList` impide que un consumidor suponga que puede modificar la colección interna del servicio. La interfaz no valida formularios HTML ni determina cuándo aparece un botón de carga: esas decisiones pertenecen a Blazor.

Diseñar una interfaz a partir del comportamiento y no de la tecnología

La interfaz IProductosApiService es una decisión arquitectónica pequeña y deliberada. Si se llamara IClienteHttp, la presentación seguiría pensando en tecnología de comunicación en vez de operaciones sobre productos. Los métodos ObtenerAsync y CrearAsync expresan lo que necesita la pantalla, mientras que la implementación ProductosApiService conoce las rutas, la serialización, el tiempo de espera y la interpretación de respuestas HTTP.

Una buena comprobación de este diseño es leer el componente después del refactor. Si contiene literales de rutas como api/productos o crea un HttpClient mediante IHttpClientFactory, la extracción está incompleta. Si solo usa la interfaz y mantiene la lógica de presentación, la frontera es visible.

No toda clase necesita una interfaz. Aquí tiene valor porque ya hemos enseñado el antes y el después: podemos comparar cómo queda el componente y por qué la comunicación queda

localizada. No se debe convertir este ejercicio en una regla dogmática que obligue a crear interfaces vacías para todos los objetos de una aplicación.

La extracción como una transformación que podemos observar línea a línea

El primer componente del laboratorio obtiene un cliente a partir de IHttpClientFactory. Para obtener los productos combina creación del cliente, ejecución de GET, comprobación implícita de respuesta y conversión del JSON. Para crear uno utiliza POST, comprueba el estado y transforma el cuerpo de respuesta. Es razonable empezar así porque el alumno ve el protocolo sin intermediarios propios de nuestra aplicación.

El inconveniente aparece cuando queremos reutilizar la misma lógica. Imaginemos que, además del catálogo, añadimos una tarjeta resumen que necesita obtener los productos para contar cuántos existen. Si copiamos la llamada HTTP en cada componente, dispersamos la ruta y repetimos el comportamiento de deserialización. Un cambio de contrato puede obligar a localizar todas esas copias. También sería fácil que una de ellas mostrara un catálogo vacío ante un error mientras otra informara de fallo.

El refactor de 4.2 concentra esas operaciones en una implementación única. El componente ya no sabe cómo realizar un GET; expresa que necesita productos. Eso no hace desaparecer HTTP, simplemente coloca su código donde podemos estudiarlo, modificarlo y verificarlo sin contaminar todas las páginas.

Para evaluar la transformación utilizamos dos perspectivas. La primera es **la conducta observable**: lista, formulario y errores permanecen como antes. La segunda es **la responsabilidad interna**: solo ProductosApiService conoce api/productos. Un cambio que satisfaga una perspectiva pero falle la otra no cumple el objetivo del ejercicio.

4.2.2. Implementación HTTP y gestión de errores

La clase `ProductosApiService` recibe un `HttpClient` configurado por el contenedor y ejecuta las operaciones:

```
using System.Net.Http.Json;
using CursoWeb.Blazor.Models;

namespace CursoWeb.Blazor.Services;

public sealed class ProductosApiService(HttpClient cliente)
    : IProductosApiService
{
    public async Task<IReadOnlyList<ProductoDto>> ObtenerAsync()
    {
        return await cliente.GetFromJsonAsync<List<ProductoDto>>(
            "api/productos") ?? [];
    }

    public async Task<ProductoDto> CrearAsync(CrearProductoDto solicitud)
    {
        using var respuesta = await cliente.PostAsJsonAsync(
            "api/productos", solicitud);
        respuesta.EnsureSuccessStatusCode();

        return await respuesta.Content.ReadFromJsonAsync<ProductoDto>()
            ?? throw new InvalidOperationException(
                "La API confirmó un alta sin devolver un producto.");
```

```
    }
}
```

El constructor primario de C# 12 deja explícita una única dependencia: `HttpClient`. `GetFromJsonAsync` puede lanzar `HttpRequestException` ante fallo de red o estado HTTP insatisfactorio, `TaskCanceledException` ante una operación cancelada o un tiempo límite, y `JsonException` ante JSON incompatible. La implementación no captura estas excepciones para devolver silenciosamente una lista vacía: hacerlo confundiría un error con un catálogo legítimamente vacío.

En `CrearAsync`, la respuesta POST se examina antes de leer su JSON. `EnsureSuccessStatusCode` evita registrar como creado un producto cuando el servidor devolvió una respuesta de error. `using` libera el mensaje de respuesta. Este servicio no registra contraseñas, no escribe en la base de datos y no sustituye a la API.

Por qué el constructor primario no altera la inyección de dependencias

La implementación está escrita con la sintaxis de constructor primario de C# 12:

```
public sealed class ProductosApiService(HttpClient cliente)
    : IProductosApiService
{
    public async Task<IReadOnlyList<ProductoDto>> ObtenerAsync()
    {
        return await cliente.GetFromJsonAsync<List<ProductoDto>>(
            "api/productos") ?? [];
    }
}
```

La sintaxis abreviada no significa que el servicio obtenga un cliente global. Al registrar AddHttpClient<IProductosApiService, ProductosApiService>, ASP.NET Core prepara la creación del objeto concreto y proporciona la instancia configurada de HttpClient cuando resuelve la dependencia. El nombre del parámetro cliente pertenece al constructor de la clase, no a la ruta de la página.

Podríamos escribir una versión equivalente con un constructor convencional, un campo privado y una asignación explícita. La decisión entre ambas sintaxis no cambia el contrato del servicio. Utilizamos la forma breve porque el curso ya presupone C# y queremos mantener visible el recorrido del dato, no detenernos a impartir construcciones básicas del lenguaje.

El servicio no decide cómo dibujar un error

Supongamos que ObtenerAsync lanza HttpRequestException. La implementación del servicio indica que no ha podido obtener los datos previstos. El componente recibe esa situación y decide mostrar un mensaje al usuario. Las dos acciones son diferentes: una interpreta un problema de infraestructura y la otra define la experiencia de presentación.

Una mala extracción consistiría en mover al servicio no solo HTTP, sino también la cadena HTML que se dibujará, la variable cargando y el estado del formulario. Eso haría que la clase necesitara conocer detalles de Razor y perdería el propósito de separarla.

La separación tampoco exige esconder toda excepción. En el ejemplo dejaremos que los fallos de HTTP o de serialización lleguen al componente, que conserva sus bloques catch. Así puede distinguir un error de conectividad de una respuesta que no se puede convertir al DTO. En un

producto de mayor tamaño podría definirse un tipo resultado explícito o un modelo centralizado de errores, pero no resulta necesario para comprender la frontera que estamos enseñando.

4.2.3. Registrar el servicio en Program.cs

Una interfaz no es instanciable por sí misma. El contenedor de inyección de dependencias necesita saber qué implementación proporcionar y cómo construir sus dependencias. El registro **typed HttpClient** hace ambas cosas:

```
builder.Services.AddHttpClient<IProductosApiService, ProductosApiService>(
    cliente =>
    {
        cliente.BaseAddress = new Uri(
            builder.Configuration["ProductosApi:BaseUrl"]
            ?? "http://localhost:5081/");
        cliente.Timeout = TimeSpan.FromSeconds(10);
    });
```

Con este registro, cuando una página solicita `IProductosApiService`, el contenedor crea una instancia de `ProductosApiService` con un `HttpClient` ya configurado. `AddHttpClient` está diseñado para gestionar la creación de clientes y la reutilización de manejadores HTTP sin que cada componente gestione su vida útil.

En 4.1 utilizábamos un cliente **con nombre**, recuperado con `IHttpClientFactory.CreateClient("ProductosApi")`. En este punto lo sustituimos por un cliente **tipado**. Son dos patrones legítimos, pero **no hace falta registrar los dos** para el mismo recorrido didáctico. Elegimos el cliente tipado porque el componente solo necesita la interfaz de negocio.

Inyección de dependencias: quién crea realmente el objeto

Una clase necesita poder obtener sus colaboradores. Sin inyección, el componente podría construir una implementación concreta, crear su HttpClient y escribir la URL del servidor. Así conocería muchos detalles que no pertenecen a una vista.

Con inyección, el contenedor ASP.NET Core mantiene un conjunto de registros. Al solicitar IProductosApiService resuelve ProductosApiService y le suministra un cliente configurado. El componente recibe el contrato, no una lista de instrucciones de construcción.

```
builder.Services.AddHttpClient<IProductosApiService, ProductosApiService>(
    cliente =>
    {
        cliente.BaseAddress = new Uri(
            builder.Configuration["ProductosApi:BaseUrl"]
            ?? "http://localhost:5081/");
        cliente.Timeout = TimeSpan.FromSeconds(10);
    });
```

Este registro combina dos ideas diferentes: la relación entre una interfaz y una clase concreta, y la configuración del cliente HTTP que esa clase recibe. Una modificación del puerto afecta a la configuración y no al código de la página. Una nueva implementación que obtuviera datos de otra fuente podría satisfacer la misma interfaz mientras mantuviera sus operaciones y significados.

Por qué no construimos HttpClient en cada pulsación

HttpClient administra mecanismos de conexión que conviene reutilizar. La fábrica de clientes gestiona adecuadamente los manejadores subyacentes y facilita una configuración centralizada. Eso evita que cada componente resuelva por su cuenta cómo mantener conexiones o cuánto debe esperar una respuesta.

En el primer punto utilizamos un cliente con nombre porque queríamos visualizar cada llamada. En este segundo punto empleamos un cliente tipado porque su configuración aparece asociada a ProductosApiService. Ambos enfoques son válidos, pero muestran grados distintos de encapsulación.

La mejora no consiste en ganar velocidad visible por cambiar una única línea. Consiste en reducir la cantidad de lugares que deben conocer decisiones de infraestructura y permitir que la página conserve únicamente las responsabilidades de presentación.

Alcance de un servicio tipado en una aplicación Blazor Interactive Server

El servicio vive en el host Blazor y utiliza la infraestructura HTTP de ese proceso. No se ha convertido en una API adicional ni en un microservicio separado por el hecho de implementar una interfaz. Su papel consiste en abstraer la comunicación con la API que ya existe.

El contenedor de ASP.NET Core gestiona la construcción de dependencias según el registro. No necesitamos utilizar un patrón Singleton manual para reutilizar HttpClient ni almacenar clientes en variables estáticas del componente. La fábrica integrada permite administrar los manejadores necesarios.

Aunque un equipo profesional pueda incorporar circuitos de resiliencia, autenticación de clientes, trazas distribuidas y políticas de reintento, no las introducimos de manera automática. Cada una resolvería un problema adicional y requeriría criterios propios. En particular, reintentar POST de creación sin controlar duplicados puede cambiar el resultado de negocio.

4.2.4. Inyectar la interfaz desde Razor

La página ya no necesita `IHttpClientFactory`:

```
@inject IProductosApiService CatalogoApi
```

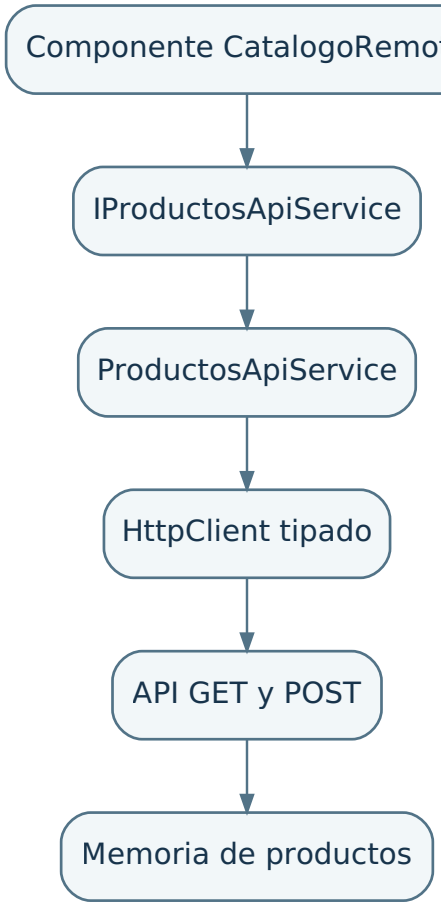
Durante la inicialización, el componente espera un resultado que ya son DTO:

```
protected override async Task OnInitializedAsync()
{
    try
    {
        productos = (await CatalogoApi.ObtenerAsync()).ToList();
    }
    catch (HttpRequestException)
    {
        error = "No se pudo obtener el catálogo remoto.";
    }
    catch (TaskCanceledException)
    {
        error = "La petición superó el tiempo disponible.";
    }
    catch (System.Text.Json.JsonException)
    {
```

```
            error = "La API no devolvió un catálogo JSON válido.";
        }
        finally
        {
            cargando = false;
        }
    }
```

El formulario sigue validando nombre y precio antes de enviar los datos. El método de alta construye `CrearProductoDto`, llama a `await CatalogoApi.CrearAsync(...)` y añade a la colección el objeto que devolvió el servidor. **El HTML y el resultado que observa el usuario no tienen por qué cambiar** al reorganizar las dependencias.

Un diagrama del refactor y sus fronteras



La flecha de la interfaz al servicio concreto debe interpretarse como la resolución efectuada por DI, no como una construcción manual desde Razor. El componente solicita una abstracción y el contenedor decide qué implementación proporcionar. La API es un proceso independiente: recibe HTTP, valida operaciones y devuelve JSON.

Este diagrama también permite detectar dónde debe ir una configuración. El puerto pertenece al registro del cliente, la ruta a la implementación HTTP, el nombre del producto creado al formulario y el ID del nuevo recurso al servidor API. Son cuatro decisiones repartidas entre partes distintas del mismo caso de uso.

Una variación que no debería provocar modificaciones en la UI

El docente cambia temporalmente la dirección base para apuntar a otra API que conserva el mismo contrato. El componente no debe cambiar. Si hubiera que buscar una URL en CatalogoRemoto.razor, el refactor habría dejado parte de la infraestructura en la vista.

Ahora supongamos una variación distinta: la API deja de publicar POST /api/productos y lo sustituye por POST /api/articulos. El cliente que conoce la ruta necesita adaptarse, y la API y el servicio deben coordinar ese cambio. No tenemos que cambiar el método @onclick ni la presentación, porque la operación de dominio sigue siendo CrearAsync.

Por último, cambiemos la respuesta para que el servidor deje de devolver el ID. En este caso la compatibilidad se rompe a nivel de DTO y la API debe restablecer el contrato o negociarse una nueva versión. Un servicio no puede inventar siempre una traducción correcta para datos que faltan. Una arquitectura con capas no elimina la obligación de acordar contratos entre procesos.

4.2.5. Distinguir responsabilidades y alcance

Responsabilidad	Componente Blazor	Servicio HTTP	API
Mostrar campos y mensajes	Sí	No	No
Decidir cuándo cargar	Sí	No	No
Construir petición HTTP y leer JSON	No	Sí	No
Asignar ID del producto	No	No	Sí
Validar reglas del servidor	No	No	Sí
Gestionar indicadores visuales de carga	Sí	No	No

La validación preliminar de un formulario no sustituye la validación de la API. El cliente puede cambiarse o saltarse: la API debe seguir rechazando entradas inválidas.

Esta separación facilita pruebas: un componente puede recibir una implementación falsa de `IProductosApiService` que devuelva datos conocidos, y el servicio real puede contrastarse contra la API de laboratorio. No añadimos aquí un framework de mocking ni una arquitectura compleja: el propósito es comprender por qué resulta útil tener un contrato inyectable.

No confundir responsabilidad con ubicación física

Extraer una clase de servicio no significa que la comunicación se traslade a otro servidor. ProductosApiService sigue viviendo en el proceso Blazor, igual que el componente; simplemente concentra la implementación del cliente HTTP. La API auxiliar permanece en su propio proceso y responde a las peticiones.

La separación conceptual es útil incluso cuando dos clases comparten un ensamblado. Un servicio de aplicación no se convierte automáticamente en microservicio por llamarse servicio. Del mismo modo, llamar a una API no obliga a introducir colas, autenticación externa o una arquitectura distribuida compleja.

Manejar errores sin esconder información relevante

Nuestro servicio lanza excepciones cuando no puede completar HTTP o deserializar datos. El componente las transforma en mensajes comprensibles. Otra alternativa sería devolver un tipo resultado que indicara explícitamente éxito o error; no lo implementaremos porque el objetivo de la jornada no es diseñar una biblioteca general de gestión de resultados.

Sí necesitamos comprender que convertir todas las excepciones en una lista vacía sería un error semántico. El consumidor ya no sabría si existe un catálogo vacío o si la API se ha detenido. La interfaz podría afirmar «No hay productos» en una situación en la que en realidad no logró consultar el servidor.

El punto donde se traduce un fallo técnico a un mensaje de usuario es una decisión de arquitectura. En este curso el servicio conserva información sobre HTTP y el componente decide qué expresar en pantalla. El alumno debe poder explicar dónde observa cada clase de problema.

Prueba mental de sustitución

Imaginemos que mañana el docente reemplaza la API de prueba por la API que construye un alumno. La dirección cambia temporalmente, pero ambas respetan GET y POST, las rutas y los DTO. Con la separación de 4.2 solo debe revisarse la configuración del cliente y comprobarse el contrato remoto.

Si en cambio la API devuelve una forma distinta, la interfaz por sí sola no puede reparar mágicamente el desacuerdo. La capa de servicio es el lugar apropiado para estudiar una posible adaptación de datos, pero el ejercicio evita esa complejidad para mantener visible el contrato original.

El valor didáctico del refactor queda así conectado con 4.3: construir otro servidor que cumpla el contrato es una prueba práctica de que las responsabilidades están bien ubicadas.

Sustituir temporalmente el servidor sin sustituir la pantalla

El contrato IProductosApiService hace posible imaginar implementaciones alternativas. No es obligatorio instalar herramientas de mocking para entender su utilidad. Pensemos en una clase que devuelve una lista predefinida de productos y asigna un ID de ejemplo cuando se le pide crear. Mientras implemente los mismos métodos y devuelva tipos coherentes, el componente podría trabajar con ella.

Este experimento conceptual sirve para responder una pregunta: si la UI puede representar datos sin que su código conozca HttpClient, hemos separado de verdad la presentación del transporte. Por supuesto, una implementación de prueba no demostraría que la API real responde 201 o valida entradas; esas responsabilidades se verificarán sobre el servidor en 4.3.

El alumno debe distinguir **aislar la vista** de **probar el contrato HTTP**. Son evidencias diferentes y ninguna sustituye a la otra. En nuestro itinerario primero validamos GET y POST reales, luego refactorizamos sin regresiones y después construimos un servidor compatible.

Distinguir un defecto de resolución de un fallo de conexión

Si olvidamos registrar IProductosApiService en el contenedor, la aplicación no podrá construir el componente que lo solicita. El problema aparece antes de ejecutar un GET. Cambiar la dirección de la API no ayudará, porque todavía no existe una implementación resuelta para iniciar la petición.

Si el registro de DI es correcto pero apunta a un puerto donde no existe un servidor, el componente sí obtiene su servicio, aunque la operación HTTP falla. Ahora las pruebas directas sobre el destino pueden señalar la causa.

Si tanto el registro como la conexión funcionan, pero la API devuelve un objeto en lugar de una lista, entramos en una tercera categoría: desajuste de contrato. Si el servicio devuelve la lista correctamente y la página permanece en cargando, la cuarta categoría es el estado del componente.

Estas cuatro clases de fallo dan una técnica de diagnóstico más valiosa que memorizar el orden de unas líneas de Program.cs. Al alumno se le pide explicar qué frontera ha dejado de funcionar y qué observación permitiría confirmarlo.

4.2.6. Comportamiento observable del refactor

La comprobación decisiva no es que exista una clase nueva: es que, tras sustituir el acceso directo, `/catalogo-remoto` siga listando Cuaderno y Lápiz, permita crear Regla, notifique un rechazo o un servidor apagado y mantenga funcional la página local `/productos` de M03. En esta práctica modificamos la organización del código sin cambiar el contrato HTTP ni la API.

En P4.3 abriremos la implementación de la API que ya proporcionamos al comenzar 4.1. De ese modo el alumno ya conoce el consumo y puede comprender ahora el origen de las respuestas sin introducir otra aplicación ajena al laboratorio.

Evaluar un refactor mediante invariantes

Un refactor cambia organización interna sin buscar modificar el comportamiento observable. Si después del cambio el catálogo deja de cargar, desaparece la validación o el botón ya no muestra el resultado de un POST, algo ha salido mal. No basta con que la aplicación compile.

Para razonar sobre la preservación del comportamiento definimos invariantes: al abrir /catalogo-remoto con la API disponible se muestran los productos; un alta válida añade exactamente el recurso confirmado; un alta inválida presenta un mensaje sin añadir elementos; una API apagada produce un error visible; la pantalla no conoce ya rutas HTTP.

Las primeras cuatro invariantes hablan de experiencia y comportamiento. La última habla de organización del código. Una comprobación completa necesita ambas categorías: una página puede conservar la UI y seguir mal acoplada, o separar perfectamente las clases y haber introducido una regresión funcional.

Lista razonada de errores de implementación

Error de registro: el componente solicita IProductosApiService pero el contenedor no tiene una implementación. La página puede fallar al activarse. La corrección consiste en registrar el servicio, no en capturar una excepción HTTP que todavía no ha ocurrido.

Error de URL: el servicio se resuelve y ejecuta, pero el cliente apunta a un puerto incorrecto. Ahora sí nos encontramos ante un problema de configuración o comunicación.

Error de contrato: la API responde con JSON incompatible. No es un problema de DI; debemos comparar la forma del JSON y los tipos del DTO.

Error de estado: el servicio devuelve productos, pero la interfaz continúa mostrando cargando. Debemos revisar el código del componente.

La razón para separar estas cuatro categorías es que cada una se manifiesta en un límite distinto. Cuando el alumno aprende a situar el fallo, empieza a poder investigar aplicaciones que no conoce de antemano.

Criterios razonados de éxito del refactor

Al revisar el estado final de 4.2 esperamos que el componente todavía contenga las variables productos, nombreNuevo, precioNuevo, cargando, enviando y mensaje. Son estados de presentación vinculados con el usuario. También esperamos encontrar ObtenerAsync y CrearAsync, porque son las operaciones que la vista necesita.

No esperamos encontrar rutas HTTP, PostAsJsonAsync o GetFromJsonAsync dentro del componente. Esos detalles deben aparecer en ProductosApiService, donde son coherentes con la responsabilidad de comunicación. A su vez, no esperamos encontrar llamadas a @onclick ni marcado HTML dentro del servicio.

El hecho de que las cadenas y tipos estén bien ubicados no prueba por sí solo el comportamiento. Debemos volver a ejecutar GET, POST, un fallo de red y la regresión de la página /productos anterior. Una buena separación de responsabilidades se reconoce tanto leyendo el código como observando que la aplicación sigue haciendo lo que hacía.

Ampliación 4.2 — Diseñar servicios que puedan probarse sin depender siempre de la red

El refactor del catálogo remoto introduce IProductosApiService y ProductosApiService. Hasta ahora hemos justificado ese diseño por la eliminación de rutas HTTP del componente. Existe otra consecuencia importante: podemos examinar por separado qué hace la vista, qué hace la implementación del servicio y qué responde realmente la API.

No debe confundirse esta separación con una afirmación de que las pruebas sin red son superiores a las pruebas HTTP. Ambas responden preguntas distintas. Un servicio simulado puede demostrar que la pantalla muestra productos obtenidos por una interfaz, pero no demuestra que el servidor responda con 201 ni que el cuerpo JSON sea compatible. Una prueba HTTP real demuestra el contrato entre procesos, pero no necesariamente la experiencia visual de la pantalla.

Un experimento de sustitución de implementación

Consideremos que el componente recibe un objeto IProductosApiService. Podríamos proporcionar una implementación que devolviera dos productos de memoria:

```
public sealed class ProductosApiSimuladoService : IProductosApiService
{
    private readonly List<ProductoDto> productos =
    [
        new(101, "Libreta simulada", 4.25m),
        new(102, "Bolígrafo simulado", 1.10m)
    ];

    private int siguienteId = 102;

    public Task<IReadOnlyList<ProductoDto>> ObtenerAsync()
    {
        IReadOnlyList<ProductoDto> respuesta = productos.ToArray();
        return Task.FromResult(respuesta);
    }
```

```
    public Task<ProductoDto> CrearAsync(CrearProductoDto solicitud)
    {
        if (string.IsNullOrWhiteSpace(solicitud.Nombre) ||
            solicitud.Precio <= 0)
            throw new ArgumentException("Datos incorrectos.");

        var creado = new ProductoDto(
            Interlocked.Increment(ref siguienteId),
            solicitud.Nombre.Trim(),
            solicitud.Precio);

        productos.Add(creado);
        return Task.FromResult(creado);
    }
}
```

No hay rutas, HttpClient ni JSON en esta implementación. Aun así, el componente puede invocar ObtenerAsync y CrearAsync, porque esas operaciones pertenecen al contrato público de la interfaz.

Para seleccionar esta variante se cambia el registro del contenedor de DI:

```
builder.Services.AddScoped<
    IProductosApiService, ProductosApiSimuladoService>();
```

La modificación se hace como alternativa al registro tipado, no añadiendo sin criterio dos registros que compiten para el mismo consumidor. Cuando se restaura la implementación real vuelve a utilizarse AddHttpClient<IProductosApiService, ProductosApiService> con su dirección configurada.

La observación didáctica es que la pantalla continúa utilizando los mismos métodos y DTO, mientras que la fuente de datos cambia. No significa que el servicio simulado sea una versión publicable de la API: su memoria y su ciclo de vida son distintos.

Qué demuestra una sustitución y qué no demuestra

Si el componente representa correctamente Libreta simulada y Bolígrafo simulado, sabemos que el estado de UI puede alimentarse mediante la abstracción sin conocer detalles HTTP. Si crea otro producto mediante CrearAsync y muestra el objeto devuelto, sabemos que su recorrido de alta respeta ese contrato.

No sabemos, sin embargo, si la API real devuelve 201 o si su puerto está configurado correctamente. La implementación simulada no emite HTTP. Para demostrar el contrato remoto seguimos necesitando ejecutar la API y enviar peticiones directas o desde el servicio real.

El estudiante debe poder expresar esa diferencia. No sería correcto decir «ya he probado la API porque el componente funciona con una implementación simulada». Tampoco sería correcto decir «la UI está bien porque curl obtiene 200»: ese resultado solo observa un lado del sistema.

Probar una implementación HTTP sin arrancar una API real

Otra técnica formativa consiste en sustituir el manejador interno de HttpClient. HttpClient acepta un HttpMessageHandler que decide cómo procesar una solicitud. Podemos escribir una implementación pequeña que devuelve una respuesta conocida y registra el método y la ruta que recibió:

```
public sealed class ManejadorDePrueba : HttpMessageHandler
{
```

```
    public HttpMethod? UltimoMetodo { get; private set; }
    public string? UltimaRuta { get; private set; }

    protected override Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage request,
        CancellationToken cancellationToken)
    {
        UltimoMetodo = request.Method;
        UltimaRuta = request.RequestUri?.AbsolutePath;

        var respuesta = new HttpResponseMessage(
            System.Net.HttpStatusCode.OK)
        {
            Content = new StringContent(
                "[{\"id\":1,\"nombre\":\"Cuaderno\",\"precio\":2.50}]",
                System.Text.Encoding.UTF8,
                "application/json")
        };

        return Task.FromResult(respuesta);
    }
}
```

En una prueba controlada podemos construir HttpClient utilizando ese manejador y una BaseAddress ficticia, y después pasar el cliente a ProductosApiService. El servicio realiza su GET habitual, pero el manejador de prueba devuelve el JSON conocido sin utilizar la red.

La prueba podría comprobar que UltimoMetodo es GET y UltimaRuta es /api/productos. También podría verificar que el resultado contiene Cuaderno. Son observaciones del comportamiento de ProductosApiService: la ruta que construye, el verbo que utiliza y cómo deserializa una respuesta.

Hay que limitar el alcance de esta técnica. Un manejador de prueba no reproduce por sí mismo el enrutamiento real de ASP.NET Core, sus reglas de validación ni el almacenamiento del servidor. Complementa la prueba de integración, no la reemplaza.

Relacionar la prueba con la responsabilidad de cada capa

Podemos organizar las comprobaciones del módulo en cuatro categorías:

Comprobación	Componente real	Servicio HTTP real	API real
UI con servicio simulado	Sí	No	No
Servicio con manejador simulado	No	Sí	No
GET/POST directo a API	No	No	Sí
Recorrido desde navegador hasta API	Sí	Sí	Sí

Ninguna fila por sí sola reemplaza todas las demás. El objetivo pedagógico no es multiplicar herramientas de pruebas, sino reconocer qué límite se observa cuando aparece un fallo.

Si el test del manejador demuestra que el servicio solicita /api/productos pero un GET directo devuelve 404, debemos revisar el servidor o la ubicación. Si el servidor devuelve el JSON correcto pero el servicio no puede deserializarlo, revisamos DTO y contrato. Si ambos funcionan, pero el componente permanece en carga, revisamos el ciclo de vida y sus estados de presentación.

Ciclo de vida del cliente tipado y del componente

La inyección de dependencias suministra objetos a componentes y servicios, pero no decide por nosotros la duración adecuada de todo estado mutable. En particular, no deberíamos guardar la lista del formulario o el estado de selección en un cliente HTTP compartido sin haber definido su semántica. Esos datos pertenecen normalmente a la presentación.

La fábrica de HttpClient gestiona el uso y la reutilización de manejadores HTTP. Eso no implica que una instancia concreta de ProductosApiService deba almacenarse indefinidamente en una variable estática. Un registro con AddHttpClient<TInterface,TImplementation> utiliza las convenciones de clientes tipados y ofrece un lugar central para configurar dirección y timeout.

En Blazor Interactive Server los componentes pueden permanecer activos en un circuito interactivo; el alumnado debe evitar pensar que cada clic crea una nueva aplicación ASP.NET Core. El botón invoca un método del componente existente, que utiliza las dependencias disponibles en ese contexto.

Una implementación simulada registrada como scoped tendrá un ciclo de vida asociado al ámbito de inyección. No debemos afirmar que esa memoria es una base de datos compartida por todos los usuarios. La elección de duración y persistencia forma parte del diseño y es independiente de que exista una interfaz.

Qué sucede al registrar dos alternativas sin intención

Una confusión frecuente aparece cuando se añade un registro para IProductosApiService sin retirar el anterior. Dependiendo de cómo se resuelva una dependencia única, el contenedor puede suministrar una implementación que no sea la que el alumno creía estar probando. El síntoma es una pantalla que muestra datos de la API real cuando se esperaba la lista simulada, o viceversa.

La solución didáctica es elegir expresamente una implementación para cada ejecución. Primero se prueba el cliente real con su API en 5081. Después se comenta ese registro y se habilita el servicio simulado. Finalmente se revierte la variante y se comprueba que la pantalla vuelve a depender de HTTP.

Este experimento también muestra por qué los proyectos de prácticas deben conservar estados claros: si muchas variantes permanecen activas a la vez, deja de ser evidente qué comportamiento se está estudiando.

La interfaz no garantiza equivalencia de todas las implementaciones

Dos clases pueden implementar los mismos métodos y seguir comportándose de manera distinta en aspectos que no expresa directamente la firma. Una puede conservar una lista solo durante una sesión; otra consulta un servidor externo; otra podría introducir retrasos o responder con errores.

Por tanto, la interfaz expresa un contrato de operaciones y tipos, pero debemos documentar también la semántica relevante: quién asigna IDs, qué significa un fallo, cuándo se confirma una creación y qué persistencia se espera. En el laboratorio ambas implementaciones devuelven ProductoDto para que el componente sea compatible, pero la prueba simulado no se presenta como equivalente funcional a un servidor real.

Cierre: por qué merece la pena el refactor

Después de estudiar estas alternativas, la utilidad de IProductosApiService resulta menos abstracta. Permite centralizar rutas y serialización, mantener el componente dedicado a la UI,

intercambiar fuentes de datos en una prueba y delimitar qué comprobaciones corresponden a cada capa.

No hemos añadido una arquitectura compleja sin justificación. Hemos empezado con un componente que funcionaba directamente con HttpClient, identificado el problema de dispersar comunicación y aplicado una transformación cuya ventaja podemos demostrar.

4.3. Construir desde cero una API REST y comprobar su contrato HTTP

Por qué estudiamos ahora el servidor

En 4.1 y 4.2 hemos consumido una API que ya estaba disponible: primero desde el componente Razor, después mediante `IProductosApiService`. Sabemos que `GET /api/productos` devuelve un catálogo y `POST /api/productos` crea una entrada, pero todavía no hemos seguido esa petición dentro del servidor. En este punto abriremos **el mismo proyecto** que se entregó al comenzar M04. No escribiremos una API empresarial ni repetiremos el temario previo de Web API: queremos entender las piezas que hacen posible el contrato que Blazor ya utiliza.

El recorrido completo es:

```
Blazor /catalogo-remoto
  → IProductosApiService
  → ProductosApiService (HttpClient)
  → GET o POST http://localhost:5081/api/productos
  → aplicación API: endpoint y reglas
  → respuesta HTTP + JSON
  → servicio cliente
  → estado de la pantalla
```

La API del laboratorio funciona en un proceso separado. Su modelo `ProductoApi` es propio; el cliente utiliza `ProductoDto`. Coinciden las propiedades públicas del contrato HTTP, pero no comparten necesariamente el mismo ensamblado.

4.3.1. El host mínimo en Program.cs

La API se inicia con el patrón habitual de .NET 8:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
// Aquí se declaran rutas mediante app.MapGet y app.MapPost.
app.Run();
```

`CreateBuilder` prepara configuración y servicios; `Build` crea la aplicación; `MapGet` y `MapPost` asocian un método HTTP y una ruta a un delegado que procesa la petición; `Run` mantiene el servidor atendiendo solicitudes. Este proyecto es pequeño porque el alumno ya conoce Web API: **Minimal APIs** nos permiten mostrar el contrato y las reglas sin añadir controladores y capas que no aportan aprendizaje a M04.

Un servidor ASP.NET Core mínimo sigue siendo un servidor completo

La plantilla web mínima crea un host que acepta peticiones mediante el servidor web configurado para la aplicación. La ausencia de controladores y de una estructura grande de carpetas no significa que no estemos usando ASP.NET Core. Minimal APIs es una manera concisa de describir rutas y manejadores dentro del mismo pipeline HTTP que utilizan otras aplicaciones ASP.NET Core.

La variable app representa la aplicación construida a partir del builder. Antes de llamar a Run, se registran endpoints y middleware. Run inicia el procesamiento de solicitudes. En los ejemplos mantenemos todos los endpoints dentro de Program.cs porque facilita que el alumno relacione la sintaxis con el protocolo. En proyectos mayores podría organizarse en otros archivos, pero esa refactorización no mejoraría por sí sola la comprensión del contrato estudiado.

Una vez que el host está operativo, una URL y un método HTTP permiten seleccionar un endpoint. El sistema de enrutamiento decide si una ruta como /api/productos/2 coincide con una plantilla que espera un identificador entero. Después se ejecuta la función asociada, que devuelve un resultado HTTP. Entender esta secuencia facilita distinguir un 404 originado por no encontrar una ruta del 404 devuelto por la lógica de un recurso inexistente.

Cómo enseñar un endpoint sin convertirlo en una clase magistral de Web API

El alumnado conoce C# y Web API. Por tanto, el docente puede comenzar con un problema concreto: el cliente espera un catálogo remoto y necesitamos construir el servidor mínimo que lo satisfaga. Primero se arranca un host vacío; luego se añade GET, después el detalle, POST y validación. No hace falta repetir toda la teoría de controladores y middleware de ASP.NET Core para entender estos pasos.

La práctica aprovecha esa experiencia previa y dedica esfuerzo a las decisiones que sí afectan al curso: contratos HTTP, funcionamiento del cliente Blazor, mensajes de error, aislamiento de responsabilidades y verificación del servidor por separado. El resultado es una API pequeña, pero completa para la historia que hemos elegido.

Por qué no empezamos por los controladores ni por EF Core

En la enseñanza de ASP.NET Core es habitual presentar primero las capas de una aplicación completa: controladores, servicios, repositorios y persistencia. Esos elementos tienen sentido cuando hay requisitos que justifican separar operaciones y conservar datos. Sin embargo, nuestro problema de M04 es mucho más acotado: queremos comprender cómo una aplicación Blazor consume datos de otra aplicación mediante un contrato HTTP.

La plantilla dotnet new web y las Minimal APIs son suficientes para estudiar ese límite con poco código auxiliar. El endpoint GET expresa una lectura y el POST una creación. El cuerpo JSON se enlaza a un DTO y el resultado HTTP expresa el desenlace. El alumno que conoce Web API de módulos o cursos anteriores puede reconocer conceptos equivalentes a los de acciones de controlador, sin necesidad de repetir una explicación extensa sobre la estructura MVC de una API.

El catálogo se almacena en memoria para que los escenarios sean deterministas y fáciles de reiniciar. Si usáramos EF Core aquí, aparecerían nuevas responsabilidades: cadena de conexión, esquema, migraciones, ciclo de vida de DbContext y persistencia. Ya se presupone que el alumnado conoce EF Core, pero esas operaciones no son la meta formativa del día. Una implementación más compleja podría ocultar la relación básica entre solicitud HTTP y respuesta.

La decisión tiene un precio conocido. Al reiniciar la API desaparecen los productos creados; el diccionario no realiza transacciones; y la operación de creación no dispone de reglas avanzadas

contra duplicados. La enseñanza adecuada consiste en señalar estos límites, no en disfrazar una colección temporal de sistema de datos empresarial.

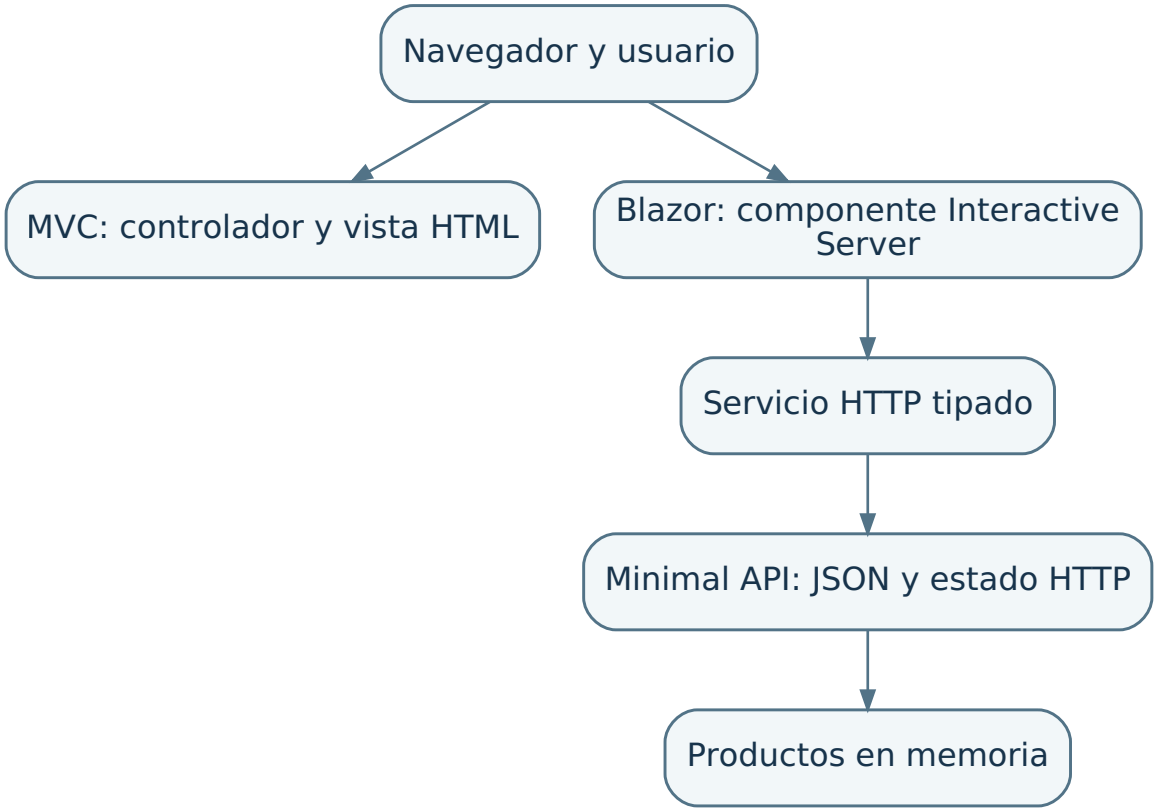
Tres aplicaciones ASP.NET Core, tres responsabilidades de presentación

En este curso hemos utilizado MVC, Blazor Web App Interactive Server y una pequeña Minimal API. Todas pueden construirse sobre ASP.NET Core, pero eso no significa que sus responsabilidades deban mezclarse.

En MVC, el navegador envía solicitudes a la aplicación, una acción del controlador coordina la operación y una vista Razor produce HTML para la respuesta. La interacción posterior del usuario suele originar nuevas solicitudes que pueden terminar en acciones distintas. En los primeros módulos el alumno aprendió a relacionar controlador, modelo y vista, y a gestionar formularios y validaciones.

En Blazor Interactive Server, el navegador representa componentes y participa en una conexión interactiva con el servidor Blazor. Un evento del usuario puede invocar código C# de un componente sin requerir que escribamos manualmente una acción MVC para cada pulsación. El componente mantiene estado de presentación y puede llamar a servicios registrados mediante DI.

En la Minimal API auxiliar, no diseñamos páginas. El servidor publica recursos HTTP y devuelve datos JSON junto con códigos de estado. Un navegador, una aplicación Blazor, una terminal o cualquier cliente compatible pueden utilizar esas operaciones. Por eso las reglas de creación y asignación de ID tienen que estar en la API: la interfaz no es el único consumidor posible.



El diagrama no afirma que MVC sea parte del recorrido de una petición de Blazor en M04. Presenta dos modos de construir interfaz y el recurso HTTP que utiliza el segundo. El alumno debe poder

explicar cuándo está procesando una solicitud de navegación, cuándo ejecuta un evento de componente y cuándo consume un recurso JSON.

4.3.2. Qué devuelve realmente GET

La colección se crea en memoria al arrancar:

```
var productos = new ConcurrentDictionary<int, ProductoApi>();
productos[1] = new ProductoApi(1, "Cuaderno", 2.50m);
productos[2] = new ProductoApi(2, "Lápiz", 0.80m);
```

`ConcurrentDictionary` permite accesos concurrentes seguros a las operaciones de diccionario que usamos. Los registros iniciales son datos del laboratorio, no datos de SQL Server. Esta línea publica el catálogo:

```
app.MapGet("/api/productos", () =>
    Results.Ok(productos.Values.OrderBy(p => p.Id)));
```

`Results.Ok` devuelve **200 OK**. ASP.NET Core serializa la colección en JSON: los nombres de propiedades se exponen en *camelCase* bajo las opciones web habituales. `OrderBy` evita presentar el orden interno de un diccionario concurrente como si fuera un criterio estable de listado.

Si consultamos `/api/productos/999`, el endpoint de detalle devuelve `Results.NotFound()`. **404 no es una excepción de C# obligatoria**: es una decisión explícita de protocolo que el cliente debe interpretar.

Rutas, verbos y datos de entrada

Dos endpoints pueden compartir el mismo camino literal si utilizan verbos diferentes. GET /api/productos obtiene una colección; POST /api/productos intenta crear un recurso. El método forma parte del contrato, no es un detalle intercambiable. Enviar POST a una ruta preparada únicamente para GET no significa que se ejecute la misma acción de consulta.

Para pedir un producto individual utilizamos /api/productos/{id:int}. El sufijo :int restringe el parámetro de ruta a valores enteros; evita tratar cualquier texto arbitrario como identificador numérico. Sin embargo, la restricción de ruta no significa que la colección contenga ese ID. La comprobación efectiva se realiza con TryGetValue, y su resultado decide entre 200 y 404.

El POST recibe un objeto CrearProductoApi. ASP.NET Core se encarga del enlace del cuerpo JSON a ese tipo de entrada. A partir de ese momento, el propio endpoint debe aplicar las reglas funcionales del laboratorio. JSON bien formado no implica nombre válido ni precio positivo.

Analizar el enrutamiento sin confundir parámetros y literales

Consideremos estas dos plantillas de ruta:

```
app.MapGet("/api/productos/{id:int}", (int id) =>
    productos.TryGetValue(id, out var producto)
        ? Results.Ok(producto)
        : Results.NotFound());

app.MapGet("/api/productos/resumen", () =>
{
    var actuales = productos.Values.ToArray();
    return Results.Ok(new
```

```
    {
        cantidad = actuales.Length,
        precioTotal = actuales.Sum(p => p.Precio)
    });
});
```

En la primera, el segmento id tiene una restricción de entero. En la segunda, resumen es un literal. Por tanto, una solicitud a /api/productos/resumen no necesita convertirse en un identificador numérico. Esta distinción permite ampliar la API con un recurso de resumen sin alterar la consulta individual.

Ahora pensemos en /api/productos/999. El servidor puede seleccionar correctamente el endpoint con id=999, pero la búsqueda en el diccionario no encuentra el elemento. Se devuelve 404 por decisión del manejador. Si en cambio solicitamos /api/catalogo, ninguna de estas plantillas corresponde a la ruta; el resultado no proviene de un producto que falta, sino de una URL no publicada.

Dos observaciones con el mismo estado HTTP no siempre proceden de la misma causa. Cuando diagnosticamos una API conviene preguntar primero si la ruta y el verbo coinciden, y después si la operación pudo encontrar los datos que necesitaba.

Qué significa comunicar servidores en Interactive Server

En nuestro laboratorio el código HttpClient de Blazor se ejecuta en el servidor que aloja los componentes. El navegador se comunica con Blazor, y el servidor Blazor se comunica con la API. Es distinto de construir una aplicación JavaScript que emite desde el propio navegador todas las solicitudes a otra URL.

Esa distinción explica por qué no añadimos indiscriminadamente políticas CORS a la API de esta jornada. CORS es una política del navegador relacionada con solicitudes de origen cruzado; la comunicación de HttpClient entre dos procesos .NET del servidor no está sometida a esa comprobación del navegador. Si cambiáramos el diseño para realizar fetch desde código del navegador, sí deberíamos analizar los orígenes permitidos según el escenario real.

Conviene no transformar esta observación en la frase «Blazor nunca necesita CORS». Blazor puede utilizar diferentes modelos de ejecución y las aplicaciones pueden combinar componentes con otros clientes. La conclusión exacta es que **el GET y POST de este proyecto concreto se realizan desde el proceso Blazor Interactive Server**, por lo que añadir CORS a la API no es una condición necesaria para esos ejercicios.

4.3.3. Cómo procesa POST la entrada

La API recibe un objeto de entrada:

```
record CrearProductoApi(string Nombre, decimal Precio);
```

Ese contrato **no tiene Id**: el identificador lo asigna el servidor. El delegado de POST valida nombre y precio:

```
if (string.IsNullOrWhiteSpace(entrada.Nombre) || entrada.Precio <= 0)
    return Results.BadRequest("Nombre y precio deben ser válidos.");
```

Si los datos cumplen las reglas, se ejecuta:

```
var id = Interlocked.Increment(ref siguienteId);
var creado = new ProductoApi(id, entrada.Nombre.Trim(), entrada.Precio);
productos[id] = creado;
return Results.Created($"/api/productos/{id}", creado);
```

`Interlocked.Increment` produce IDs distintos aunque dos peticiones se procesen simultáneamente. `Results.Created` devuelve **201 Created**, el cuerpo del producto creado y una cabecera `Location` que apunta a su recurso. Por eso nuestra API expone también `GET /api/productos/{id}`.

El cliente no debe inventar el ID ni tratar una respuesta 400 como creación satisfactoria. La validación previa de la página Blazor facilita la experiencia de usuario, pero la API siempre vuelve a validar: el servidor no puede confiar en que todas las llamadas provienen de nuestro formulario.

200, 201, 400 y 404 como decisiones de diseño

El 200 de GET expresa éxito en la consulta. La respuesta de detalle puede incluir un objeto y la del listado un arreglo; ambas son 200, pero tienen formas distintas. El 201 de POST indica que se ha creado un recurso. Con Results.Created incluimos la dirección relativa del recurso creado y el objeto final, de modo que un cliente pueda identificarlo sin inventar el nuevo ID.

El 400 tiene otro significado: la petición fue comprendida lo suficiente para detectar una entrada inaceptable. En el ejemplo ocurre si Nombre es vacío o si Precio no es positivo. No debemos cambiar arbitrariamente este resultado por 500; las reglas han rechazado datos, no hemos demostrado un fallo interno de ASP.NET Core.

El 404 del detalle es una decisión normal cuando no existe el identificador solicitado. La ausencia de un producto no exige lanzar una excepción. En un servicio real podrían intervenir otras decisiones de dominio, pero no necesitamos introducir una arquitectura adicional para comprender la semántica del recurso.

Concurrencia y almacenamiento en memoria: qué garantiza cada pieza

El catálogo utiliza ConcurrentDictionary<int, ProductoApi>. Su propósito es permitir operaciones de diccionario concurrentes sin diseñar manualmente un mecanismo de bloqueo para las inserciones y lecturas simples del laboratorio. Los identificadores se obtienen mediante Interlocked.Increment; así dos peticiones que lleguen próximas en el tiempo no dependen de una lectura y escritura no atómica de un contador común.

Son dos decisiones diferentes: una protege las operaciones sobre la colección y otra incrementa el contador. Ninguna aporta persistencia en disco. Después de detener y reiniciar la aplicación, el proceso vuelve a construir los dos productos iniciales y siguienteId vuelve a 2. Esta observación es una ventaja didáctica porque permite repetir el ejercicio con un estado conocido.

Tampoco debemos interpretar esta estructura como un reemplazo de transacciones de base de datos. El curso presupone conocimientos de EF Core; aquí evitamos introducirlo porque el objetivo es estudiar la frontera HTTP entre cliente y servidor. Mezclar persistencia en este punto aumentaría la superficie de problemas sin mejorar el aprendizaje específico de GET, POST y contratos.

La frontera entre deserializar JSON y validar una regla

ASP.NET Core puede convertir un cuerpo JSON a un tipo como CrearProductoApi. Esa conversión establece que los datos recibidos tienen una representación compatible con el tipo de entrada, pero no decide por sí sola si el producto es aceptable para nuestro dominio.

La función del ejemplo comprueba dos reglas: nombre no vacío y precio positivo. Son deliberadamente sencillas para poder observarlas. Una petición con nombre formado por espacios es JSON válido, pero se rechaza porque no aporta un nombre significativo. Una petición con precio cero también es sintácticamente correcta, pero no cumple el contrato funcional de creación.

La distinción importa especialmente al enseñar depuración. Si el cuerpo HTTP está mal formado, la solicitud puede fallar antes de entrar al bloque de validación. Si el cuerpo enlaza correctamente pero no cumple una regla, el manejador devuelve 400. Y si cumple las reglas, el servidor asigna un ID y devuelve 201.

Podemos identificar tres capas conceptuales en una única operación:

```
Bytes y cabeceras HTTP
    → interpretación del JSON
    → CrearProductoApi
    → reglas de nombre y precio
    → asignación de identificador
    → respuesta 201 y ProductoApi
```

Cada transición puede explicar un fallo distinto. El alumno debe aprender a localizar cuál de ellas no se ha completado, en lugar de cambiar todos los elementos de la solución ante un mensaje de error.

La cabecera Location como parte de la creación

Cuando una API crea un recurso, puede informar de la dirección donde será consultable. Results.Created permite incluir una dirección relativa y el objeto creado. En nuestro contrato, POST /api/productos devuelve Location: /api/productos/{id}, y GET de esa dirección puede obtener el producto mientras permanezca en memoria.

El valor de Location no sustituye el cuerpo JSON. Para la interfaz que estamos construyendo es cómodo recibir también el producto completo, porque puede actualizar su lista sin realizar otra consulta de detalle. El cuerpo permite conocer el ID, el nombre normalizado y el precio que ha confirmado el servidor.

Imaginemos una respuesta que declarara 201 y devolviera un cuerpo vacío. Podría existir un diseño de API en el que el cliente realizara una petición posterior a Location, pero no es el contrato que espera nuestro ProductosApiService, que intenta leer un ProductoDto inmediatamente. Una decisión legítima en abstracto puede ser incompatible con un consumidor concreto.

Del almacenamiento local a la comunicación entre procesos

En M03 la colección podía depender de un servicio de memoria dentro del proceso Blazor. En M04, aunque el servidor también utilice una colección en memoria, la ubicación de esa memoria cambia: ahora pertenece al proceso API y Blazor solo recibe representaciones serializadas.

Esto tiene consecuencias prácticas. Al recargar la página Blazor sin reiniciar la API, los productos creados continúan en el servidor. Si detenemos y reiniciamos la API, la colección se reconstruye desde los datos iniciales. Si detenemos Blazor pero la API sigue funcionando, los datos del servidor no tienen por qué desaparecer. Estos experimentos demuestran la separación de procesos sin introducir una base de datos.

También muestran por qué el componente no debería tratar su lista de DTO como fuente universal de verdad. Es una representación del estado que la API devolvió en un instante. Si otro cliente crea un recurso, la pantalla necesitaría consultar nuevamente para observarlo. El ejemplo no

implementa notificaciones en tiempo real, pero sí permite reconocer que una copia local y el estado remoto no son la misma cosa.

4.3.4. Modificar una respuesta de forma controlada

Vamos a añadir una operación de lectura que no cambie el contrato que ya consume Blazor: `GET /api/productos/resumen`. Responderá con el número de productos y el valor agregado de sus precios.

```
app.MapGet("/api/productos/resumen", () =>
{
    var actuales = productos.Values.ToArray();
    return Results.Ok(new
    {
        cantidad = actuales.Length,
        precioTotal = actuales.Sum(p => p.Precio)
    });
});
```

El material devuelve, por ejemplo:

```
{ "cantidad": 2, "precioTotal": 3.30 }
```

Tras crear Regla a 1,75, el resumen indicará tres productos y un precio total de 5,05. Son valores ilustrativos que dependen de partir del estado inicial de la API. **No cambiaremos** `GET /api/productos`, porque 4.1 y 4.2 dependen de que esa ruta siga devolviendo una lista. Esta es la idea del ejercicio: ampliar sin romper un cliente existente.

La ruta literal `/api/productos/resumen` no se confunde con `/api/productos/{id:int}`, porque esta última exige un parámetro entero. El resultado de resumen tiene su propia forma JSON; no debe deserializarse como `ProductoDto`.

Añadir una ruta sin romper la ruta existente

El resumen utiliza una URL propia. La operación lee los valores actuales, calcula cantidad y suma de precios, y devuelve un objeto con ambas propiedades. El endpoint original continúa devolviendo una colección de productos. Esto evita imponer al cliente existente un cambio estructural que nunca solicitó.

Hay un matiz de enrutamiento: /api/productos/resumen no debe interpretarse como un identificador entero. La ruta de detalle usa la restricción {id:int}; por ello el literal resumen no satisface la plantilla de detalle, y puede resolverse el endpoint específico.

La variante del reto añade hayProductos, un booleano obtenido de comparar la longitud con cero. Al activar ese reto no debemos registrar dos endpoints GET idénticos para la misma ruta, pues crearíamos una definición conflictiva. Se utiliza una variante u otra. El comentario de activación del proyecto permite observar esta decisión de forma tangible.

Cuándo una extensión del contrato es segura para este cliente

El endpoint /api/productos/resumen devuelve un objeto diferente del listado. La ruta nueva evita alterar la forma del GET que ya consume Blazor. Añadir hayProductos al objeto de resumen es una variación de su salida que podemos estudiar en aislamiento.

No debemos concluir que añadir siempre cualquier propiedad a cualquier respuesta es automáticamente seguro. Un cliente puede imponer reglas estrictas sobre contratos, versiones o validación. En el laboratorio actual, el consumidor del catálogo no consulta la nueva ruta y por eso no cambia. Si quisiéramos crear una página de resumen, definiríamos su DTO y su operación cliente de manera explícita.

Este ejemplo introduce una idea útil para trabajos reales: la compatibilidad de una API no consiste únicamente en que el servidor compile. Hay que comprobar las operaciones que consumen las aplicaciones existentes y distinguir entre ampliar recursos y sustituir contratos ya publicados.

Una ampliación opcional con sentido: buscar productos por nombre

Hasta este momento GET /api/productos devuelve todos los elementos y GET /api/productos/{id:int} busca un recurso por su identificador. Un tercer caso frecuente consiste en consultar **una selección** de la colección a partir de un criterio proporcionado por el usuario. Podemos construirlo como reto opcional sin cambiar el contrato de los endpoints anteriores.

La URL puede ser:

```
GET /api/productos/buscar?nombre=cuad HTTP/1.1
Host: localhost:5081
Accept: application/json
```

La ruta identifica la operación y la cadena de consulta contiene el criterio nombre. A diferencia del ID incluido en un segmento de ruta, el parámetro de consulta representa una condición de filtrado. No significa que cuad sea una clave única: es solo un fragmento de nombre que podría coincidir con varios productos.

ASP.NET Core permite enlazar el valor de la consulta a un parámetro del manejador. Nuestro endpoint opcional utiliza string? para representar que puede no existir un valor. En ese caso no devuelve todos los productos sin avisar; decide rechazar la solicitud con 400 porque la operación se ha definido como una búsqueda con término obligatorio.

```
app.MapGet("/api/productos/buscar", (string? nombre) =>
{
    if (string.IsNullOrWhiteSpace(nombre))
        return Results.BadRequest("Indica el nombre que deseas buscar.");

    var termino = nombre.Trim();
    var encontrados = productos.Values
        .Where(p => p.Nombre.Contains(
            termino, StringComparison.OrdinalIgnoreCase))
        .OrderBy(p => p.Id)
        .ToArray();

    return Results.Ok(encontrados);
});
```

Hay varias decisiones didácticas dentro de este bloque. Primero, la búsqueda no modifica el estado del catálogo: es GET. Segundo, normaliza el texto del filtro con Trim, pero no cambia los nombres almacenados. Tercero, Contains con StringComparison.OrdinalIgnoreCase realiza una comparación insensible a mayúsculas de manera explícita para este laboratorio. Cuarto, ordena los resultados por identificador, evitando basarse en el orden interno del diccionario concurrente.

La salida mantiene la misma **forma de lista de productos** que el cliente ya conoce. Si se encuentra Cuaderno, devuelve un arreglo con ese objeto. Si no hay coincidencias, devuelve un arreglo vacío con 200, porque la búsqueda se ha ejecutado correctamente aunque no haya resultados. Si no hay un término válido, devuelve 400 porque no se cumplen los requisitos de esa operación. Esta secuencia permite distinguir de nuevo un éxito vacío de una entrada rechazada.

Cómo convive una ruta literal con una ruta parametrizada

Nuestro servidor ya publica /api/productos/{id:int}. ¿Puede entrar en conflicto con /api/productos/buscar? No en el contrato definido: buscar es un segmento literal y no cumple la restricción de entero del endpoint de detalle. Eso permite conservar ambas operaciones sin obligar a cambiar la dirección del catálogo.

El diseño también muestra por qué no conviene utilizar una única ruta para operaciones semánticamente diferentes si eso obliga a interpretar todo a partir de formatos ambiguos. Un segmento con un número se utiliza para consultar un producto concreto; la ruta buscar junto a un parámetro nombre expresa un criterio de selección. La decisión se hace visible para quien prueba la API con un archivo .http.

Con independencia de este ejemplo, una API profesional debe acordar sus rutas y criterios de búsqueda en función de sus consumidores. Nuestro laboratorio no pretende establecer una convención universal de búsqueda, sino demostrar el papel distinto de segmentos de ruta y parámetros de consulta.

Tres resultados que enseñan más que una sola respuesta

Con los productos iniciales, la solicitud a /api/productos/buscar?nombre=cuad debe devolver un arreglo con Cuaderno. No se crea ni elimina ningún producto.

Si consultamos /api/productos/buscar?nombre=nohaycoincidencia, el resultado es 200 con []. La API ha entendido y ejecutado el filtro y no ha encontrado datos. Esa respuesta es diferente de un 404, porque no estamos pidiendo por su dirección un recurso individual que deba existir.

Si consultamos /api/productos/buscar sin nombre, el endpoint responde 400. La entrada incumple la regla de término obligatorio. En el laboratorio decidimos que un filtro vacío sea una petición incorrecta y no un listado completo; ese comportamiento debe documentarse para los consumidores.

Podemos ampliar el experimento creando mediante POST un producto llamado Cuaderno grande y repitiendo el filtro cuad. La respuesta pasará a incluir más de un objeto. El servicio de búsqueda no contiene resultados prefijados; calcula la respuesta a partir del diccionario actual. Al reiniciar la API, ese producto desaparece porque el almacenamiento sigue siendo temporal.

Cuándo un componente necesitaría conocer un filtro

La pantalla actual de Blazor solo consume ObtenerAsync y CrearAsync. El reto de búsqueda se prueba directamente sobre la API mediante el editor HTTP para conservar el alcance obligatorio. Si decidiéramos añadir a la interfaz un campo de búsqueda remota, deberíamos introducir una nueva operación en IProductosApiService y una implementación que formara correctamente la URL con el criterio proporcionado.

Es importante codificar los caracteres de un parámetro de consulta cuando proceden de un usuario, en lugar de concatenar sin más espacios, símbolos o caracteres especiales a la dirección. El cliente dispone de utilidades para preparar componentes de URL y las APIs HTTP cuentan con

mecanismos para representarlos correctamente. En el ejercicio no hace falta desarrollar una biblioteca de construcción de URLs, pero sí identificar el problema.

Tampoco deberíamos alterar el método ObtenerAsync para que comenzara a devolver silenciosamente una búsqueda filtrada. Eso cambiaría el significado de una operación ya utilizada en el catálogo. La separación de responsabilidades que aprendimos en 4.2 nos permite agregar funciones sin romper los métodos anteriores.

Verificar el reto sin modificar el itinerario obligatorio

El código de búsqueda permanece comentado con un marcador de activación en Program.cs del proyecto 4.3. Se puede habilitar en una copia aislada, comprobar el funcionamiento y desactivarlo después. La versión obligatoria del módulo sigue cumpliendo GET de lista y detalle, POST validado y el resumen del punto 4.3.

La ventaja de un reto así no es simplemente añadir líneas, sino reunir varias decisiones que el alumno puede justificar: criterio de consulta frente a identificador en ruta, respuesta vacía frente a 400, filtrado sin mutación, comparación de texto, compatibilidad de DTO y prueba directa mediante solicitudes HTTP.

No obliga a incorporar EF Core, Identity, paginación ni una interfaz adicional. Permite trabajar un poco más con la API que el alumno ya ha construido, sin convertir una jornada de integración Blazor en un curso completo de servicios REST.

4.3.5. Diferencias entre JSON válido, reglas de negocio y persistencia

Una petición puede tener JSON sintácticamente válido y, sin embargo, incumplir las reglas: `{"nombre":"","precio":0}` contiene JSON legible, pero no un producto aceptable. Una petición de JSON incompatible con el DTO puede producir una respuesta de error antes de que el delegado de POST ejecute su comprobación. Nuestro ejemplo no pretende normalizar todos los mensajes de error posibles.

Tampoco basta con que un POST devuelva 201 para afirmar que existe persistencia: **la información vive en memoria del proceso**. Al reiniciar la API se restauran Cuaderno y Lápiz; el producto añadido desaparece. La persistencia ya fue estudiada con EF Core en el contexto previo del alumnado; reintroducirla aquí impediría concentrarse en la comunicación Blazor–API.

El principio de compatibilidad cuando reconstruimos la API

Durante 4.1 y 4.2 el cliente utilizó una API proporcionada. Ahora el alumno construye otra. Para que esa sustitución sea transparente, el cliente debe poder seguir solicitando las mismas rutas, con los mismos verbos y las mismas formas de entrada y salida.

La compatibilidad no significa conservar necesariamente los nombres privados de las clases del servidor. Podemos llamar ProductoApi al modelo dentro del proceso API y ProductoDto al tipo que usa Blazor, siempre que sus campos serializados expresen el contrato esperado.

Tampoco significa que el servidor deba generar los mismos identificadores en todas las ejecuciones. La regla relevante es que cada nueva creación reciba un identificador válido y que el cliente utilice el devuelto. Eso permite separar la semántica de un contrato de detalles concretos de una ejecución en memoria.

Si el nuevo servidor responde correctamente al GET pero devuelve 200 en lugar de 201 para un POST, quizá la UI siga mostrando un elemento, pero habremos cambiado la semántica de la

respuesta. Si devuelve un objeto resumen donde el cliente espera una lista, la deserialización no funciona. Reconocer esas incompatibilidades es parte esencial de la ingeniería de integración.

Leer un archivo de peticiones HTTP como documentación ejecutable

Un archivo .http puede conservar una colección pequeña de solicitudes representativas. Cada bloque especifica un método y una URL, y para POST incluye cabecera Content-Type y cuerpo JSON. Visual Studio permite ejecutar esos bloques individualmente y observar respuesta y cabeceras.

Esto resulta útil en un curso porque evita que el alumno tenga que instalar o programar un cliente distinto para cada escenario. También permite conservar en el repositorio las operaciones del contrato junto al código que las implementa. El archivo no garantiza por sí mismo que todas las respuestas sean correctas: hace posible ejecutar solicitudes reproducibles y compararlas con resultados esperados.

No debemos interpretar una colección de peticiones como un sustituto universal de pruebas automatizadas. Puede ser una excelente herramienta de aprendizaje y diagnóstico, mientras que las pruebas de integración automatizadas aportan comprobaciones repetibles cuando se modifica el código. Ambos enfoques se complementan, pero persiguen objetivos diferentes.

Construir un contrato antes de crear componentes

En un sistema distribuido pequeño, el diseño puede comenzar por cuatro preguntas. ¿Qué necesita consultar el consumidor? ¿Qué datos debe enviar al crear un recurso? ¿Quién asigna los identificadores? ¿Qué estados expresan éxito, rechazo y ausencia?

Para nuestro catálogo respondemos de la siguiente manera. El cliente consulta GET /api/productos y recibe una lista de elementos con id, nombre y precio. Para crear envía POST /api/productos con nombre y precio, sin id. El servidor valida esos campos, asigna el nuevo identificador y devuelve 201 junto al producto completo. Para un ID no encontrado, la consulta individual devuelve 404. Para una entrada incorrecta, POST devuelve 400.

El contrato descrito es suficiente para iniciar los ejercicios. No necesitamos decidir todavía detalles de estructura interna del servidor, nombres de variables o estilo de componentes Razor. Cuando escribimos esos archivos, cada uno debe respetar el acuerdo que ya hemos definido.

Esto ayuda a comprender por qué el cliente no debería depender de compartir exactamente la misma clase C# con la API. Lo que viaja entre procesos son solicitudes y representaciones HTTP. Los tipos locales ayudan a expresar esos datos en cada lenguaje o ensamblado, pero no sustituyen la obligación de mantener compatibilidad.

Observar dos instancias de API sin confundir sus estados

Podemos arrancar la misma implementación de API en dos puertos diferentes, siempre que cada proceso mantenga su propio estado en memoria. Por ejemplo, uno escucha en 5081 y otro en 5088. Si enviamos POST a la primera instancia, la segunda no incorpora automáticamente el producto; no existe una base de datos compartida ni un mecanismo de sincronización entre ambas.

Este experimento ofrece una demostración clara de los límites de proceso. Cambiar ProductosApi:BaseUrl en Blazor determina qué instancia consulta el cliente. Si una muestra Regla y la otra todavía muestra solo Cuaderno y Lápiz, el componente no está funcionando mal: está observando dos servidores distintos.

No debemos utilizar este laboratorio para concluir que dos instancias reales de una aplicación deban funcionar siempre con memorias independientes. Es una característica de nuestra decisión de almacenamiento temporal. En sistemas de producción, las instancias suelen necesitar una estrategia de persistencia y coordinación que no desarrollaremos en este módulo.

Lista de decisiones de implementación y su razón formativa

Decisión	Motivo en esta formación	Qué no pretende resolver
Minimal API	Hacer visibles los endpoints	Arquitectura empresarial completa
DTO de creación distinto	Asignar ID en servidor	Gestión avanzada de entidades
ConcurrentDictionary	Soportar operaciones básicas en memoria	Persistencia o transacciones
Interlocked para ID	Evitar incrementos no atómicos	Identificadores distribuidos
Results.Created	Expresar alta con recurso nuevo	Diseño general de hipermedia
GET de resumen separado	Conservar el catálogo consumido	Versionado completo de APIs
Búsqueda opcional por query	Distinguir filtro y ruta	Motor avanzado de búsquedas
Archivo .http	Ejecutar solicitudes directas	Reemplazar todas las pruebas automatizadas

La tabla no debe entenderse como un conjunto de recetas invariables para cualquier producto. Son decisiones pequeñas, justificadas por el objetivo docente de comprender comunicación HTTP y construcción progresiva sin perder de vista el tiempo disponible. Otras aplicaciones necesitarán soluciones diferentes.

4.3.6. Construcción gradual desde una plantilla vacía

Hasta ahora hemos leído la API suministrada. Para comprender cómo se llega a ese resultado, podemos reconstruirla sin copiar su archivo final. Esta secuencia no supone que el alumno desconozca ASP.NET Core Web API; aprovecha sus conocimientos previos y se centra en las decisiones que afectan al cliente Blazor.

Estado A — el host responde. Desde una terminal distinta de Blazor:

```
dotnet new web -n CursoWeb.ProductosApi -f net8.0
cd CursoWeb.ProductosApi
dotnet run --urls http://localhost:5081
```

El proyecto inicial permite arrancar Kestrel. Antes de repetir peticiones, detén la API preconstruida utilizada en 4.1 para evitar que ambos procesos intenten escuchar en el mismo puerto. El objetivo del primer estado no es devolver productos: es distinguir que el host está iniciado del hecho de que existan endpoints funcionales.

Sustituye el contenido inicial de `Program.cs` por:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
app.MapGet("/", () => Results.Ok("API de Productos disponible"));

app.Run();
```

Si el navegador puede consultar `http://localhost:5081/`, la comunicación básica funciona. Si no funciona, no tiene sentido investigar todavía DTO o serialización.

Estado B — aparecen datos con contrato público. Incorporamos un modelo pequeño y un almacenamiento de memoria. La parte superior del archivo pasa a contener:

```
using System.Collections.Concurrent;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var productos = new ConcurrentDictionary<int, ProductoApi>();
productos[1] = new ProductoApi(1, "Cuaderno", 2.50m);
productos[2] = new ProductoApi(2, "Lápiz", 0.80m);

app.MapGet("/", () => Results.Ok("API de Productos disponible"));
app.MapGet("/api/productos", () =>
    Results.Ok(productos.Values.OrderBy(p => p.Id)));

app.Run();

record ProductoApi(int Id, string Nombre, decimal Precio);
```

Ahora podemos comprobar dos contratos distintos: una respuesta de salud o bienvenida en la raíz y un arreglo de productos en `/api/productos`. Es importante observar la **forma** de la respuesta: Blazor espera una colección de objetos con Id, Nombre y Precio. No bastaría con devolver una cadena de texto que diga «hay dos productos».

Estado C — el servidor distingue un recurso inexistente. Antes de `app.Run`, incorporamos:

```
app.MapGet("/api/productos/{id:int}", (int id) =>
    productos.TryGetValue(id, out var producto)
        ? Results.Ok(producto)
        : Results.NotFound());
```

En este punto tenemos una regla útil para razonar sobre 404: el servidor recibe una petición correctamente formada, pero el diccionario no contiene la clave solicitada. Esto no es un error de conexión; el servidor sigue operativo.

Estado D — el servidor crea productos. Agregamos el segundo contrato, que no contiene identificador:

```
record CrearProductoApi(string Nombre, decimal Precio);
```

Y antes de `app.Run`, la acción:

```
var siguienteId = 2;

app.MapPost("/api/productos", (CrearProductoApi entrada) =>
{
    if (string.IsNullOrWhiteSpace(entrada.Nombre) || entrada.Precio <= 0)
        return Results.BadRequest("Nombre y precio deben ser válidos.");
```

```
    var id = Interlocked.Increment(ref siguienteId);
    var creado = new ProductoApi(id, entrada.Nombre.Trim(), entrada.Precio);
    productos[id] = creado;
    return Results.Created($"/api/productos/{id}", creado);
});
```

El contador comienza en 2 porque los productos iniciales tienen identificadores 1 y 2. `Interlocked.Increment` evita que dos peticiones simultáneas reciban el mismo identificador por una operación de incremento no atómica. No convierte la colección en una base de datos ni garantiza persistencia tras reiniciar. La validación de nombre y precio reside en el servidor incluso aunque el formulario de Blazor ya rechace esos valores.

El resultado de este estado es una API compatible con los GET/POST que nuestro cliente consume. Por ello podemos detener la API suministrada, arrancar la construida por el alumno en el mismo puerto y **volver a utilizar Blazor sin cambiar su interfaz**.

Estado E — extensión de respuesta independiente. Finalmente, podemos añadir el endpoint `/api/productos/resumen` mostrado en el apartado anterior. Devolverá una estructura diferente de la lista, sin alterar las rutas que el servicio tipado necesita. El objetivo no es acumular endpoints, sino entender qué significa extender un contrato público conservando sus consumidores.

Reconstrucción, no copia: el propósito de los estados intermedios

Si entregáramos únicamente el Program.cs final, el alumno podría ejecutarlo sin conocer el orden de decisiones que permitió construirlo. Por eso comenzamos con un host vacío que responde en /, añadimos los modelos, publicamos GET, incorporamos detalle, creamos POST y finalmente ampliamos el servidor con el resumen.

Cada estado debe poder compilar y demostrar un comportamiento único. No adelantamos la creación antes de comprobar la consulta, ni mezclamos la validación de POST con un error del puerto. Este modo de trabajar permite aislar causas y favorece que el estudiante pueda reproducir la API en otra solución de ASP.NET Core.

La habilidad transferible no es recordar todas las líneas de memoria. Es poder definir un contrato pequeño, construir sus endpoints por etapas y verificarlo con un cliente independiente antes de conectarlo a la interfaz principal.

4.3.7. Cómo reconocer tres fallos frecuentes en la construcción

La primera equivocación habitual consiste en ubicar una declaración `record` en medio de las instrucciones de nivel superior de `Program.cs`, de forma que las instrucciones ejecutables posteriores ya no estén permitidas. Los tipos auxiliares pueden declararse al final del archivo, después de `app.Run`, como hemos hecho en los ejemplos.

La segunda consiste en cambiar `GET /api/productos` para devolver una forma JSON diferente mientras Blazor sigue esperando `List<ProductoDto>`. El problema no se resuelve cambiando el puerto: es una incompatibilidad de contrato. Una vía de diagnóstico consiste en leer el JSON directamente y compararlo con el DTO esperado.

La tercera es construir una API nueva y olvidar que la antigua sigue ocupando 5081. Dos procesos no pueden escuchar normalmente en la misma dirección y puerto; uno fallará al arrancar. Antes de investigar HttpClient, verifica qué aplicación está escuchando.

Estas incidencias se estudian aquí porque proporcionan criterios transferibles a otras integraciones: distinguir alojamiento, contrato y estado de aplicación.

Diagnosticar errores habituales de Program.cs

Un error frecuente consiste en colocar declaraciones de tipo antes de las últimas instrucciones de nivel superior; el compilador puede rechazar el orden. Los registros ProductoApi y CrearProductoApi se declaran al final del archivo después de las instrucciones ejecutables, respetando el formato del ejemplo.

Otro error consiste en registrar POST pero olvidar poner la variable siguienteId en el alcance correcto. La función de creación captura ese contador, que debe persistir mientras dure el proceso. Si se declara nuevamente en cada petición, el supuesto identificador secuencial dejaría de representar una colección compartida.

También es común confundir el puerto de escucha con la ruta. Cambiar la ruta a /api/catalogo mientras el cliente continúa solicitando /api/productos rompe el contrato; cambiar el puerto sin actualizar la configuración impide alcanzar la API. Son dos problemas distintos y requieren comprobaciones diferentes.

Por último, puede ocurrir que el servidor acepte un precio decimal pero el cuerpo POST esté escrito con un formato incompatible con JSON, por ejemplo una coma decimal sin comillas. El error puede producirse antes de entrar a la lógica de validación. Conviene distinguir errores de sintaxis del formato, errores de enlace al tipo y errores de reglas del dominio.

4.3.8. Probar el contrato sin depender de la pantalla

Para diagnosticar una integración conviene separar observaciones. Primero ejecutamos la API y probamos GET, POST y errores con un cliente HTTP independiente. Después abrimos Blazor para comprobar que conserva su comportamiento. Si el GET directo funciona, pero la página falla, investigaremos URL configurada, puerto, servicio tipado, DTO o estado del componente. Si el GET directo también falla, examinaremos el proceso API y sus rutas.

En este punto **no** implantamos autenticación sobre el catálogo; la demostración limitada de seguridad corresponde a 4.4. Una API sin protección no se vuelve segura por el mero hecho de que la interfaz oculte un botón. La autorización debe comprobarse en el servidor del recurso protegido.

Probar el contrato sin navegador: razonamiento sobre la evidencia

Podemos comprobar un endpoint directamente con una herramienta HTTP. Esa prueba reduce el número de piezas implicadas. Si un GET directo responde 200 con la lista esperada, sabemos que el servidor está disponible y que ese endpoint satisface al menos el caso ensayado. No demuestra que Blazor esté configurado correctamente ni que todas las entradas posibles funcionen, pero sí aporta una evidencia localizada.

Por el contrario, si el botón de Blazor no crea un producto, todavía no sabemos si el problema está en el formulario, en el servicio HTTP, en la configuración de URL, en el endpoint o en los datos. Ejecutar un POST independiente permite comprobar una parte importante del recorrido antes de modificar componentes.

En un entorno docente es útil elegir casos con resultados exactos: GET inicial con dos productos, consulta ID 1 con Cuaderno, consulta ID 999 con 404, POST de Regla por 1,75 con 201 y un nuevo ID, POST de precio cero con 400. Después puede observarse que el listado ha aumentado y que al reiniciar el proceso vuelve a su estado inicial.

Estas pruebas no convierten la aplicación de laboratorio en un producto validado para producción. Su valor es la trazabilidad entre un requisito observable y la respuesta HTTP real de la implementación.

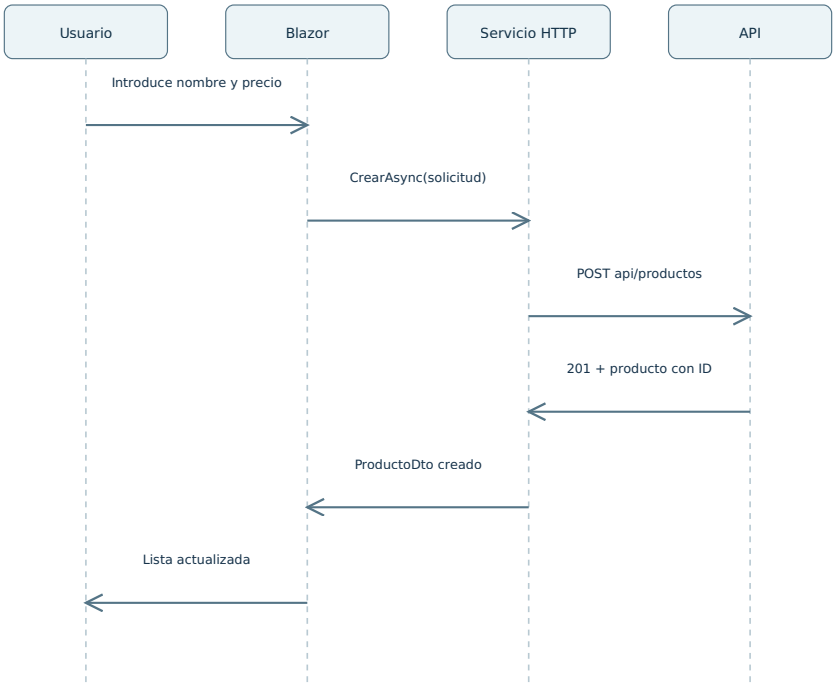
Verificaciones después de construir la API desde cero

La secuencia de comprobación debe seguir los hitos de construcción. Primero, el host responde en /. Segundo, el listado inicial devuelve dos productos. Tercero, un identificador existente devuelve su recurso y uno inexistente produce 404. Cuarto, un POST válido crea un objeto con ID y 201. Quinto, un POST inválido produce 400 sin modificar el número de productos. Sexto, el endpoint de resumen refleja los valores actuales cuando se activa.

Solo después iniciamos Blazor y sustituimos la API que se entregaba al principio de 4.1. Si ambas implementaciones satisfacen el mismo contrato, el cliente puede seguir funcionando sin alterar su código. Esta última comprobación es especialmente reveladora porque une 4.1, 4.2 y 4.3: no basta con construir endpoints, deben respetar los acuerdos de la aplicación consumidora.

La herramienta HTTP y el proyecto Blazor ofrecen evidencias distintas. Una consulta directa demuestra la respuesta concreta del servidor para un caso. Una consulta desde Blazor demuestra además que el servicio tipado, sus DTO y el componente colaboran correctamente. Las dos son necesarias para localizar problemas con precisión.

Diagrama de secuencia de una creación completa



Este diagrama muestra la ruta de éxito. En la primera etapa, los campos del formulario pertenecen a Blazor. En la segunda, la solicitud pasa a la interfaz del servicio. En la tercera, la implementación transforma una llamada C# en HTTP. En la cuarta, la API valida, asigna un ID y devuelve el recurso creado. Finalmente, el componente añade ese objeto a la lista y actualiza la presentación.

Podemos comparar ese recorrido con el de una creación rechazada. Si el servidor devuelve 400, el servicio no obtiene un ProductoDto creado; su comprobación de estado genera una excepción que el componente transforma en mensaje de error. La lista no debería añadir un producto que el

servidor nunca confirmó. La diferencia entre ambas rutas no está en el botón, sino en la respuesta que la API emitió.

Ampliación 4.3 — Un recurso también puede modificarse y eliminarse

El recorrido esencial del curso se limita a consultar y crear. Aun así, para comprender qué significa un contrato REST completo conviene estudiar cómo evolucionaría la API si el catálogo tuviera que aceptar cambios sobre recursos existentes. Esta ampliación es de lectura y experimentación autónoma: no forma parte de los cuarenta y cinco minutos inicialmente reservados a la construcción mínima del servidor.

Hasta ahora identificábamos un producto mediante su ID y obteníamos su representación con GET /api/productos/{id}. Para modificarlo necesitamos decidir qué recurso recibe la operación, qué datos nuevos son admisibles y cuál será la respuesta si ese recurso ya no existe. Es importante separar esas decisiones del formulario Blazor: la API también podría recibir una actualización desde PowerShell, una aplicación de escritorio o un cliente escrito en otro lenguaje.

Un contrato de actualización sin confundir el identificador con los datos

Para este ejemplo elegimos PUT /api/productos/{id}. El identificador aparece en la ruta, mientras que los nuevos atributos editables viajan en el cuerpo. Definimos un tipo de entrada que no incorpora el ID:

```
record ActualizarProductoApi(string Nombre, decimal Precio);
```

¿Por qué no reutilizamos directamente ProductoApi, que sí incluye Id? Porque no queremos plantear dos identificadores potencialmente distintos, uno en la URL y otro en el JSON. La URL designa el recurso que vamos a modificar y el cuerpo describe el nuevo nombre y precio. Un contrato diferente podría exigir un identificador también en el cuerpo y comprobar su coincidencia, pero eso añadiría una regla que no necesitamos aquí.

El cliente envía, por ejemplo:

```
PUT /api/productos/1 HTTP/1.1
Host: localhost:5081
Content-Type: application/json

{"nombre":"Cuaderno A4","precio":3.20}
```

Si la operación es correcta, devolvemos 200 y el objeto actualizado. Elegimos esa respuesta porque facilita observar el resultado desde un cliente HTTP; también existen diseños que responden 204 sin cuerpo, pero nuestro caso mantiene la continuidad pedagógica con POST y GET.

Implementar PUT a partir de los contratos existentes

En la API de memoria disponemos de un diccionario por ID. Debemos comprobar la entrada, verificar que existe un recurso y actualizarlo. Para un laboratorio que no aborda transacciones complejas, la solución puede escribirse de manera contenida:

```
app.MapPut("/api/productos/{id:int}",
    (int id, ActualizarProductoApi entrada) =>
{
    if (string.IsNullOrWhiteSpace(entrada.Nombre) ||
        entrada.Precio <= 0)
```

```
        return Results.BadRequest("Nombre y precio deben ser válidos.");

    if (!productos.ContainsKey(id))
        return Results.NotFound();

    var actualizado = new ProductoApi(
        id, entrada.Nombre.Trim(), entrada.Precio);
    productos[id] = actualizado;
    return Results.Ok(actualizado);
});
```

Este código permite explicar el recorrido de una solicitud y los tres estados de salida. Una entrada inválida se rechaza con 400. Un identificador desconocido produce 404. Una actualización correcta devuelve 200 y el producto con el mismo ID. La modificación afecta a la representación que GET devolverá posteriormente.

Hay, sin embargo, una limitación concreta de concurrencia que no debemos ocultar. Entre comprobar ContainsKey y asignar el nuevo valor, otra operación podría haber eliminado el producto. Si necesitáramos que «actualizar solo si todavía existe» fuera una operación atómica, deberíamos utilizar un mecanismo que garantizara esa condición. En el mismo ConcurrentDictionary existe TryUpdate, pero requiere comparar el valor anterior. Aquí dejamos visible la limitación del ejemplo básico en vez de afirmar que proporciona garantías que no se han implementado.

Podemos mejorar ese punto con un bucle de actualización condicional:

```
app.MapPut("/api/productos/{id:int}/condicional",
    (int id, ActualizarProductoApi entrada) =>
{
    if (string.IsNullOrWhiteSpace(entrada.Nombre) ||
        entrada.Precio <= 0)
        return Results.BadRequest("Datos de actualización inválidos.");

    while (productos.TryGetValue(id, out var anterior))
    {
        var nuevo = new ProductoApi(
            id, entrada.Nombre.Trim(), entrada.Precio);

        if (productos.TryUpdate(id, nuevo, anterior))
            return Results.Ok(nuevo);
    }
    return Results.NotFound();
});
```

Esta variante compara el valor observado con el valor que todavía existe en el diccionario. Si otro consumidor modificó el producto antes, el bucle vuelve a leer y reintenta con la nueva instantánea. No constituye por sí sola un sistema general de control de versiones de entidades: simplemente muestra cómo pensar sobre una operación concurrente al utilizar memoria compartida.

En la práctica adicional podremos elegir una de las dos versiones como ejercicio de razonamiento. No se activan simultáneamente dos rutas de actualización para sustituir un contrato obligatorio: la ruta condicional utiliza una dirección separada.

Eliminar un recurso con DELETE

Para eliminarlo publicamos DELETE /api/productos/{id}. El cuerpo de solicitud no necesita describir de nuevo el producto: la URL identifica lo que queremos quitar.

```
app.MapDelete("/api/productos/{id:int}", (int id) =>
    productos.TryRemove(id, out _)
        ? Results.NoContent()
        : Results.NotFound());
```

TryRemove comprueba y elimina una entrada como operación del diccionario. Cuando encuentra el recurso, elegimos 204 No Content. El cliente sabe que se ha completado la eliminación sin esperar un objeto en la respuesta. Si el recurso ya no existe, este contrato devuelve 404.

El significado de idempotencia merece una explicación precisa. Que DELETE sea idempotente significa que repetir una operación debería dejar el servidor en el mismo estado final que ejecutarla una sola vez; no exige necesariamente que todas las respuestas HTTP tengan el mismo código. En nuestro ejemplo, la primera eliminación de ID 1 produce 204 y la segunda produce 404. Tras ambas, el diccionario continúa sin contener ese ID.

Con POST de creación ocurre algo diferente: repetir la petición puede producir dos recursos, cada uno con un nuevo identificador. Por eso no debemos aplicar indiscriminadamente reintentos automáticos de POST sin diseñar cómo evitaremos duplicaciones.

Comparar las cuatro operaciones sobre el mismo recurso

Operación	Qué expresa	Entrada principal	Resultado habitual
GET /api/productos	Consulta colección	Ninguna	200 con arreglo
GET /api/productos/1	Consulta detalle	ID en ruta	200 o 404
POST /api/productos	Crea recurso	Nombre y precio	201 o 400
PUT /api/productos/1	Modifica recurso	ID y nuevos valores	200, 400 o 404
DELETE /api/productos/1	Elimina recurso	ID en ruta	204 o 404

La tabla es una simplificación del contrato que hemos elegido para el laboratorio. No afirma que todas las APIs reales deban utilizar exactamente esas respuestas en cualquier situación. Su propósito es que el alumno reconozca cómo método, ruta, entrada y estado cooperan para describir una operación.

Cómo afecta una ampliación del servidor al cliente tipado

El servicio actual de Blazor solo expone ObtenerAsync y CrearAsync, y eso es correcto porque son las operaciones requeridas por la pantalla. Que el servidor incorpore PUT y DELETE no obliga al cliente a empezar a utilizarlas: ampliar una API no implica actualizar todos sus consumidores.

Si más adelante quisiéramos ofrecer edición remota, podríamos definir en IProductosApiService una operación ActualizarAsync que recibiera ID y datos. La implementación de infraestructura utilizaría PutAsJsonAsync o enviaría una solicitud equivalente, y el componente decidiría cuándo mostrar el resultado. El punto clave es conservar la separación aprendida: la UI no introduce rutas HTTP directamente porque necesita un nuevo botón.

De forma análoga, un método EliminarAsync podría ejecutar DELETE y comprobar si el estado es 204, 404 u otro resultado. El componente mostraría una confirmación solo después de recibir una respuesta satisfactoria y actualizaría su lista a partir de la operación confirmada.

No estamos obligados a construir aquí una pantalla de administración completa. El alumno debe entender qué nuevo contrato sería necesario, qué tipos circularían entre capas y por qué los controles visuales no sustituyen las validaciones del servidor.

Recurso inexistente, entrada inválida y conflicto son conceptos diferentes

Imaginemos que se recibe PUT con precio cero sobre un ID que no existe. Hemos escrito la validación antes de la búsqueda, de modo que la respuesta será 400 por entrada inválida. Si quisiéramos priorizar 404, tendríamos que cambiar el orden de las comprobaciones y definir explícitamente esa regla en el contrato.

Ninguna de esas elecciones es una consecuencia automática de HTTP. Son decisiones del diseño de la API que los consumidores necesitan conocer. Lo importante es que las respuestas sean coherentes, estén documentadas y puedan comprobarse.

Un problema distinto aparecería si dos usuarios editaran un recurso basándose en versiones distintas. En un sistema real podrían utilizarse tokens de concurrencia, ETag o condiciones como If-Match, que permiten prevenir la pérdida silenciosa de cambios. No incorporamos ese mecanismo completo al módulo, pero sí identificamos la razón de su existencia: comprobar que el estado sobre el que actuamos sigue siendo el esperado.

La enseñanza formativa de esta ampliación es aprender a hacer visibles esas preguntas antes de escribir una solución. Un endpoint mínimo puede compilar y, aun así, no tener definidas las garantías de concurrencia que una aplicación de negocio necesitaría.

Cierre de la ampliación: qué se debe conservar

Después de estudiar PUT y DELETE, el contrato obligatorio del módulo sigue intacto: GET de catálogo y detalle, POST de creación y las ampliaciones de resumen y búsqueda. El nuevo código no obliga a modificar CatalogoRemoto.razor ni el servicio utilizado por la jornada presencial. Esta independencia es una señal de que la API ha podido crecer mediante operaciones nuevas sin romper el trabajo realizado.

El alumno debe poder explicar por qué el ID forma parte de la ruta de modificación, por qué el objeto de entrada carece de ID, por qué DELETE puede devolver 204, por qué una actualización concurrente necesita más atención y por qué el componente Blazor no incorpora directamente detalles del protocolo.

Ampliación 4.3 — Diseñar respuestas de validación que el cliente pueda comprender

Nuestra primera API devuelve una cadena sencilla cuando el POST contiene un nombre vacío o un precio no positivo. Este contrato resulta suficiente para observar 400 Bad Request durante una introducción a HTTP. Sin embargo, si una aplicación tuviera varios campos y necesitara mostrar la razón exacta de cada error, un texto arbitrario no ofrecería una estructura fácil de procesar.

Podemos mejorar el razonamiento mediante un formato de problema estructurado. ASP.NET Core permite producir respuestas de validación con una representación que identifica los campos afectados y sus mensajes. El objetivo de esta ampliación no es introducir un framework de formularios adicional: es comprender cómo se diseña un **contrato de error** del mismo modo que se diseña un contrato de éxito.

El límite de un mensaje de error no estructurado

En la primera versión, la API podía responder así:

```
HTTP/1.1 400 Bad Request
Content-Type: text/plain

Nombre y precio deben ser válidos.
```

Ese mensaje informa de que la petición fue rechazada. Pero si el formulario tiene un nombre correcto y un precio incorrecto, el cliente no puede identificar de forma fiable cuál de los campos provocó el rechazo a partir de un contrato de texto libre, salvo que intente interpretar palabras de un idioma.

Tampoco sería una buena decisión construir la UI buscando literalmente «precio» en una frase devuelta por el servidor. Si mañana cambiamos el texto, corregimos una traducción o incluimos un nuevo campo, esa lógica podría romperse aunque el contrato funcional siguiera siendo el mismo.

Por eso conviene separar el código HTTP, que expresa la categoría de resultado, y el cuerpo estructurado, que explica qué datos no cumplieron las reglas.

Representar errores por campo

Podemos preparar un diccionario cuya clave sea el nombre del campo y cuyo valor sea la lista de errores que le corresponden:

```
var errores = new Dictionary<string, string[]>();

if (string.IsNullOrWhiteSpace(entrada.Nombre))
    errores["nombre"] = ["El nombre es obligatorio."];

if (entrada.Precio <= 0)
    errores["precio"] = ["El precio debe ser positivo."];
```

Las dos condiciones se evalúan independientemente. Así, una solicitud con nombre vacío y precio cero puede producir dos errores distintos. En una interfaz de formulario, el componente podría relacionar cada entrada con su mensaje correspondiente.

En .NET 8, una Minimal API puede utilizar Results.ValidationProblem para construir una respuesta HTTP de validación:

```
if (errores.Count > 0)
    return Results.ValidationProblem(errores);
```

Esta solución expresa un contrato más preciso que una única cadena: si existe un error de nombre, aparece bajo la clave nombre; si el precio es inválido, aparece bajo precio. El servidor sigue siendo quien aplica ambas reglas.

Un endpoint alternativo completo para estudiar ese contrato

En una copia de ensayo de la API podemos sustituir temporalmente el POST inicial por esta variante:

```
app.MapPost("/api/productos", (CrearProductoApi entrada) =>
{
```

```
    var errores = new Dictionary<string, string[]>();

    if (string.IsNullOrWhiteSpace(entrada.Nombre))
        errores["nombre"] = ["El nombre es obligatorio."];

    if (entrada.Precio <= 0)
        errores["precio"] = ["El precio debe ser positivo."];

    if (errores.Count > 0)
        return Results.ValidationProblem(errores);

    var id = Interlocked.Increment(ref siguienteId);
    var creado = new ProductoApi(
        id, entrada.Nombre.Trim(), entrada.Precio);
    productos[id] = creado;
    return Results.Created($"/api/productos/{id}", creado);
});
```

La ruta y el método no cambian. Las entradas correctas continúan creando un producto y devolviendo 201 con el recurso completo. La diferencia se encuentra únicamente en la forma del error cuando la entrada no es válida. Por ello un cliente que solo consume el camino de éxito puede seguir funcionando, mientras que un cliente que desee mostrar errores específicos dispone ahora de información estructurada.

No registres las dos variantes de POST para la misma ruta simultáneamente. Son implementaciones alternativas de un contrato que queremos comparar, no endpoints diferentes.

¿Qué observa el alumno al enviar un nombre vacío?

Imaginemos que enviamos:

```
{
  "nombre": "",
  "precio": 4.50
}
```

El manejador encuentra un error para nombre y ninguno para precio. La respuesta de validación tendrá estado 400 y un cuerpo de problema que incluye el campo errors con la clave nombre. El servidor no incrementa el contador de identificadores ni añade un recurso al diccionario.

Si enviamos nombre Regla y precio cero, aparecerá solamente el error de precio. Si ambos datos son inválidos, podremos observar mensajes para las dos claves. Esto hace posible comprobar la independencia de las reglas.

La estructura completa puede incluir miembros estándar como type, title, status y errors. El alumno no debe depender de una serialización textual exacta del orden de esas propiedades; lo importante es la semántica del estado y la presencia de las claves que hemos diseñado.

¿Por qué una respuesta de error no puede convertirse en ProductoDto?

El método CrearAsync de ProductosApiService espera un producto válido cuando POST tiene éxito. Por eso ejecuta EnsureSuccessStatusCode antes de ReadFromJsonAsync. Si la API responde con ValidationProblem, la llamada debe reconocer el rechazo y no intentar tratar el cuerpo como si fuera un producto.

Para desarrollar una interfaz que muestre los mensajes por campo tendríamos que definir cómo leer el cuerpo de error y mapearlo al estado de validación del formulario. Esa evolución pertenece a

un requisito adicional: el ejemplo básico conserva un mensaje general y la API puede seguir aplicando sus reglas de forma independiente.

No conviene confundir un error de formato JSON mal formado con una validación de negocio. Si la petición no puede enlazarse a CrearProductoApi, puede ser rechazada antes de ejecutar la lógica del endpoint. Si se enlaza correctamente pero contiene nombre o precio no válidos, es el código del manejador el que produce ValidationProblem.

Comprobar efectos secundarios ante entradas rechazadas

Una respuesta 400 no es suficiente por sí sola para afirmar que los datos no cambiaron. La implementación debe devolver el error **antes** de llamar a Interlocked.Increment y antes de asignar un elemento al diccionario.

Para comprobarlo, comenzamos con dos productos, enviamos un POST inválido y después consultamos GET /api/productos. La colección debe seguir teniendo dos elementos. Al enviar un POST válido a continuación, se asignará el primer identificador de creación correspondiente al estado inicial.

Esta comprobación conecta dos propiedades: el contrato observable indica rechazo y la regla de implementación impide que una solicitud rechazada modifique el estado.

Diferenciar una API clara de una API extensa

El objetivo de la variante no es añadir rutas por volumen. De hecho, podemos mejorar la precisión del contrato sin aumentar el número de endpoints. La decisión importante está en cómo representamos la información de error y qué puede hacer el cliente con ella.

Un servicio bien diseñado no necesita exponer todos los detalles internos del programa. Puede devolver un problema de validación comprensible sin revelar una traza de excepción, rutas del sistema de archivos ni información sensible del servidor. El formato de error se convierte así en parte del contrato público, no en una copia accidental de los mensajes internos de .NET.

Relación con los formularios de M02

El alumnado ya ha trabajado validación de entradas en la parte web del curso. Ahora puede reconocer que una validación de presentación y una validación de API no se sustituyen. El formulario ayuda a introducir datos correctos; la API rechaza los que incumplen sus reglas, independientemente del cliente que los envía.

Si en el futuro necesitáramos mostrar mensajes exactos por campo desde una API, podríamos convertir el contrato de error en información de validación visual. La interfaz seguiría sin determinar las reglas definitivas del servidor.

La competencia formativa de esta ampliación consiste en poder explicar por qué 400 y su cuerpo representan cosas distintas, cómo se estructuran errores por campo, por qué se valida antes de modificar memoria y por qué el cliente no interpreta un problema de validación como un producto creado.

Síntesis del punto 4.3

Un endpoint es una asociación entre método HTTP, ruta y ejecución de código. Los DTO establecen el contrato; los resultados HTTP indican éxito o rechazo; el estado en memoria explica la duración

de los datos. Hemos añadido una operación de resumen sin romper las rutas existentes, y disponemos de una API reproducible que se puede observar tanto directamente como desde Blazor.

4.4. Autenticación, autorización y AuthorizeView

Identificar al usuario no es darle permisos

Autenticación responde quién es el usuario. **Autorización** responde qué puede hacer ese usuario ya identificado. En ASP.NET Core la identidad puede representar datos mediante **claims**: afirmaciones como el nombre o un rol. Un rol de ejemplo, `Formador`, servirá para observar la autorización sin construir una infraestructura de cuentas.

La página Blazor de 4.4 demostrará estos conceptos con una cookie de sesión **solo en el laboratorio local**. No implantaremos ASP.NET Core Identity ni JWT, OAuth u OpenID Connect; mencionar Identity permitirá comprender dónde encaja la gestión real de usuarios y contraseñas en una aplicación profesional. El código fijo de demostración **no es una credencial válida para producción**.

4.4.1. Configurar una autenticación de demostración

ASP.NET Core permite registrar autenticación por cookie y autorización con:

```
builder.Services.AddAuthentication("CookiesDemo")
    .AddCookie("CookiesDemo", opciones =>
    {
        opciones.LoginPath = "/seguridad";
    });
builder.Services.AddAuthorization();
builder.Services.AddCascadingAuthenticationState();
```

Tras construir la aplicación se usan `app.UseAuthentication()` y `app.UseAuthorization()`, antes de mapear los recursos protegidos. La autenticación examina la cookie enviada por el navegador. La autorización aplica reglas usando el principal autenticado. `AddCascadingAuthenticationState` hace disponible ese estado a los componentes Razor.

Para construir una identidad de prueba:

```
var claims = new[]
{
    new Claim(ClaimTypes.Name, "AlumnoDemo"),
    new Claim(ClaimTypes.Role, "Formador")
};
var identidad = new ClaimsIdentity(claims, "CookiesDemo");
var principal = new ClaimsPrincipal(identidad);
await contexto.SignInAsync("CookiesDemo", principal);
```

La cookie debe emitirse en una respuesta HTTP normal. Una operación `SignInAsync` dentro de un evento de Blazor Interactive Server que ya ha iniciado su respuesta no es el enfoque apropiado; por ello haremos el acceso mediante un formulario HTML POST y una redirección de página completa.

Autenticación por cookie: seguir el intercambio completo

Cuando el usuario completa correctamente el formulario de demostración, el navegador envía una petición HTTP POST al servidor Blazor. El servidor compara el código local del laboratorio, crea una identidad con claims y ejecuta `SignInAsync`. En la respuesta aparece una cookie de autenticación; en solicitudes posteriores el navegador la envía al servidor correspondiente. El middleware vuelve a interpretar esa cookie y crea el principal asociado al usuario.

Es útil distinguir tres instantes. **Antes de entrar**, el navegador no presenta una cookie reconocida y el principal es anónimo. **Después del POST correcto**, la respuesta establece la sesión y redirige a la página de demostración. **Después de salir**, el servidor solicita al navegador que elimine o invalide la cookie y una navegación posterior vuelve a producir una identidad anónima.

Esto explica por qué no debemos intentar emitir una cookie dentro de un evento cualquiera de un componente interactivo cuando ya se ha iniciado la respuesta HTTP. El formulario POST convencional facilita que el servidor controle la respuesta y sus cabeceras.

En el laboratorio la autenticación ocurre en el servidor Blazor. No implica que el proceso de la API REST auxiliar acepte automáticamente esa misma identidad ni que comparta el mismo esquema de cookies. Autenticación y autorización deben diseñarse explícitamente en los límites de cada aplicación.

Del navegador a la identidad: seguir el primer acceso

Antes de comenzar la autenticación, la aplicación Blazor puede dibujar páginas y consumir la API auxiliar. El navegador no presenta una cookie de sesión reconocida por el esquema de demostración. En ese estado, el principal es anónimo y AuthorizeView muestra la rama NotAuthorized. Esta vista condicional no decide todavía el permiso de un recurso remoto; solo permite presentar información adecuada al estado de autenticación conocido por el componente.

El usuario escribe el código de laboratorio en un formulario HTML y pulsa acceder. La petición POST se envía a /demo/entrar, un endpoint configurado en el mismo host de Blazor. El servidor recibe los campos, comprueba el token antifalsificación y compara el código de demostración. Si la comparación no se satisface, no emite una cookie válida y redirige a la página de acceso. Si el código es correcto, construye un ClaimsPrincipal a partir de una ClaimsIdentity con nombre AlumnoDemo y rol Formador.

La llamada SignInAsync solicita al middleware de cookie que emita la información de autenticación en la respuesta HTTP. Una vez devuelta esa respuesta, el navegador guarda la cookie según sus atributos y la envía en posteriores peticiones compatibles con su ámbito. Así, cuando el usuario vuelve a /seguridad, la aplicación puede reconocer el principal y mostrar la rama Authorized.

La cookie no es una variable global accesible a cualquier proceso del ordenador. La API auxiliar es otra aplicación y no queda automáticamente protegida por el hecho de que el usuario haya entrado en la pantalla Blazor. Para proteger una operación del servidor remoto, esa API necesitaría su propio esquema y sus políticas. No construiremos tal infraestructura dentro del alcance de esta jornada.

¿Por qué no hacemos el inicio desde un evento interactivo?

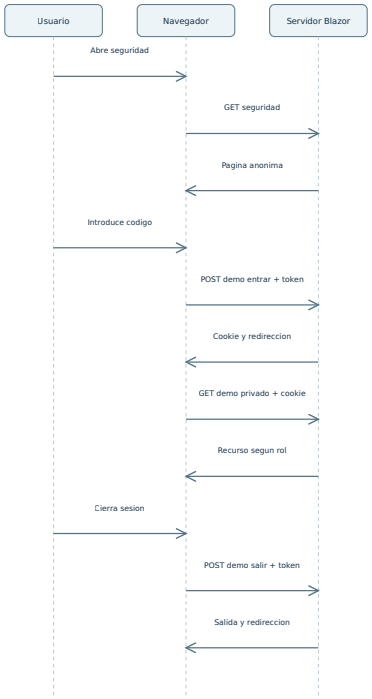
Blazor Interactive Server mantiene un circuito para procesar eventos y actualizar componentes. Cuando un evento se ejecuta, el servidor no dispone necesariamente de una respuesta HTTP normal cuyas cabeceras pueda modificar. Emitir una cookie requiere establecer una cabecera en una respuesta HTTP antes de que se haya enviado su contenido.

Intentar resolver esa discrepancia con un método SignInAsync invocado ingenuamente desde el evento de un botón interactivo puede provocar errores relacionados con cabeceras ya iniciadas. El laboratorio evita esa confusión mediante formularios HTML normales que se envían a endpoints POST del servidor. La redirección posterior recarga la página y establece un nuevo contexto de navegación.

No debemos concluir que Blazor es incompatible con la autenticación moderna; simplemente estamos eligiendo un flujo didáctico que hace observable dónde se crea una cookie. Una aplicación real puede integrar Identity y otros proveedores mediante configuraciones apropiadas y componentes preparados para ello, pero esas opciones quedan deliberadamente fuera del ejercicio.

Secuencia completa de la sesión y lugar de cada decisión

Para cerrar el estudio de autenticación conviene representar las transiciones como un intercambio de solicitudes y respuestas. El formulario se dibuja en un componente, pero quien emite la cookie es el servidor, y quien la presenta en solicitudes posteriores es el navegador. El rol no se decide en el HTML: se almacena como un claim asociado al principal y se comprueba por la política del endpoint.



El esquema sirve para localizar cuándo se transmite el token antifalsificación, cuándo se emite la cookie y cuándo se evalúa la autorización. No significa que el servidor acepte cualquier POST con token: debe comprobar también el código del laboratorio antes de establecer una identidad.

Consideremos el primer GET. El navegador abre /seguridad y el servidor entrega una página que permite iniciar sesión. Todavía no hay un principal reconocido, de modo que AuthorizeView puede mostrar NotAuthorized.

En el POST de acceso, el navegador envía un formulario. El servidor comprueba el token antifalsificación y después la entrada de demostración. Si el código es incorrecto, no emite una cookie de autenticación. Si es correcto, construye el principal con nombre y rol y prepara la respuesta con la cookie.

En la siguiente consulta del recurso privado, el navegador presenta la cookie válida. El middleware identifica al usuario y la política exige el rol Formador. Solo si ambas condiciones se satisfacen se devuelve el contenido protegido. Si el usuario es anónimo o tiene un rol distinto, no debe recibir esa respuesta de éxito.

Por último, la salida utiliza un POST protegido por antiforgery. SignOutAsync hace que el navegador deje de disponer de una autenticación válida para esa sesión. Una nueva consulta del recurso protegido debe volver a ser denegada.

El punto esencial es que cada etapa sucede en un límite distinto: representación en el componente, petición en el navegador, cookie y políticas en el servidor. Los nombres concretos de las clases pueden variar, pero esas responsabilidades siguen existiendo.

4.4.2. Proteger recursos en el servidor

Un endpoint protegido es distinto de un botón oculto:

```
app.MapGet("/demo/privado", () =>
    Results.Ok(new { mensaje = "Recurso protegido" }))
    .RequireAuthorization(p => p.RequireRole("Formador"));
```

La autorización se evalúa en el servidor. Aunque se conozca la URL, el usuario anónimo no recibe el contenido. Una identidad autenticada sin `Formador` tampoco recibe autorización. La API de Productos utilizada en 4.1–4.3 permanece deliberadamente pública: aquí no añadimos credenciales a todo el proyecto.

¿Qué diferencia hay entre 401 y 403?

La semántica es importante aunque algunas aplicaciones web redirijan al formulario de acceso. **401 Unauthorized** indica que no existe una autenticación válida para acceder al recurso. **403 Forbidden** corresponde a una identidad reconocida que carece del permiso exigido. Con autenticación por cookie, los manejadores pueden convertir esas situaciones en redirecciones a páginas de entrada o de acceso denegado. En un endpoint consultado como API puede ser preferible conservar los códigos HTTP para que el cliente los interprete.

Nuestro endpoint de laboratorio requiere el rol `Formador`. Una persona anónima no satisface esa política. Un principal autenticado con otro rol tampoco, pero la razón es diferente: ya sabemos quién es y lo que falta es autorización.

En pruebas, no basta con ver que un botón está oculto: debemos consultar directamente el endpoint y comprobar que el servidor rechaza la solicitud cuando falta la identidad o el permiso.

La autorización es una segunda decisión

Un usuario autenticado no recibe automáticamente permiso para cualquier recurso. En nuestra aplicación, /demo/privado requiere el rol Formador mediante RequireRole. La comprobación ocurre en el servidor, de manera que escribir directamente la URL no evita la política.

Supongamos tres identidades. La primera es anónima y carece de sesión válida. La segunda está autenticada con rol Alumno. La tercera está autenticada con rol Formador. Solo la tercera satisface la regla definida para el recurso privado. Este experimento permite observar la diferencia entre demostrar quién eres y demostrar que puedes ejecutar una operación determinada.

En general, 401 se asocia a una autenticación inexistente o no válida, mientras que 403 representa una identidad conocida a la que no se concede permiso. En aplicaciones web con cookies pueden producirse redirecciones a páginas de acceso o denegación en lugar de entregar directamente esos códigos. Para el endpoint HTTP del laboratorio configuramos explícitamente una respuesta 401 cuando se solicita de manera anónima, de modo que la observación sea sencilla.

Roles, claims y políticas

Un claim expresa una afirmación sobre el principal, como su nombre o un valor de rol. Un rol es una clasificación que puede utilizarse en reglas de autorización. En una aplicación real, el diseño de permisos no siempre puede reducirse a roles; algunas reglas dependen de recursos, operaciones y contexto. Nuestro caso solo necesita demostrar el mecanismo básico de comprobación de una afirmación.

Al crear el principal de laboratorio incluimos ClaimTypes.Name y ClaimTypes.Role. Al registrarlo en la cookie, la aplicación puede recuperar posteriormente esos datos para tomar decisiones. En la presentación los usamos para mostrar AlumnoDemo y reconocer Formador. En el endpoint usamos RequireRole para una regla efectiva.

Si reemplazamos temporalmente Formador por un rol diferente en la política, la identidad puede seguir autenticada y la pantalla puede seguir mostrando el nombre, pero el recurso privado ya no estará autorizado. Esa observación es más formativa que limitarse a mostrar un mensaje de éxito al iniciar sesión.

Analizar un recurso protegido desde tres perfiles

Imaginemos tres situaciones independientes. En la primera no existe cookie reconocida. En la segunda existe una sesión con nombre AlumnoDemo pero sin el rol Formador. En la tercera existe una sesión con nombre AlumnoDemo y con ese rol.

Situación	Autenticación	Autoriza RequireRole("Formador")
Navegador anónimo	No	No
Usuario reconocido con otro rol	Sí	No
Usuario reconocido con Formador	Sí	Sí

La tabla ayuda a explicar que autenticación y autorización no son sinónimos. En particular, la segunda fila demuestra que conocer la identidad no significa permitir automáticamente cualquier operación. En aplicaciones de negocio reales es común que usuarios reconocidos tengan permisos distintos sobre recursos concretos.

La interfaz puede reflejar esas distinciones mediante AuthorizeView, pero las decisiones efectivas deben ser comprobadas por el servidor. Si el componente no dibuja el enlace al recurso privado, una persona todavía podría escribir la dirección manualmente; por eso la política del endpoint es la que protege los datos.

4.4.3. AuthorizeView: presentación condicional, no seguridad del servidor

```
<AuthorizeView>
    <Authorized>
        <p>Sesión: @context.User.Identity?.Name</p>
    </Authorized>
```

```
    <NotAuthorized>
        <p>La sesión no está autenticada.</p>
    </NotAuthorized>
</AuthorizeView>
```

`AuthorizeView Roles="Formador"` muestra contenido solo si los claims del usuario satisfacen esa condición. Eso cambia la **interfaz visible**; no reemplaza la regla `RequireAuthorization` del servidor. Para páginas protegidas con `[Authorize]` puede emplearse `AuthorizeRouteView` al resolver las rutas, pero el recurso de demostración será el endpoint HTTP protegido.

AuthorizeView no es un cortafuegos

AuthorizeView permite expresar en Razor que una parte del contenido se representa solo para un usuario reconocido o para un rol concreto. Eso mejora la experiencia: puede ocultar controles que el usuario no podrá utilizar y mostrar mensajes de acceso. Sin embargo, ni ocultar un botón ni omitir una sección HTML impiden que alguien escriba manualmente una URL HTTP.

La diferencia es sencilla de demostrar. Podemos comentar temporalmente la rama visual que anuncia el rol y volver a consultar /demo/privado: el endpoint sigue requiriendo autorización. También podríamos mostrar siempre el enlace al recurso, pero el servidor continuaría denegando accesos no autorizados. Por lo tanto, el criterio de seguridad debe definirse en el recurso real, no únicamente en el componente.

Cuando una página completa usa un atributo de autorización, puede convenir que el enrutador Blazor trabaje con AuthorizeRouteView. No debemos confundir ese mecanismo de navegación y representación con la protección de la API auxiliar, alojada en otro proceso. Cada mecanismo protege el límite donde se aplica.

Análisis final comparado

Pregunta	Componente	Servidor Blazor	API auxiliar
¿Quién dibuja la interfaz?	Sí	Aloja su ejecución	No
¿Quién consume HTTP remoto?	Invoca servicio C#	Ejecuta cliente HTTP	Recibe peticiones
¿Quién valida nombre y precio del POST?	Puede ofrecer ayuda inicial	No sustituye la API	Sí
¿Quién emite cookie local de demostración?	Presenta formulario	Sí, en POST	No
¿Quién protege /demo/privado?	Puede ocultar enlace	Sí, mediante política	No
¿Quién asigna el ID del producto?	No	Solo solicita el alta	Sí

Esta tabla describe fronteras, no una obligación de multiplicar proyectos. Saber qué proceso ejecuta cada acción evita atribuir a AuthorizeView una protección que en realidad depende del servidor.

Qué no protege el inicio de sesión de Blazor en este proyecto

La API auxiliar escucha en otro proceso y publica /api/productos. El laboratorio no transmite a esa API una credencial de usuario ni configura en ella una política de autorización para el catálogo. Por eso no podemos afirmar que iniciar sesión en Blazor protege automáticamente la creación remota de productos.

Este límite puede desconcertar al estudiante si solo observa dos pantallas en un navegador. Desde su perspectiva visual, catálogo remoto y seguridad son páginas de una misma aplicación Blazor. Pero desde el punto de vista de los servidores, una de ellas utiliza una API separada que continúa recibiendo sus propias solicitudes.

Para proteger una API real habría que definir cómo se identifica a sus clientes, qué credenciales acepta y qué políticas se aplican a cada operación. Ese diseño podría implicar tokens, cookies compartidas bajo condiciones concretas u otros mecanismos adecuados, pero no se resuelve añadiendo AuthorizeView alrededor de un botón en Blazor.

En M04 deliberadamente mantenemos independientes las dos demostraciones: la integración HTTP permite aprender GET, POST, DTO y servicios tipados; el recurso local privado permite aprender identidad, cookie, rol y autorización. La separación evita presentar una seguridad ficticia sobre el CRUD remoto.

Seguridad del servidor y experiencia del usuario no son rivales

Una interfaz puede informar de que falta iniciar sesión, mostrar el nombre de la persona o esconder opciones que no puede utilizar. Ese comportamiento facilita la navegación. El servidor, por su parte, debe comprobar los permisos con independencia de lo que se haya dibujado.

No deberíamos elegir entre una interfaz comprensible y una autorización correcta. Las dos responsabilidades se complementan. La interfaz reduce confusión para el usuario, mientras que la política del servidor impide el acceso indebido al recurso.

En nuestro ejemplo esta combinación se observa sin implantar un proveedor completo de identidades. Primero reconocemos un estado de sesión de demostración; después mostramos la información correspondiente y comprobamos que el endpoint exige Formador. El alumnado debe poder explicar por qué las dos comprobaciones siguen siendo necesarias aunque las pantallas parezcan coherentes.

4.4.4. Formularios HTTP y antifalsificación

El inicio de sesión de laboratorio usará un formulario HTML normal con `method="post"`, `<AntiforgeryToken />` y `data-enhance="false"`. El servidor comprobará el formulario, creará los claims y emitirá la cookie. La salida también se hará mediante POST con token y `SignOutAsync`.

El token antifalsificación dificulta que un sitio externo envíe solicitudes que cambian estado usando la sesión del navegador; **no comprueba la identidad** y no transforma una contraseña fija en un sistema seguro. Nunca debemos desactivar su protección simplemente para que la demo funcione.

Comprender el antiforgery en un formulario de entrada

Un navegador autenticado envía automáticamente cookies apropiadas al servidor. Esta comodidad introduce un riesgo: otro sitio podría intentar provocar un POST usando la sesión del navegador. El token antifalsificación permite al servidor comprobar que la solicitud proviene de un formulario que recibió un token legítimo.

En Razor declaramos:

```
<form method="post" action="/demo/salir" data-enhance="false">
    <AntiforgeryToken />
    <button type="submit">Cerrar sesión</button>
</form>
```

En un endpoint de demostración que lee el formulario directamente, conviene hacer explícita la comprobación:

```
app.MapPost("/demo/salir",
    async (HttpContext http, IAntiforgery antiforgery) =>
{
    if (!await antiforgery.IsRequestValidAsync(http))
        return Results.BadRequest("Token antifalsificación inválido.");

    await http.SignOutAsync("CookiesDemo");
    return Results.Redirect("/seguridad");
});
```

El código requiere el espacio de nombres `Microsoft.AspNetCore.Antiforgery` y `Microsoft.AspNetCore.Authentication`, así como la configuración adecuada del middleware. El resultado de un POST sin token no es una salida válida sino una respuesta de error. El token no sustituye la comprobación del usuario ni convierte un código fijo de laboratorio en una contraseña segura.

Antifalsificación: qué amenaza aborda y cuál no

Los navegadores envían ciertas cookies de manera automática cuando una petición coincide con las reglas de la cookie. Esta propiedad facilita mantener una sesión, pero también crea la posibilidad de que un sitio ajeno intente provocar peticiones que modifican el estado en nombre del usuario.

Un token antifalsificación aporta una comprobación adicional de procedencia para esas solicitudes. En el formulario Razor declaramos AntiforgeryToken y el servidor valida el token antes de procesar la acción. Cuando se utiliza una ruta que lee manualmente formularios, la validación explícita mediante IAntiforgery.IsRequestValidAsync ayuda a dejar visible el requisito. Un POST sin token correcto debe rechazarse, aunque incluya el código del laboratorio.

El token antifalsificación **no** demuestra la identidad de una persona y tampoco convierte un código fijo en una credencial segura. Su finalidad es diferente: reducir solicitudes de cambio de estado falsificadas. Por eso las dos comprobaciones aparecen en momentos distintos del endpoint: primero validar la solicitud, después comprobar el código de acceso o ejecutar la salida.

La salida de sesión también modifica el estado y debe realizarse mediante POST con antiforgery. Usar un enlace GET que cierre la sesión cuando se visita una URL mezclaría la semántica de consulta con una operación que cambia el estado de autenticación.

Describir el alcance sin ofrecer falsas garantías

Nuestro ejemplo entrega una identidad de prueba mediante un código fijo en un entorno de desarrollo. Es adecuado para demostrar el papel de una cookie, los claims y una política local. No es adecuado para publicar una aplicación expuesta a Internet.

Una implementación profesional necesitaría mecanismos legítimos de verificación de identidad, gestión de contraseñas o proveedores externos, secretos protegidos, HTTPS, configuración segura de cookies, expiración de sesiones, auditoría apropiada y políticas de autorización acordes con los datos protegidos. No se obtienen estas propiedades simplemente porque AddAuthentication aparezca en Program.cs.

Para reforzar el carácter experimental, las rutas /demo/entrar, /demo/salir y /demo/privado se registran solo en Development. La página puede existir como contenido, pero el mecanismo de acceso no debe desplegarse como sistema de seguridad público.

4.4.5. Casos que debe observar el alumno

Estado	Resultado
Usuario anónimo	`AuthorizeView` presenta `NotAuthorized`
Anónimo consulta `/demo/privado`	El servidor deniega el recurso
Código local correcto	Se crea una sesión de demostración
Sesión con rol `Formador`	Se muestra el rol y se accede al recurso protegido
Salida POST	La sesión deja de estar autenticada

La demostración es intencionalmente superficial. Identity, proveedores externos, permisos de producción y protección del catálogo completo quedan fuera de este curso de 30 horas.

Los límites concretos de una demostración segura

Un proyecto de aula con código fijo de acceso debe expresar su finalidad. No es un sistema de identidad para Internet. No debe reutilizarse en un despliegue y no debe almacenar datos sensibles. Para mantener controlado el ejemplo, sus rutas especiales pueden registrarse únicamente bajo `app.Environment.IsDevelopment()`.

Los mecanismos que faltarían en un producto real incluyen verificación de credenciales mediante un proveedor adecuado, gestión del ciclo de vida de usuarios, configuración segura de cookies, HTTPS, protección de secretos, políticas de acceso y procedimientos de recuperación. No se desarrollan aquí porque la finalidad de M04 es observar los conceptos básicos dentro de seis horas.

Diagnóstico guiado de un acceso fallido

Si la página continúa mostrando una identidad anónima después de enviar el formulario, comprueba primero que el POST se ha enviado a la ruta correcta y que el navegador no lo ha tratado como una navegación parcial que impide observar la nueva cookie. Comprueba después que el token antifalsificación se envió y que el servidor aceptó la solicitud. Si la respuesta es satisfactoria, observa si la cookie se almacenó y si el navegador la envía al volver a consultar el servidor.

Si la página muestra el usuario pero el recurso protegido deniega acceso, revisa la autorización del endpoint. Una identidad autenticada puede carecer del claim de rol requerido. Si la interfaz muestra el rol y aun así el endpoint rechaza la solicitud, revisa si ambas acciones se están ejecutando en el mismo host y bajo el mismo esquema de autenticación.

Este procedimiento está orientado a clasificar errores. No necesitamos integrar un sistema de gestión de cuentas para aprender a diferenciar identidad, permisos, HTTP y presentación.

Diagnóstico de un inicio que parece correcto

Si el usuario introduce el código y sigue viendo NotAuthorized, no deberíamos cambiar inmediatamente los claims. Primero se comprueba que el formulario envía un POST HTTP normal a la ruta correcta. Después se observa si el token antifalsificación es válido y si el endpoint devuelve

una redirección de éxito o de rechazo. Solo entonces examinamos la cookie de respuesta y su posterior envío por el navegador.

Si el usuario está reconocido pero no puede abrir el recurso privado, revisamos el claim de rol y la política del endpoint. Una identidad con nombre no implica poseer Formador. Si en cambio el recurso privado es accesible estando anónimo, existe un problema más grave: la autorización del servidor no se está aplicando como debía.

Si la aplicación se ejecuta en producción y el formulario deja de funcionar, debemos recordar que las rutas de demostración se limitan deliberadamente a Development. No debemos cambiar ese límite para «hacer que funcione en producción»: la autenticación fija de laboratorio no está diseñada para ese uso.

Ordenar una investigación cuando una sesión parece no funcionar

Una sesión fallida puede tener varias causas y el orden de análisis importa. Primero, ¿se envió realmente el formulario mediante POST y a la ruta correcta? Segundo, ¿el servidor recibió un token antifalsificación válido? Tercero, ¿la comprobación del código de demostración aceptó la solicitud? Cuarto, ¿se emitió una cookie y el navegador la presentó después? Quinto, ¿el principal resultante contiene el rol requerido por el recurso?

Si falla el primer paso, mirar ClaimsIdentity todavía es prematuro. Si el token no valida, cambiar RequireRole no ayudará. Si existe un principal con nombre pero no con el rol correcto, el problema se sitúa en la autorización y no en la carga de la página.

También debemos comprobar el entorno. La implementación de laboratorio registra las rutas especiales solo en Development, por lo que no deben estar accesibles como sistema de acceso en Production. Habilitarlas indiscriminadamente para conseguir una demostración pública sería contrario al objetivo de seguridad.

Esa secuencia de preguntas ofrece una estrategia transferible: no mezclar fallos de transporte, formulario, identidad, cookie y permisos. Cada comprobación debe aportar información sobre uno de esos límites.

Ampliación 4.4 — Analizar la seguridad de una aplicación que combina Blazor y una API

El ejemplo de 4.4 utiliza una cookie local de demostración para mostrar qué son autenticación y autorización. Ese escenario es deliberadamente pequeño, pero conviene comprender qué **no** demuestra. Una aplicación real que muestre productos a través de Blazor y permita modificarlos mediante una API tiene al menos dos superficies diferentes: el host que sirve y ejecuta los componentes, y el host que acepta operaciones HTTP sobre productos.

En la pantalla de seguridad podemos iniciar sesión como AlumnoDemo con rol Formador y abrir /demo/privado. Esa autorización pertenece al servidor Blazor y a su política local. El catálogo remoto, por su parte, sigue consumiendo una API auxiliar pública. La identidad de la pantalla no modifica mágicamente la configuración de la API. La separación está presente aunque ambos proyectos se ejecuten en la misma máquina.

Una amenaza concreta: acceder a un endpoint sin utilizar la pantalla

Imaginemos que una página Blazor solo representa el botón Crear en API para usuarios con el rol Formador. El desarrollador podría pensar que la operación está protegida porque el usuario

anónimo no ve el botón. Sin embargo, cualquier cliente HTTP que conozca la ruta POST /api/productos puede intentar enviar una petición directamente al servidor API.

Si ese endpoint no tiene una política de autorización propia, la ausencia del botón no impide que reciba y procese la petición. Para protegerlo correctamente habría que diseñar cómo reconoce la identidad del cliente remoto, qué credenciales acepta y qué permiso se exige para esa operación.

Este razonamiento permite separar una regla de experiencia de usuario de una regla de seguridad. AuthorizeView ayuda a mostrar elementos apropiados, pero el servidor que posee el recurso debe comprobar sus propios requisitos antes de devolver información o modificar datos.

No vamos a simular una protección de la API que realmente no existe. El laboratorio conserva sus endpoints de productos públicos y los identifica como demostración. El único recurso protegido del punto 4.4 es /demo/privado, que pertenece al host Blazor y utiliza su política de rol.

Un segundo riesgo: confundir código de acceso con autenticación de usuarios

El formulario del laboratorio acepta una cadena fija para crear una identidad temporal de prueba. Su finalidad es poder observar SignInAsync, la emisión de una cookie y la comprobación de un rol sin instalar un sistema de cuentas.

La cadena fija no tiene las propiedades de una contraseña de usuario gestionada correctamente. No está asociada a una cuenta con ciclo de vida, no dispone de un proceso de cambio y recuperación, y no constituye un mecanismo de identidad adecuado para publicar un sitio en Internet. Su valor didáctico no justifica convertirla en una característica de producción.

Por eso restringimos las rutas /demo/entrar, /demo/salir y /demo/privado al entorno Development. Si una ejecución publicada no encuentra esas rutas, no significa que debamos eliminar la protección de entorno: significa que el mecanismo de laboratorio no se ha habilitado para un contexto en el que no sería apropiado utilizarlo.

Una aplicación real podría utilizar ASP.NET Core Identity u otro sistema legítimo de autenticación. Eso aportaría funcionalidades como gestión de usuarios, credenciales, confirmaciones, recuperación y políticas adicionales que no hemos desarrollado aquí. Es importante saber que existen sin convertir esta jornada en una instalación y configuración completa de Identity.

Un tercer riesgo: pensar que una cookie sirve para todo servidor

Cuando el navegador inicia sesión con el host Blazor, guarda una cookie emitida por ese servidor. No debemos suponer que otro proceso ASP.NET Core reconocerá automáticamente su contenido ni que el navegador enviará una credencial apropiada a cualquier URL.

El ámbito de una cookie depende de sus atributos y de las reglas del navegador, mientras que su interpretación depende del esquema y de la configuración de autenticación del servidor receptor. Compartir autenticación entre aplicaciones requiere un diseño explícito. No se consigue simplemente copiando el nombre CookiesDemo a un segundo Program.cs.

En nuestro recorrido además existe una distinción de origen de las peticiones: el HttpClient del componente Interactive Server realiza sus llamadas desde el host Blazor. Ese HttpClient no hereda por sí mismo la cookie de sesión del navegador para autenticar automáticamente la API remota.

Por ello una futura protección del catálogo exigiría decidir cómo se transmitirá identidad entre procesos, cómo se verificará y qué autorización se aplicará. No basta con poner un atributo de autorización en un componente de presentación.

El papel de HTTPS

HTTP sin cifrar sobre localhost resulta cómodo para el laboratorio porque permite reconocer puertos y rutas sin introducir certificados de desarrollo como un problema adicional. Esa comodidad no debe confundirse con una recomendación de utilizar HTTP sin cifrar para credenciales reales.

En una aplicación publicada con autenticación, HTTPS protege el transporte de información sensible entre navegador y servidor. La configuración de cookies y las políticas de sesión también deben pensarse para ese contexto. El laboratorio se limita a una ejecución local de demostración y no introduce contraseñas reales.

Esta distinción debe aparecer claramente en el razonamiento del alumno: el código que sirve para observar una cookie en Development no es una plantilla segura para publicar autenticación en Internet sin transformaciones adicionales.

Un cuarto riesgo: una operación POST no protegida frente a solicitudes falsificadas

Las cookies permiten que el navegador envíe automáticamente información de sesión en peticiones compatibles con su ámbito. Un sitio ajeno podría intentar aprovechar ese comportamiento para provocar solicitudes que cambian el estado. La protección antifalsificación se utiliza para comprobar que ciertos envíos se realizan con un token válido asociado al formulario esperado.

En el laboratorio incorporamos AntiforgeryToken a los formularios y verificamos la validez del token antes de iniciar o cerrar sesión. El servidor no debe aceptar un POST sin token simplemente porque su cuerpo contenga el código de demostración correcto.

Conviene distinguir cuatro conceptos:

Concepto	Pregunta que responde
Autenticación	¿Quién realiza la acción?
Autorización	¿Está permitido acceder al recurso?
Antiforgery	¿El envío de este formulario satisface la comprobación antifalsificación?
HTTPS	¿Está protegida la comunicación durante el transporte?

Son mecanismos complementarios. Ninguno de ellos sustituye automáticamente a los otros. Un formulario puede llevar un token válido y, aun así, contener credenciales incorrectas. Un usuario puede estar autenticado y no tener rol suficiente. Una conexión cifrada puede transportar una petición que el servidor debe rechazar por reglas de autorización.

¿Qué significa que una operación sea segura si la interfaz es interactiva?

Blazor Interactive Server puede actualizar la UI después de un evento sin ejecutar una navegación completa visible. Esa capacidad no convierte todas las operaciones en acciones que puedan establecer cabeceras HTTP en cualquier instante. Para crear o eliminar cookies necesitamos una respuesta HTTP donde el servidor pueda escribir las cabeceras apropiadas.

Nuestro ejemplo utiliza un formulario POST convencional con data-enhance=false para abrir un intercambio HTTP normal. Después del inicio o cierre de sesión, el servidor redirige al navegador, que vuelve a consultar la página. Ese recorrido permite observar cómo cambia el estado reconocido.

Si intentáramos realizar SignInAsync directamente dentro de un evento interactivo cuando la respuesta HTTP correspondiente ya está iniciada, podríamos recibir una incidencia de cabeceras no modificables. La elección del formulario normal no es una limitación general de Blazor; es una manera clara de explicar el mecanismo subyacente de cookies.

Evaluar una política mediante escenarios y no mediante una captura

Una captura de la página que muestra «Rol Formador reconocido» es evidencia de presentación, pero no demuestra por sí sola que la política del servidor funcione. Debemos consultar también el recurso protegido y observar la respuesta para distintos estados.

Primero realizamos una solicitud anónima: el servidor debe denegar el recurso. Después iniciamos sesión y comprobamos que el rol correcto permite acceder. Si cambiamos temporalmente la política para exigir OtroRol, el usuario sigue identificado, pero debe ser rechazado. Finalmente cerramos sesión y repetimos la solicitud anónima.

Los casos forman una secuencia de razonamiento:

```
Sin identidad       → recurso denegado
Identidad incorrecta → recurso denegado
Identidad Formador   → recurso permitido
Cierre de sesión    → recurso denegado de nuevo
```

La denegación de un usuario con rol incorrecto requiere configurar esa condición para observarla: no es suficiente ver un rechazo de usuario anónimo y afirmar que se han probado los permisos por rol.

Evaluar un formulario mediante escenarios independientes

El inicio de sesión también necesita más de una observación. Un POST con código incorrecto debe rechazar el acceso. Un POST sin token antifalsificación debe rechazarse, aunque incluya el código correcto. Un POST válido con código correcto y token válido puede establecer la sesión. El cierre con token válido debe eliminarla.

No debemos mezclar todos los defectos en una sola prueba. Si enviamos simultáneamente código incorrecto y token ausente, el servidor puede rechazar el token antes de comprobar el código. Esa observación no demostraría que la comprobación de credenciales funciona. Para estudiar cada regla, mantenemos las demás condiciones correctas.

Este principio ya apareció en los ejercicios HTTP del catálogo: pruebas pequeñas con una sola variación proporcionan diagnósticos mucho más claros que una petición que falla por múltiples causas.

Seguridad e interoperabilidad no son objetivos opuestos

Una API bien diseñada puede publicar operaciones consumibles por distintos clientes y, al mismo tiempo, comprobar permisos. La interoperabilidad describe cómo se comunican las aplicaciones; la autorización decide qué operaciones acepta el servidor para una identidad determinada. Para este curso mantenemos pública la API de productos y protegemos un endpoint separado, precisamente para no presentar un sistema falso que aparenta ser seguro por mostrar mensajes de rol.

El siguiente paso en un proyecto real sería definir requisitos de seguridad para el propio catálogo antes de implementar credenciales o tokens. ¿Quién puede consultar? ¿Quién puede crear? ¿Qué permisos se necesitan para modificar o eliminar? ¿Se protegen datos privados? Una política debe

responder a esos requisitos, no elegirse simplemente porque el framework ofrece una API determinada.

Conclusión de seguridad aplicada

El conocimiento que debe conservar el estudiante es una cadena de responsabilidades: el navegador y Blazor representan una interfaz; el servidor de Blazor reconoce una cookie de demostración; las políticas del servidor protegen sus propios recursos; la API remota mantiene sus operaciones y decisiones; y el uso de una interfaz condicional no sustituye autorización en el servidor que posee el dato.

Esa comprensión es más importante que memorizar una llamada aislada a AddCookie o una plantilla concreta de AuthorizeView. Permite evaluar diseños reales y detectar cuándo una funcionalidad visual se ha confundido peligrosamente con una garantía de seguridad.

Ampliación 4.4 — Respuestas de desafío y acceso denegado en aplicaciones con cookies

La autenticación por cookie resulta muy cómoda para una aplicación web porque permite redirigir a una página de entrada cuando el usuario no está reconocido. Sin embargo, cuando el recurso protegido se consume como un endpoint HTTP, una redirección puede resultar menos clara que un código de estado explícito.

Imaginemos una solicitud anónima a GET /demo/privado. El endpoint exige el rol Formador. El servidor necesita comprobar primero que existe una identidad reconocida y después que satisface la política. Si no hay autenticación válida, se produce un desafío de autenticación; si existe identidad pero carece del permiso, el acceso se prohíbe.

Estas decisiones están relacionadas con 401 y 403, pero la configuración de cookies puede convertirlas en respuestas de redirección apropiadas para aplicaciones que esperan páginas HTML. En el laboratorio elegimos hacer visible el código 401 para el recurso protegido, de modo que el alumno pueda comprobarlo con una herramienta HTTP.

Qué implica un desafío de autenticación

Un desafío aparece cuando la aplicación necesita que el usuario se autentique para continuar. En un navegador que visita una página, redirigir al formulario de entrada puede ser una experiencia razonable. En un cliente que espera JSON, recibir una página HTML de login puede causar una confusión: el cliente podría tratar de convertir ese HTML a su DTO y concluir equivocadamente que la API envió datos incompatibles.

Por eso hay que distinguir el tipo de consumidor. Una aplicación de interfaz HTML y un endpoint de datos pueden utilizar el mismo esquema de autenticación, pero necesitar representaciones diferentes del rechazo. El criterio debe decidirse explícitamente, no depender de un comportamiento accidental que el alumnado desconoce.

En el proyecto de demostración configuramos una excepción para la ruta privada:

```
options.Events.OnRedirectToLogin = contexto =>
{
    if (contexto.Request.Path.StartsWithSegments("/demo/privado"))
    {
        contexto.Response.StatusCode = 401;
        return Task.CompletedTask;
    }
```

```
    contexto.Response.Redirect(contexto.RedirectUri);
    return Task.CompletedTask;
};
```

La condición muestra cómo conservar una navegación habitual hacia la página de login mientras que el endpoint de laboratorio produce un resultado HTTP observable. No estamos proponiendo que toda aplicación deba copiar ese fragmento sin analizar su propia API y sus rutas.

Identidad existente, permiso ausente

El segundo caso requiere imaginar un principal autenticado, pero con un rol diferente de Formador. No es lo mismo que un usuario anónimo: la aplicación reconoce una identidad y, aun así, deniega el recurso por su política.

En un diseño que necesite exponer 403 directamente para un endpoint de datos, puede configurarse la respuesta de acceso denegado mediante el evento apropiado de cookies. Por ejemplo, para el mismo recurso de laboratorio:

```
options.Events.OnRedirectToAccessDenied = contexto =>
{
    if (contexto.Request.Path.StartsWithSegments("/demo/privado"))
    {
        contexto.Response.StatusCode = 403;
        return Task.CompletedTask;
    }

    contexto.Response.Redirect(contexto.RedirectUri);
    return Task.CompletedTask;
};
```

Ambas configuraciones corresponden a situaciones diferentes. La primera observa falta de autenticación, la segunda observa falta de permiso. Para comprobar 403 no basta con enviar una solicitud anónima; necesitamos crear deliberadamente una identidad válida que no satisfaga RequireRole("Formador").

Esto proporciona una lección general de pruebas: cada resultado debe corresponder al estado real del usuario en el instante de la petición, no al mensaje que creemos que se mostrará.

El cliente no debe convertir una redirección en éxito de negocio

Un cliente HTTP puede seguir redirecciones según su configuración. Si una solicitud destinada a un recurso JSON se redirige a una página HTML, el estado final podría llegar a ser 200 para la página de entrada. Eso no demuestra que se haya autorizado el recurso original.

Por este motivo, cuando depuramos autenticación en un servicio necesitamos observar no solo el último número de estado, sino también la ruta, las posibles redirecciones, el tipo de contenido y el cuerpo efectivo. Esta atención es especialmente importante cuando una biblioteca intenta deserializar automáticamente JSON.

La misma disciplina se utilizó en 4.1 con GET y POST: una respuesta aparentemente exitosa puede seguir siendo incompatible con el contrato previsto. En 4.4 añadimos la identidad del usuario como una condición que también afecta al recorrido HTTP.

El código de seguridad pertenece a un contexto concreto

Los ejemplos usan rutas de demostración y una identidad de laboratorio. No definen cómo debe autenticarse una API empresarial ni cómo transmitir credenciales entre procesos distintos. Esas decisiones necesitarían requisitos de negocio y un diseño de seguridad específico.

Tampoco debemos interpretar la capacidad de configurar 401 y 403 como sustituto de proteger efectivamente el recurso. El endpoint debe seguir exigiendo autorización mediante su política. Un mensaje de error bien presentado no añade por sí mismo controles que no existen.

Cómo examinar la diferencia en una sesión

El alumno puede comenzar por una solicitud anónima a /demo/privado y comprobar 401. Después inicia sesión como Formador y comprueba 200. Para observar una identidad autenticada sin permiso, se puede utilizar una copia de ensayo con un principal de rol diferente y comprobar la denegación correspondiente. Finalmente, al cerrar la sesión, la solicitud debe volver a producir rechazo anónimo.

Este experimento conecta el estado del principal, la política y la semántica de HTTP. No exige desplegar un proveedor completo de Identity ni compartir cookies con la API auxiliar.

La conclusión es que **reconocer una identidad, aplicar un permiso y decidir cómo se comunica una denegación son tres responsabilidades distintas**. La comprensión de esa separación permite diagnosticar mejor una aplicación ASP.NET Core protegida.

4.4.6. Ampliación técnica — ASP.NET Core Identity y JWT Bearer reales

Hasta aquí hemos estudiado una cookie local de demostración y una política de autorización que protege un endpoint del servidor Blazor. Es un buen primer laboratorio para diferenciar autenticación, autorización y presentación condicional, pero no contiene un sistema real de usuarios ni ofrece tokens firmados para una API remota. Esta ampliación responde a esa necesidad sin sustituir los ejercicios previos: vamos a integrar **ASP.NET Core Identity con Entity Framework Core y una API que emite y valida JWT auténticos**.

Ubicación en el curso. Se trata de una ampliación autónoma de 4.4. El temario presencial conserva seis horas y el itinerario anterior sigue siendo válido. El proyecto adicional se encuentra en M04_APIS_SEGURIDAD/PROYECTO/4.5/CursoWeb.IdentityJwtApi; la numeración 4.5 de la carpeta identifica el estado adicional del proyecto, no crea un nuevo módulo de seis horas.

Del acceso con código fijo a ASP.NET Core Identity

ASP.NET Core Identity proporciona componentes para gestionar usuarios, credenciales, hashes de contraseña, roles, reglas de acceso, bloqueo y almacenamiento. En la API adicional utilizaremos IdentityUser, UserManager, SignInManager y RoleManager. No guardaremos las contraseñas directamente en una columna de texto: UserManager.CreateAsync utiliza la infraestructura de hashing de Identity y las comprobaciones de contraseña se realizan a través de SignInManager.

El almacenamiento será **SQL Server Express LocalDB**, instalado con Visual Studio en Windows, utilizando el proveedor `Microsoft.EntityFrameworkCore.SqlServer` y un contexto derivado de `IdentityDbContext<IdentityUser>`. Eso permite crear tablas reales de usuarios y roles: AspNetUsers, AspNetRoles y sus tablas de relaciones, entre otras. Aunque el alumnado ya conoce EF Core, aquí nos interesa observar qué información persiste y cuáles son las responsabilidades del sistema de identidad.

El usuario tiene Email, UserName, Id y propiedades de control de identidad. El identificador no se deriva necesariamente de su correo: la clase IdentityUser define un Id propio. La contraseña no forma parte del perfil que enviamos como JSON al cliente. Podemos consultar un perfil autenticado sin publicar jamás el hash almacenado.

En este laboratorio utilizamos `EnsureCreatedAsync` para crear rápidamente las tablas de Identity en una base de SQL Server Express LocalDB exclusiva de la práctica. **No es una política de evolución de esquemas para producción.** Una solución mantenida utilizaría migraciones, configuración de persistencia y procedimientos de despliegue adecuados. La base se llama `CursoWeb_M04_IdentityJwt` y se aloja en la instancia `(localdb)\\MSSQLLocalDB` de Windows. No se genera un fichero SQLite dentro de `.work`; puede inspeccionarse con el Explorador de objetos SQL Server de Visual Studio.

SQL Server Express LocalDB de Visual Studio: conexión y persistencia

La instancia de desarrollo es `(localdb)\\MSSQLLocalDB`. **LocalDB no es SQLite**, sino una modalidad ligera de SQL Server Express para Windows y desarrollo local. Se integra en Visual Studio mediante las herramientas de datos. Antes de ejecutar el laboratorio, comprueba que la característica SQL Server Express LocalDB está instalada y que puedes conectarte desde el Explorador de objetos de SQL Server.

La aplicación declara una cadena de conexión en `appsettings.json`, sin nombre de usuario ni contraseña SQL, porque utiliza autenticación integrada de Windows:

```
{
  "ConnectionStrings": {
    "IdentityDb": "Server=(localdb)\\\
\MSSQLLocalDB;Database=CursoWeb_M04_IdentityJwt;Trusted_Connection=True;TrustServerCertificate=
True;MultipleActiveResultSets=true"
  }
}
```

La doble barra dentro de JSON representa una única barra invertida de la instancia. En una cadena C# literal precedida por `@` se escribiría `@"Server=(localdb)\MSSQLLocalDB;Database=CursoWeb_M04_IdentityJwt;Trusted_Connection=True;"`. El proveedor de EF Core es `Microsoft.EntityFrameworkCore.SqlServer` y la configuración correspondiente utiliza `UseSqlServer`:

```
var conexion = builder.Configuration.GetConnectionString("IdentityDb")
    ?? throw new InvalidOperationException("Falta la conexión IdentityDb.");

builder.Services.AddDbContext<IdentidadDbContext>(opciones =>
    opciones.UseSqlServer(conexion));
```

No se utiliza `UseSqlite`, no se crea `identity-m04.db` y no se sustituye LocalDB por una base InMemory en las pruebas. La instancia debe estar accesible desde el usuario de Windows que ejecuta Visual Studio o el proceso ASP.NET Core.

Al ejecutar por primera vez el proyecto docente, `EnsureCreatedAsync` crea el esquema de Identity en una **base de prácticas exclusiva**. En el Explorador de objetos de SQL Server verás tablas `AspNetUsers`, `AspNetRoles` y `AspNetUserRoles`. El hash de contraseña se almacena en `PasswordHash`, nunca se devuelve al cliente.

`EnsureCreatedAsync` se limita aquí a un modelo sencillo de inicio. **No se mezcla con un historial de migraciones**: para aplicaciones en evolución deberíamos utilizar migraciones EF Core, y para entornos publicados, un proceso controlado de despliegue. Para reiniciar desde cero, detén la API y elimina exclusivamente la base `CursoWeb_M04_IdentityJwt` desde las herramientas de SQL Server; no borres otras bases existentes en la instancia.

Las pruebas automáticas de esta ampliación también necesitan un entorno Windows con LocalDB. Por tanto, se ejecutan en un runner Windows que arranca `MSSQLLocalDB`, crea una base aislada por ejecución y valida SQL Server real. Las verificaciones de Blazor que no utilizan Identity pueden seguir funcionando en otras plataformas, pero no demostrarían persistencia LocalDB.

La diferencia entre Identity y la autenticación JWT

Identity administra usuarios y ayuda a verificar contraseñas y roles. JWT, por otra parte, es un formato de token que representa afirmaciones firmadas. No son dos alternativas intercambiables.

Nuestro flujo tendrá estas fases:

1. La API registra un usuario mediante UserManager y le asigna el rol Alumno.
2. Un cliente envía el correo y la contraseña al endpoint de acceso.
3. SignInManager verifica la contraseña y aplica las reglas de bloqueo configuradas.
4. La API obtiene las funciones o roles del usuario desde Identity.
5. El servidor crea un JWT firmado, con emisor, audiencia, subject, expiración e identificador de token.
6. El cliente incluye el valor obtenido en la cabecera Authorization: Bearer de las consultas protegidas.
7. El middleware JwtBearer valida firma, emisor, audiencia y vigencia antes de conceder acceso.
8. Una política de autorización exige el rol Formador para el recurso más restringido.

No emitimos una cookie de sesión en este recorrido. El cliente recibe un token de acceso y lo transmite explícitamente en cada solicitud protegida.

Una distinción importante en .NET 8: bearer tokens de Identity y JWT

ASP.NET Core 8 incluye los métodos AddIdentityApiEndpoints y MapIdentityApi. Son útiles para exponer endpoints de identidad y pueden emitir tokens bearer propietarios de Identity, pero **esos tokens no deben describirse automáticamente como JWT**. Un JWT tiene una representación reconocible formada por tres segmentos codificados y separados por puntos, con header, payload y firma.

Para no mezclar conceptos, nuestra ampliación utiliza AddIdentityCore con Entity Framework Core para gestionar usuarios y roles, y el paquete Microsoft.AspNetCore.Authentication.JwtBearer para validar JWT. La emisión de JWT se realiza de manera explícita para que el alumno vea de dónde proceden las afirmaciones y cómo se firma el token.

Esta construcción manual es **una demostración controlada**, no una recomendación de crear un servidor de identidad propio para aplicaciones de producción. Para escenarios empresariales conviene delegar la emisión en un proveedor de identidad y utilizar protocolos adecuados como OpenID Connect/OAuth 2.0, evitando inventar soluciones de autenticación.

Los tres segmentos de un JWT

Un JWT firmado suele representarse como header.payload.signature. Los segmentos se codifican en Base64Url, lo que no equivale a cifrar el contenido. Por eso **no debemos colocar contraseñas, secretos, documentos privados o datos sensibles innecesarios en el payload**.

El encabezado indica, entre otros elementos, el algoritmo que se ha utilizado; el payload contiene claims; y la firma permite comprobar que el contenido no ha sido modificado por alguien que desconozca la clave de firma. Un cliente puede decodificar los segmentos sin disponer del secreto. Decodificar no significa que el token sea válido: confiar en él exige verificar firma y demás condiciones.

En el laboratorio utilizamos un algoritmo HMAC SHA-256 con una clave de longitud suficiente. La clave se lee de Jwt__SigningKey mediante la configuración de .NET. **No figura en appsettings.json ni en el repositorio.** Cada equipo genera su secreto aleatorio en su entorno de desarrollo.

Esta firma simétrica es suficiente para demostrar el mecanismo en un proceso local. Una arquitectura con servidores emisores y consumidores independientes podría utilizar firmas asimétricas y validación mediante claves públicas; esa infraestructura excede el propósito de la jornada.

Claims, roles y sujeto

El token contiene identificadores y afirmaciones. El claim sub identifica al sujeto; iss identifica al emisor; aud señala el destinatario previsto; exp expresa la expiración; iat el momento de emisión; y jti un identificador del token. Además, incorporamos claims de identidad y de rol que permiten a ASP.NET Core establecer un ClaimsPrincipal cuando el JWT supera su validación.

Un usuario registrado desde el endpoint público recibe el rol Alumno, no Formador. Este principio es esencial: permitir que el cliente envíe el rol deseado al registrarse convertiría una simple creación de cuenta en una escalada de permisos.

La cuenta Formador de la demostración se inicializa **solo si el docente proporciona desde el entorno** correo y contraseña. El alumno no puede obtener ese rol registrándose ni modificando el cuerpo JSON del endpoint público.

La política de autorización del endpoint /api/identidad/formador exige RequireRole("Formador"). Un JWT de Alumno puede estar perfectamente firmado y vigente y, sin embargo, recibir 403. Un cliente que no envía JWT o presenta uno inválido recibe 401. Esta diferencia reproduce con otra tecnología los conceptos estudiados al utilizar cookies.

Cómo valida realmente JwtBearer

El servidor tiene que rechazar tokens emitidos por un tercero que utilice otra clave, tokens destinados a otra API y tokens caducados. Por ello, el middleware configura parámetros explícitos:

```
new TokenValidationParameters
{
    ValidateIssuer = true,
    ValidIssuer = emisor,
    ValidateAudience = true,
    ValidAudience = audiencia,
    ValidateIssuerSigningKey = true,
    IssuerSigningKey = new SymmetricSecurityKey(bytesFirma),
    ValidateLifetime = true,
```

```
    RequireExpirationTime = true,
    RequireSignedTokens = true,
    ClockSkew = TimeSpan.FromSeconds(5),
    NameClaimType = ClaimTypes.NameIdentifier,
    RoleClaimType = ClaimTypes.Role
}
```

La comprobación de firma acredita integridad y procedencia según la clave configurada. El emisor evita confiar en tokens de otra autoridad. La audiencia asegura que el token está dirigido al recurso esperado. La validación temporal impide aceptar tokens fuera de su vigencia. Una API que solo decodificara el payload sin validar estos atributos no estaría autenticando correctamente.

En el ejemplo el token expira quince minutos después de su emisión. El tiempo reducido permite experimentar con caducidad sin añadir renovación de tokens. El código no implementa refresh tokens ni revocación centralizada: cuando el cliente «olvida» el JWT en la pantalla, elimina su copia, pero **eso no revoca un token que otra aplicación ya tenga**. Para requisitos de revocación y sesiones prolongadas necesitaríamos un diseño adicional.

Demostración técnica completa: del esquema SQL Server a un JWT utilizable

El proceso que sigue **no depende de copiar el programa terminado**. Lo recorreremos como una transformación del servidor. Primero construiremos identidad persistente y usuarios; después aprenderemos a identificar y registrar al solicitante; a continuación emitiremos un token y finalmente protegeremos las rutas. En el manual de prácticas se repetirán esos estados como operaciones que realiza el alumno y se comprobará cada uno por separado.

Estado teórico 1 — la base LocalDB y los servicios. La aplicación necesita un `DbContext` específico de Identity y un registro de servicios capaz de proporcionar `UserManager`, `RoleManager` y `SignInManager`. El ejemplo usa esta configuración real:

```
var conexion = builder.Configuration.GetConnectionString("IdentityDb")
    ?? throw new InvalidOperationException(
        "Configura ConnectionStrings:IdentityDb apuntando a SQL Server Express LocalDB.");

builder.Services.AddDbContext<IdentidadDbContext>(opciones =>
    opciones.UseSqlServer(conexion));

builder.Services.AddIdentityCore<IdentityUser>(opciones =>
{
    opciones.User.RequireUniqueEmail = true;
    opciones.Password.RequiredLength = 8;
    opciones.Password.RequireDigit = true;
    opciones.Password.RequireUppercase = true;
    opciones.Password.RequireLowercase = true;
    opciones.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
    opciones.Lockout.MaxFailedAccessAttempts = 5;
    opciones.Lockout.AllowedForNewUsers = true;
})
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<IdentidadDbContext>()
.AddSignInManager()
.AddDefaultTokenProviders();
```

El registro `AddDbContext` decide dónde se almacena la información. `AddIdentityCore` configura reglas para cuentas y contraseñas; `AddRoles` añade la administración de roles; `AddEntityFrameworkStores` conecta esas operaciones al contexto SQL Server. `AddSignInManager` proporciona las comprobaciones de inicio de sesión. En particular, `UserManager` no representa una

tabla que debamos recorrer manualmente para comparar contraseñas: proporciona operaciones que aplican la configuración de Identity.

El contexto queda declarado al final de `Program.cs`, fuera de las instrucciones de nivel superior:

```
sealed class IdentidadDbContext(DbContextOptions<IdentidadDbContext> opciones)
    : IdentityDbContext<IdentityUser>(opciones) { }
```

Tras `builder.Build()` se crea un ámbito de servicios y se llama a `EnsureCreatedAsync` para preparar las tablas. El ámbito es necesario porque `IdentidadDbContext` es un servicio con ciclo de vida acotado; no debemos resolverlo y conservarlo globalmente como si fuera un objeto estático. El bloque de referencia también crea `Alumno` y `Formador` cuando todavía no existen; comprueba con `RoleManager.RoleExistsAsync` para no duplicarlos en cada arranque.

Estado teórico 2 — registrar no significa autenticar. Este es el manejador real de registro:

```
// P4.5.4: Identity aplica hash de contraseñas; no las almacenamos manualmente.
app.MapPost("/auth/registro", async (RegistrarDto entrada,
    UserManager<IdentityUser> usuarios) =>
{
    if (string.IsNullOrWhiteSpace(entrada.Email) ||
        string.IsNullOrWhiteSpace(entrada.Password))
        return Results.BadRequest("Correo y contraseña obligatorios.");

    var nuevo = new IdentityUser
    {
        UserName = entrada.Email.Trim(),
        Email = entrada.Email.Trim()
    };
    var resultado = await usuarios.CreateAsync(nuevo, entrada.Password);
    if (!resultado.Succeeded)
        return Results.ValidationProblem(
            resultado.Errors.GroupBy(e => e.Code)
                .ToDictionary(g => g.Key, g => g.Select(e => e.Description).ToArray()));

    var asignacion = await usuarios.AddToRoleAsync(nuevo, "Alumno");
    if (!asignacion.Succeeded)
        return Results.Problem("No se pudo configurar el rol del usuario.");

    return Results.Created("/api/identidad/yo", new
    {
        id = nuevo.Id,
        email = nuevo.Email,
        rol = "Alumno"
    });
});
```

Al recibir JSON, el servidor construye `RegistrarDto` y verifica que existan correo y contraseña. Crea `IdentityUser` y llama a `UserManager.CreateAsync`; Identity aplica sus reglas y guarda un hash de contraseña en LocalDB. Los errores se convierten en una respuesta 400 estructurada. Si todo va bien, `AddToRoleAsync` concede **solo el rol Alumno**, y el servidor responde 201 sin incluir la contraseña. El resultado de registrarse no es todavía un JWT: el cliente debe demostrar posteriormente que conoce la contraseña.

La distinción también protege contra una escalada de permisos. No existe una propiedad `Rol` en el DTO que el cliente pueda usar para hacerse Formador. La cuenta Formador se prepara únicamente en una inicialización docente controlada fuera del endpoint público.

Estado teórico 3 — de una contraseña verificada a un JWT. El acceso se implementa con la siguiente operación real:

```
// P4.5.5: comprueba contraseñas y bloqueo mediante SignInManager.
app.MapPost("/auth/login", async (AccederDto entrada,
    UserManager<IdentityUser> usuarios,
    SignInManager<IdentityUser> acceso) =>
{
    var usuario = await usuarios.FindByEmailAsync(entrada.Email ?? "");
    if (usuario is null)
        return Results.Unauthorized();

    var resultado = await acceso.CheckPasswordSignInAsync(
        usuario, entrada.Password ?? "", lockoutOnFailure: true);
    if (!resultado.Succeeded)
        return Results.Unauthorized();

    var roles = await usuarios.GetRolesAsync(usuario);
    var ahora = DateTime.UtcNow;
    var caducidad = ahora.AddMinutes(15);
    var claims = new List<Claim>
    {
        new(JwtRegisteredClaimNames.Sub, usuario.Id),
        new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString("N")),
        new(JwtRegisteredClaimNames.Iat,
            DateTimeOffset.UtcNow.ToUnixTimeSeconds().ToString(),
            ClaimValueTypes.Integer64),
        new(ClaimTypes.NameIdentifier, usuario.Id),
        new(ClaimTypes.Email, usuario.Email ?? "")
    };
    claims.AddRange(roles.Select(rol => new Claim(ClaimTypes.Role, rol)));

    var token = new JwtSecurityToken(
        issuer: emisor,
        audience: audiencia,
        claims: claims,
        notBefore: ahora,
        expires: caducidad,
        signingCredentials: new SigningCredentials(
            new SymmetricSecurityKey(bytesFirma),
            SecurityAlgorithms.HmacSha256));

    return Results.Ok(new
    {
        accessToken = new JwtSecurityTokenHandler().WriteToken(token),
        tokenType = "Bearer",
        expiresAtUtc = caducidad
    });
});
```

Comenzamos buscando el usuario; si no existe se devuelve 401. `CheckPasswordSignInAsync` comprueba la credencial, incluido el mecanismo de bloqueo activado en esta demostración. Si falla, tampoco se emite token. Solo después se obtienen los roles reales desde SQL Server y se construye una lista de claims. La firma HMAC se calcula con la clave local cargada por configuración; el `JwtSecurityToken` contiene emisor, audiencia, instante inicial y expiración a quince minutos.

No se guarda el JWT en la tabla de usuarios ni se utiliza como contraseña. Una vez emitido, funciona como una credencial que el cliente debe presentar expresamente. `WriteToken` serializa el

objeto firmado; el consumidor recibe `accessToken`, `tokenType` y `expiresAtUtc`. No se registra ni se muestra el valor completo en la interfaz de referencia.

Estado teórico 4 — emitir un JWT no significa aceptarlo sin verificar. Antes de `builder.Build()` registramos la autenticación de la API con el middleware JWT:

```
// P4.5.2: JWT de verdad, validando firma, emisor, audiencia y expiración.
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opciones =>
    {
        opciones.MapInboundClaims = false;
        opciones.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = emisor,
            ValidateAudience = true,
            ValidAudience = audiencia,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(bytesFirma),
            ValidateLifetime = true,
            RequireExpirationTime = true,
            RequireSignedTokens = true,
            ClockSkew = TimeSpan.FromSeconds(5),
            NameClaimType = ClaimTypes.NameIdentifier,
            RoleClaimType = ClaimTypes.Role
        };
    });
builder.Services.AddAuthorization();
```

Obsérvese el orden de las responsabilidades. El servidor ya conoce `emisor`, `audiencia` y `bytesFirma`; el manejador `JwtBearer` necesita esos mismos valores para verificar que recibe un token dirigido a esta API y firmado con la clave de confianza. `ValidateLifetime` exige vigencia temporal. `MapInboundClaims=false` evita transformaciones automáticas de nombres de claims que dificultarían la observación en el laboratorio; el proyecto fija además los tipos usados para nombre y rol.

Es importante registrar autenticación y autorización **antes de construir el host**, y aplicar sus middleware tras `Build`:

```
app.UseAuthentication();
app.UseAuthorization();
```

Si un alumno añade `RequireAuthorization` pero no registra correctamente la autenticación, no habrá un principal autenticado con el que evaluar la política. Si añade un JWT a la respuesta de login pero omite `AddJwtBearer`, la API no estará preparada para validarlo en los recursos de destino.

Estado teórico 5 — proteger rutas por identidad y por rol. Los endpoints reales son:

```
// P4.5.6: sin Bearer válido devuelve 401.
app.MapGet("/api/identidad/yo", (ClaimsPrincipal principal) =>
    Results.Ok(new
    {
        id = principal.FindFirstValue(ClaimTypes.NameIdentifier),
        email = principal.FindFirstValue(ClaimTypes.Email),
        roles = principal.FindAll(ClaimTypes.Role).Select(c => c.Value).ToArray()
    }))
    .RequireAuthorization();
```

```
// P4.5.7: un token de Alumno es auténtico pero carece del rol => 403.
app.MapGet("/api/identidad/formador", () =>
    Results.Ok(new { mensaje = "Recurso reservado a Formador" }))
    .RequireAuthorization(p => p.RequireRole("Formador"));
```

`/api/identidad/yo` exige únicamente autenticación; un JWT de Alumno válido basta para recibir ID, correo y roles. `/api/identidad/formador` exige, además, el rol Formador: el mismo JWT de Alumno obtiene 403, aunque haya superado la validación de firma. Sin cabecera `Authorization: Bearer` la API devuelve 401. Es la comprobación concreta de la diferencia entre *quién es el solicitante* y *qué puede hacer*.

Estado teórico 6 — comprobar que el dato persistido es realmente de Identity. El fichero LocalDB debe contener las tablas del sistema y los usuarios creados. El explorador SQL Server de Visual Studio permite comprobar que `AspNetUsers` identifica los registros y `AspNetUserRoles` los relaciona con `AspNetRoles`. No copiamos hashes de contraseña a los documentos. Repetir el registro del mismo correo debe producir un error; reiniciar la API no elimina los usuarios de LocalDB.

Esto último contrasta con la API de productos del punto 4.3: aquella conserva un diccionario en memoria y reiniciarlo restaura los productos de ejemplo. Identity usa una base persistente, por lo que registrar al alumno una segunda vez no equivale a empezar desde cero.

Estado teórico 7 — integrar un consumidor Blazor sin cambiar la identidad del host. La nueva página `/perfil-jwt` utiliza `IHttpClientFactory` para obtener un cliente llamado `IdentityJwtApi`. Tras llamar a `/auth/login`, envía `Authorization: Bearer` al recurso de perfil. El servidor Blazor que ejecuta el componente y la API de Identity son procesos distintos. La cookie `CookiesDemo` de `/seguridad` no autentica automáticamente a este cliente HTTP; por eso debemos incluir el JWT de forma expresa.

Esta solución separa dos demostraciones, no intenta construir un proveedor de identidades empresarial. Ni la clave local, ni la inicialización de la cuenta docente ni el almacenamiento temporal del token representan una configuración de producción.

Comprobaciones conceptuales que preceden al ejercicio

Estado construido	Evidencia que demuestra	Límite de la evidencia
Contexto y roles en LocalDB	Existen tablas de Identity y los roles iniciales	Aún no hay registro público
POST de registro	Identity valida la contraseña, crea el usuario y asigna Alumno	Registrarse no emite JWT
POST de login	Contraseña correcta produce JWT firmado	Emitir no demuestra que la API lo acepte
Middleware JwtBearer	Firma, emisor, audiencia y vigencia se validan	No reemplaza autorización por roles
Recurso `/yo`	Token válido produce 200; ausencia, 401	No prueba el permiso Formador
Recurso `/formador`	JWT Alumno produce 403; Formador, 200	No implica proteger la API de productos
Página Blazor `/perfil-jwt`	Se envía Bearer desde cliente HTTP y se observan estados	Olvidar el JWT local no revoca tokens

La teoría presenta así las clases, la construcción y los resultados antes de que la práctica proponga escribirlos. Cuando se ejecuten los pasos J3–J6 el estudiante podrá asociar cada fragmento con una responsabilidad ya explicada.

Un contraste práctico con la cookie de Blazor

Aspecto	Cookie de demostración 4.4	JWT de ampliación
Estado de identidad	Principal reconocido por host Blazor	Claims en token firmado validado por API
Transporte habitual	Cookie gestionada por navegador	Cabecera Authorization: Bearer
Credencial de entrada	Código local fijo de laboratorio	Contraseña comprobada por Identity
Persistencia de usuarios	No, identidad construida al entrar	Usuarios y roles en SQL Server Express LocalDB
Autorización	Recurso /demo/privado	Recursos /api/identidad/yo y /formador
Fin de sesión del cliente	SignOutAsync invalida cookie del navegador	El cliente descarta su copia del JWT
Aplicación en producción	No: código fijo de ensayo	No: emisión manual de JWT docente

Los dos enfoques permiten trabajar con ClaimsPrincipal y políticas, pero no se deben mezclar sus mecanismos de transporte. En particular, una cookie válida para el host Blazor no equivale a un JWT que la API externa acepte.

Una arquitectura de demostración con Blazor como consumidor

El proyecto Blazor de 4.4 incorpora una página adicional /perfil-jwt. Esta página utiliza un cliente HttpClient con nombre IdentityJwtApi que apunta a otro puerto. Primero solicita un JWT al endpoint /auth/login y después lo presenta en la cabecera de una consulta:

```
using var solicitud = new HttpRequestMessage(
    HttpMethod.Get, "api/identidad/yo");
solicitud.Headers.Authorization =
    new AuthenticationHeaderValue("Bearer", token);
using var respuesta = await cliente.SendAsync(solicitud);
```

Este ejemplo muestra algo que la integración original de productos no necesitaba: la API requiere una credencial explícita para devolver el perfil. Blazor no recibe permiso simplemente por dibujar una pantalla de acceso. El servidor examina Authorization, valida el token y aplica la política del recurso.

La página guarda el token solamente en el estado temporal del componente Interactive Server y no lo imprime, registra ni coloca en una URL. No utiliza localStorage ni lo incorpora a un formulario de navegación. Es una elección limitada al laboratorio. En producción, el almacenamiento de tokens, los flujos de acceso y la prevención de filtraciones necesitan un diseño conforme al proveedor de identidad y al modelo de ejecución.

Seguridad adicional que el ejemplo no pretende resolver

Aunque utilizamos Identity real y JWT firmado, el laboratorio restringe su ejecución al entorno Development. El registro público y el endpoint de acceso no están diseñados como un servicio de identidad publicado.

No implementamos recuperación de contraseña, confirmación de correo, MFA, gestión de tokens de renovación, revocación, consentimiento, rotación de claves, auditoría avanzada ni una autoridad OIDC. Tampoco trasladamos el JWT a una cookie ni ofrecemos un mecanismo general de sesión persistente para Blazor.

En una aplicación real no se deberían recibir credenciales mediante HTTP sin cifrar. Los comandos del laboratorio usan localhost por claridad, pero todo acceso real con credenciales requiere HTTPS y un proveedor adecuado. Los secretos de firma y las contraseñas de la cuenta docente no deben confirmarse en Git.

El objetivo de esta ampliación es **comprender y ejecutar** el flujo completo Identity → comprobación de contraseña → emisión JWT → validación JwtBearer → autorización por roles, identificando expresamente qué capacidades quedan fuera.

Qué debe poder razonar el alumno

Al terminar, el estudiante debe responder con precisión:

- ¿Qué diferencia existe entre UserManager y JwtBearer? El primero administra usuarios y credenciales; el segundo valida tokens presentados a una API.
- ¿Por qué no basta con decodificar el JWT? Porque hay que verificar la firma, el emisor, la audiencia y el tiempo.
- ¿Por qué se obtiene 401 sin token y 403 con un token de Alumno en el endpoint Formador? Porque autenticación y autorización son comprobaciones distintas.
- ¿Por qué no debe aceptarse un rol enviado por el usuario durante el registro? Porque permitiría escalar privilegios.
- ¿Qué sucede al borrar el token en Blazor? Solo se descarta su copia local; no se revoca automáticamente un JWT ya emitido.
- ¿Puede MapIdentityApi emitir un bearer token que no sea JWT? Sí; no deben confundirse esos formatos.

Estas preguntas sitúan la nueva ampliación dentro del temario de seguridad de ASP.NET Core sin convertir una API didáctica en un servidor de identidad supuestamente preparado para producción.

Taller transversal: una operación vista desde cuatro límites

Este apartado relaciona los cuatro conceptos del módulo mediante un mismo caso: una persona desea crear un producto nuevo llamado «Agenda», con precio 6,40. Seguir la operación de un extremo a otro ayuda a detectar errores que quedan ocultos cuando se estudian las tecnologías como listas de instrucciones.

El límite de la interfaz

En Blazor la persona escribe un nombre y un precio. Estos valores viven inicialmente en el estado del componente. Puede haber un control de formulario que evite enviar campos vacíos, pero no

podemos interpretar esa comprobación como una garantía del servidor: cualquiera que conozca la dirección de la API puede enviar otra petición.

El componente debe gestionar un estado de envío: antes del POST desactiva el botón para evitar pulsaciones duplicadas y después muestra el resultado. Si ocurre un fallo, mantiene los valores para facilitar una corrección. Añadir a la lista un producto antes de recibir confirmación sería una **actualización optimista**; aquí preferimos esperar el producto que devuelve la API, porque contiene el identificador real asignado.

El límite del servicio

El componente no conoce la URL; solo pide al servicio que cree un producto. El servicio transforma la operación de negocio en una petición HTTP, serializando `CrearProductoDto` con `PostAsJsonAsync`. Debe comprobar el estado de respuesta antes de leer el producto creado, porque un 400 representa una operación rechazada.

Este diseño permite cambiar el puerto o la implementación remota sin modificar cada componente. Por el contrario, no elimina las fallas de red: solo ofrece un lugar único para tratar las particularidades de HTTP.

El límite del servidor

La API recibe el JSON, crea un objeto de entrada y comprueba sus reglas. En nuestro laboratorio, un nombre sin contenido o un precio igual o inferior a cero produce 400. Si la entrada es válida, se asigna un ID y la API responde 201 con el producto. En este punto el servidor no conoce componentes Blazor ni `AuthorizeView`; sus reglas funcionan igualmente para otro cliente HTTP.

Si un servidor devuelve 201 pero el cliente pierde la conexión antes de recibirlo, el cliente podría desconocer que el alta se realizó. Por eso el reintento automático de POST no se presenta como una decisión inocua. Un sistema real necesitaría valorar idempotencia, identificadores de petición o reconciliación, pero no añadiremos esa infraestructura al ejercicio.

El límite de la identidad

La demostración de 4.4 protege únicamente `/demo/privado`. Es importante evitar una conclusión falsa: que el usuario haya iniciado sesión en Blazor **no convierte el POST público de productos en una operación protegida**. Las políticas se aplican en cada recurso del servidor correspondiente. El control visual de una pantalla no autoriza ni desautoriza una petición HTTP.

Para ampliar la aplicación en un proyecto futuro, habría que determinar qué operaciones requieren autenticación y qué política aplica a cada una. Ese análisis es distinto de decidir si la interfaz muestra un botón.

Secuencia de diagnóstico de un fallo

Supongamos que «Agenda» no aparece después del envío. Antes de reescribir el componente, realizamos comprobaciones en orden:

1. Comprobar que la API está iniciada y que el GET directo devuelve JSON, diferenciando ausencia de conexión y ausencia de datos.
2. Confirmar que la dirección configurada en `ProductosApi:BaseUrl` y la ruta relativa del servicio componen el destino previsto.
3. Examinar el estado HTTP del POST. Un 400 indica que la API recibió una petición pero rechazó sus datos; un error de conexión indica que no se obtuvo una respuesta HTTP.

4. Si el POST devuelve 201, inspeccionar que el cuerpo puede deserializarse como `ProductoDto` y que el identificador corresponde al devuelto por el servidor.
5. Si el servicio entrega correctamente el DTO pero la pantalla no cambia, revisar la actualización de `productos` y los estados visuales en el componente.

Este recorrido clasifica los errores por responsabilidad: presentación, comunicación, contrato o regla del servidor. Cambiar simultáneamente cuatro archivos ante un único fallo dificulta aprender qué ocurrió.

Dos pruebas que parecen iguales y no lo son

Prueba A: detener la API y abrir `/catalogo-remoto`. Se espera un mensaje de conectividad; no un 404, porque la conexión no ha llegado al endpoint.

Prueba B: mantener la API funcionando y cambiar temporalmente la ruta a `api/no-existe`. El servidor sí responde, pero lo hace con 404. El componente puede mostrar un texto parecido al de A si la excepción se trata de manera general; observar el estado HTTP permite distinguir los casos.

Este contraste justifica el reto del punto 4.1 y sirve para decidir si una aplicación necesita mensajes más específicos sin llenar la pantalla de detalles técnicos.

Cómo razonar sobre una respuesta parcialmente correcta

Imaginemos una API que responde 200 con `{"cantidad":2,"precioTotal":3.30}`. El JSON es válido y la respuesta es satisfactoria, pero **no corresponde a una lista de `ProductoDto`**. Si el servicio de catálogo intenta leerlo como lista, fallará el contrato. En 4.3 añadimos el resumen como ruta diferenciada precisamente para no alterar el cuerpo que espera el cliente existente.

La compatibilidad de contratos es uno de los motivos por los que una API no debería cambiar de forma arbitraria el tipo de respuesta de un endpoint ya consumido.

Actividad razonada antes de practicar

Sin escribir todavía código, responde: ¿qué componente conoce el puerto 5081? ¿Qué capa debe saber que existe `api/productos`? ¿Quién asigna el identificador? ¿Dónde se comprueba el rol `Formador` para `/demo/privado`? Si las respuestas sitúan el puerto en configuración/cliente HTTP, la ruta en servicio/API, el ID en la API y la autorización en el servidor, la separación de responsabilidades está bien comprendida.

Estas decisiones serán observables en los proyectos de 4.1–4.4 y no dependen de nombres de clases memorísticos: derivan del recorrido completo de la petición.

Conclusión del módulo

4.1 enseña a consumir HTTP; 4.2 separa interfaz y servicios; 4.3 explica el servidor; 4.4 distingue identidad, decisión de acceso y presentación condicional. Como ampliación autónoma, 4.4.6 incorpora ASP.NET Core Identity real, persistencia EF Core y JWT firmados con validación y políticas de rol. La jornada presencial sigue siendo de seis horas.